Developing an MLOps Pipeline for Continuous Mental Health Monitoring Using Wearable Sensor Data

Master's Thesis

submitted in partial fulfillment of the requirements for the degree of

Master of Science in Mechatronics

submitted by

Shalin Vachheta

Matriculation Number: 1701359

Faculty of Engineering

Chair of Embedded Systems

University of Siegen

First Supervisor: Prof. Dr.-Ing. Roman Obermaisser
Second Examiner: M.Sc. Ugonna Oleh

Siegen, 30 June 2026

Abstract

This thesis develops an offline-first MLOps pipeline for wearable-sensor-based behavioral-proxy recognition in a mental-health-adjacent monitoring context. The work focuses on wrist-worn inertial measurement unit data and treats anxiety-related repetitive activities as observable activity classes rather than as clinical mental-health labels. The technical contribution is therefore not a diagnostic or prognosis model, but a reproducible and governed machine-learning lifecycle for wearable Human Activity Recognition.

The pipeline connects ingestion, validation, preprocessing, inference, evaluation, monitoring, trigger evaluation, and governed retraining boundaries into one auditable workflow. A central design principle is the runtime contract: the model, sensor schema, unit convention, scaler, label order, monitoring baseline, and evaluation evidence are treated as one deployment unit. This prevents silent train–serve mismatch, such as using a model with an incompatible scaler, channel order, or baseline.

The final runtime candidate uses a 1D-CNN-BiLSTM model for eleven-class wrist-motion activity recognition from 50 Hz IMU windows of shape $(200, 6)$. The system is evaluated under labeled development data, held-out-user conditions, a blind self-recorded User 7 probe, and deployment-style unlabeled inference. The thesis shows that reliable wearable monitoring requires more than model accuracy: it requires explicit preprocessing assumptions, run-scoped evidence, monitoring signals, and governance rules that separate operational risk detection from automatic model mutation.

Keywords: wearable HAR, MLOps, offline-first pipeline, distribution shift, retraining governance, train–serve consistency, trigger policy

Zusammenfassung

Diese Masterarbeit entwickelt eine offline-first MLOps-Pipeline für die Erkennung von beobachtbaren Verhaltensmustern mit tragbaren Sensordaten. Der Anwendungskontext ist mentales Gesundheitsmonitoring. Die Arbeit macht jedoch keine klinische Diagnose und keine Prognose. Stattdessen werden wiederholte Hand- und Handgelenkbewegungen, zum Beispiel Nagelkauen, Haarezupfen, Kratzen, Tippen oder Reiben, als Aktivitätsklassen behandelt.

Die Pipeline verarbeitet Sensordaten einer Smartwatch mit Beschleunigungs- und Gyroskopsignalen. Sie umfasst Datenaufnahme, Validierung, Vorverarbeitung, Inferenz, Evaluation, Monitoring, Trigger-Entscheidung und kontrollierte Grenzen für erneutes Training. Ein wichtiger Beitrag ist der sogenannte Runtime Contract. Dabei werden Modell, Sensor-Schema, Einheiten, Skalierer, Label-Reihenfolge, Monitoring-Basislinie und Evidenz gemeinsam betrachtet. Dadurch sollen stille Fehler zwischen Training und Anwendung vermieden werden.

Das finale Laufzeitmodell ist ein 1D-CNN-BiLSTM-Modell für elf Aktivitätsklassen. Es nutzt IMU-Fenster mit der Form $(200, 6)$ bei einer Abtastrate von 50 Hz. Die Arbeit zeigt, dass ein tragbares Monitoring-System nicht nur ein trainiertes Modell braucht. Es braucht auch klare Vorverarbeitung, reproduzierbare Artefakte, Monitoring-Signale und Governance-Regeln, damit Warnungen nicht automatisch zu unsicherem Modelltraining führen.

Eidesstattliche Erklärung (Declaration of Authorship)

Ich erkläre hiermit an Eides statt, dass ich die vorliegende Masterarbeit selbstständig und ohne Benutzung anderer als der angegebenen Hilfsmittel angefertigt habe. Die aus fremden Quellen direkt oder indirekt übernommenen Gedanken sind als solche kenntlich gemacht. Die Arbeit wurde bisher in gleicher oder ähnlicher Form keiner anderen Prüfungsbehörde vorgelegt und auch noch nicht veröffentlicht.

I hereby declare, to the best of my knowledge and belief, that this thesis is my own work. All sources used have been cited and attributed. This work has not been submitted to any other examination authority in the same or similar form, nor has it been published.

Siegen, 30 June 2026

Shalin Vachheta

Contents

Abstract **I**

Zusammenfassung **II**

Eidesstattliche Erklärung **III**

List of Figures **VIII**

List of Tables **X**

List of Abbreviations **XII**

1 Introduction **1**

1.1 Motivation and Context 1

1.2 Problem Statement 2

1.3 Research Objectives and Questions 3

1.4 Contributions 4

1.5 Scope and Reproducibility Boundary 4

1.6 Thesis Structure 5

2 Background and Related Work **6**

2.1 Mental Health Monitoring and Wearable Sensing 6

2.2 Anxiety-Related Behavioral Proxies and Human Activity Recognition . . . 7

2.3 Deep Learning for Wearable Sensor-Based HAR 9

2.4 Datasets, Wearables, and Domain Shift 11

2.5 MLOps, Reproducibility, and Offline-First ML Pipelines 13

2.6 Monitoring, Drift, and Confidence Limits 16

2.7 Retraining Governance, Ethical Boundaries, and Research Gap 18

3 Offline-First MLOps System Architecture **21**

3.1 Architectural Goals and Constraints . . . 21
3.2 Pipeline Stages and Execution Views . . . 22
3.3 The Operational Assessment Path: Stages 1–7 . . . 23
3.4 The Governed Mutation Path: Stages 8–10 . . . 25
3.5 Artifact Lineage and Runtime Contract . . . 26
3.5.1 Runtime Contract as a Same-Folder Invariant . . . 26
3.6 Scope Boundary . . . 28
3.7 Chapter Summary . . . 28

4 Data Sources, Preprocessing, and Model Contract 29
4.1 Data Sources and Evidence Roles . . . 29
4.2 Sensor Schema, Units, and Label Space . . . 32
4.3 Windowing and Tensor Representation . . . 33
4.4 Model Family and Proposed Architecture . . . 35
4.5 Runtime Data and Model Contract . . . 36
4.6 Chapter Summary . . . 38

5 Pipeline Implementation 39
5.1 Repository Layout and Execution Entry Point . . . 39
5.2 Technology and Design Rationale . . . 39
5.3 Configuration Layering . . . 40
5.4 Runtime Contract Implementation and Artifact Loading . . . 41
5.4.1 Same-Folder Invariant Enforcement . . . 42
5.5 Stage Execution and Artifact Trail . . . 42
5.5.1 Run-Scoped Artifact Trail . . . 43
5.6 Interfaces, Reproducibility, and Maturity . . . 44
5.6.1 Experiment Tracking Across the Two Execution Paths . . . 45
5.6.2 Reproducibility and Environment Control . . . 46
5.6.3 MLOps Maturity Position . . . 47
5.7 Chapter Summary . . . 47

6 Monitoring and Governance 48
6.1 Why Monitoring Is Needed Under Label-Scarce Operation . . . 48
6.2 Post-Inference Monitoring . . . 48
6.2.1 Confidence Limits and the Calibration Gap . . . 51
6.3 Trigger Policy and Governance . . . 51

6.3.1 Collapse Guard and Cooldown Semantics . . . 55
6.3.2 Severity Is Not Retraining: The Governance Boundary . . . 55
6.3.3 Retraining and Baseline Governance . . . 56
6.4 Evaluation Metrics: Labeled and Unlabeled Conditions . . . 56
6.5 Threshold Calibration and Drift Auditing . . . 57
6.5.1 Z-Score Severity Levels and Their Statistical Basis . . . 57
6.5.2 Distribution-Sensitive Drift Audit . . . 59
6.6 Chapter Summary . . . 59

7 Experiments and Evaluation 60
7.1 Evaluation Setup and Evidence Boundary . . . 60
7.1.1 Evidence Boundary . . . 60
7.1.2 Fixed Runtime Contract and Evaluation Protocols . . . 61
7.1.3 Evaluation Metrics . . . 61
7.2 Recognition Performance: Within-Distribution and Blind Probe . . . 61
7.2.1 Within-Distribution Five-Fold Cross-Validation . . . 62
7.2.2 Blind User 7 External Probe . . . 62
7.3 Calibration and the High-Confidence Failure Mode . . . 63
7.4 User-7 Proxy Metrics: What the Pipeline Sees Without Labels . . . 66
7.5 Monitoring and Trigger-Policy Evidence . . . 69
7.5.1 Session-Level Monitoring Outcomes . . . 69
7.5.2 Trigger-Policy Calibration . . . 72
7.5.3 Preprocessing and Contract Sensitivity . . . 73
7.6 Synthesis and Implications . . . 74
7.7 Chapter Summary . . . 74

8 Discussion and Conclusion 76
8.1 Findings and Answers to the Research Questions . . . 76
8.2 Interpretation and Implications for Wearable Mental-Health-Adjacent MLOps 77
8.3 Limitations and Threats to Validity . . . 78
8.4 Future Work . . . 79
8.5 Conclusion . . . 80

A Reproducible Commands and Runbook 88

B Runtime Contract and Artifact Inventory 90

C Experimental Evidence and Result Artifacts **92**

D Supplementary Result Figures **94**

E System Walkthrough: Web UI and Experiment Tracking **96**

E.1 Serving Path on a Labeled Session 96

E.2 Serving Path on an Unlabeled Session 98

E.3 Experiment Tracking 101

E.3.1 Inference-Production Experiment (Serving Path) 101

E.3.2 Har-Production-Pipeline Experiment (Batch Path) 104

List of Figures

2.1 Conceptual framing of wearable mental-health-adjacent monitoring... . . . 7
2.2 Sensor-based activity recognition using deep-learning approaches 10
2.3 Lab-to-life and train–serve mismatch in wearable HAR 12
2.4 MLOps as the intersection of machine learning, data engineering,... 14
2.5 High-level MLOps lifecycle across data, machine-learning, and... 15

3.1 Offline-first MLOps architecture of the defended pipeline 23
3.2 Deep-learning pipeline adopted in this thesis 24
3.3 Runtime contract package for the proposed model 27

4.1 Per-source channel mean and standard deviation for the six IMU... 30
4.2 Accelerometer and gyroscope signal-magnitude distributions by... 31
4.3 Window count per activity class for Garmin users 1–6 (3,755... 32
4.4 Preprocessing contract from raw wearable recordings to model-ready... . . . 33
4.5 Block-level architecture of the proposed model 36

5.1 Run-scoped artifact trail produced by the offline-first pipeline 44

6.1 Tiered thresholds for the three trigger-voting signals 53
6.2 Post-inference monitoring and governance flow 54

7.1 User-7 proxy monitoring signals: confidence, per-activity proxy accuracy, temporal stability 64
7.2 Per-class confusion matrix on the blind User 7 probe 66
7.3 User-7 softmax confidence and low-confidence window rate 67
7.4 Effect of majority-vote temporal smoothing on the User-7 session 68
7.5 Mean prediction confidence across the 25 unlabeled sessions 70
7.6 Temporal transition rate across the 25 unlabeled sessions: raw vs smoothed 70
7.7 Input drift z-score across the 25 unlabeled sessions 71

7.8 Dominant-class share across the 25 unlabeled sessions 71

D.1 Per-fold accuracy under five-fold cross-validation on Garmin... 94
D.2 Per-class F1 on the blind User 7 probe 95

E.1 Serving path on a labeled session (User 7), part 1 97
E.1 Serving path on a labeled session (User 7), part 2 98
E.2 Serving path on an unlabeled session, part 1 99
E.2 Serving path on an unlabeled session, part 2 100
E.3 MLflow experiments for the two execution paths 101
E.4 Inference-production experiment, part 1 102
E.4 Inference-production experiment, part 2 103
E.5 Har-production-pipeline experiment, part 1 104
E.5 Har-production-pipeline experiment, part 2 105

List of Tables

2.1 Model-centric machine learning versus an MLOps lifecycle view. 14
2.2 Synthesis of related work and the remaining gap addressed by this... 20

3.1 Comparison between standard production-oriented MLOps and the... . . . 22
3.2 Summary of the architecture stages and whether they are allowed to... . . . 25
3.3 Runtime-contract consistency rules for the same-folder invariant. 27

4.1 Data sources, evidence roles, and data scope 29
4.2 Per-source channel statistics (mean ± standard deviation) for... 30
4.3 Fixed activity-label order used by the proposed model's runtime contract. . 34
4.4 Windowing and tensor contract used by the proposed model. 34
4.5 Block-level summary of the proposed model's architecture. 35
4.6 Runtime data and model contract for the thesis pipeline. 37

5.1 Main technology and design choices and their rationale. 40
5.2 Configuration source precedence in the pipeline implementation. 41
5.3 Mapping between architecture stages and repository implementation... . . . 43
5.4 MLOps maturity position of the defended thesis pipeline. 47

6.1 Stage 6 monitoring signals and their conservative operating thresholds . . . 49
6.2 Stage 7 trigger decisions under the no-auto-retrain policy 52
6.3 Labeled evaluation metrics versus unlabeled proxy monitoring metrics . . . 56
6.4 Z-score severity levels and their statistical basis. 58

7.1 Five-fold window-level cross-validation on Garmin users 1–6 (751... 62
7.2 Decomposition of trigger actions and monitoring-signal incidence... 72
7.3 Trigger-policy calibration against sixteen reference risk episodes... 73
7.4 Contract-sensitivity ablation over window size and overlap... 73

8.1 Research questions, the evidence used, and the bounded answer. 77

B.1 Contents of the deployed proposed-model runtime contract. 90

C.1 Chapter 7 results and their source artifacts. 93

List of Abbreviations

HAR Human Activity Recognition

IMU Inertial Measurement Unit

MLOps Machine Learning Operations

CNN Convolutional Neural Network

LSTM Long Short-Term Memory

BiLSTM Bidirectional Long Short-Term Memory

FAR False Alarm Rate

ECE Expected Calibration Error

MMD Maximum Mean Discrepancy

DVC Data Version Control

CI/CD Continuous Integration / Continuous Delivery

YAML YAML Ain't Markup Language

RQ Research Question

Chapter 1

Introduction

1.1 Motivation and Context

Mental health monitoring is difficult to perform continuously in everyday life. Traditional assessment methods such as clinical interviews, questionnaires, and self-reports provide important clinical information, but they are often episodic, retrospective, and dependent on patient recall. Wearable and mobile sensing technologies offer a complementary path by capturing objective, timestamped physiological and behavioral signals during daily life [1, 2, 3, 4]. This makes wearable sensing relevant for continuous mental-health-adjacent monitoring, especially when the aim is to observe behavioral or physiological patterns over time rather than to replace clinical assessment.

Wrist-worn consumer devices are particularly relevant because they are unobtrusive and already widely used. Their inertial measurement units (IMUs), especially accelerometers and gyroscopes, can capture linear and rotational wrist motion. These signals are suitable for Human Activity Recognition (HAR), where short temporal windows of sensor data are mapped to activity classes [5, 6]. In the mental-health context, HAR is useful when the target is not an internal mental state itself, but observable behavioral proxies (externally visible actions that serve as indirect, measurable stand-ins for an internal state that cannot be sensed directly), such as nail biting, hair pulling, hand scratching, hand tapping, rubbing, and other repetitive body-focused movements. Prior work on ADAMSense and related anxiety-displaying activities has shown that such behaviors can be recognized from wearable motion signals under supervised conditions [7]. A recent Garmin-based study further demonstrates the feasibility of adapting a 1D-CNN-BiLSTM HAR model to commercial smartwatch sensor data for anxiety-related activity recognition [8].

This thesis builds on that direction, but it deliberately narrows the technical claim. The system developed here does not diagnose anxiety, depression, panic disorder, or any other mental health condition. Instead, it treats anxiety-related repetitive activities as observable behavioral proxies within a wearable monitoring pipeline. The technical task is therefore wrist-worn IMU-based HAR, while the broader application context is continuous mental-health-adjacent monitoring. This distinction is essential because recognizing a behavior from sensor data is not equivalent to clinically interpreting a mental state.

In this thesis, the phrase continuous mental health monitoring is therefore used as the application context, not as a claim of clinical diagnosis or prognosis. The implemented system monitors observable wrist-motion activity patterns that may be relevant for later mental-health-related interpretation, especially behaviors such as nail biting, hair pulling, hand scratching, hand tapping, and rubbing. Clinical interpretation, diagnosis, prognosis, and treatment recommendation remain outside the defended technical scope.

A further challenge is that a trained classifier alone is not sufficient for a trustworthy monitoring system. When a model leaves the training environment and is applied to deployment-style wearable data, its behavior can change because of user variability, sensor placement, unit differences, preprocessing mismatch, or broader distribution shift [9, 10, 11, 12]. In such settings, ground-truth labels are often unavailable at inference time, and degradation can remain hidden if the surrounding system only produces predictions without monitoring or governance. One possible failure mode is dominant-class collapse, where a model assigns most windows to one class, sometimes with misleading confidence. This is especially important in health-adjacent settings, where overconfident or poorly governed model behavior may lead to misleading interpretation.

This thesis addresses the problem from an MLOps perspective. The goal is to develop an offline-first, auditable, and reproducible pipeline in which wearable sensor data ingestion, validation, transformation, inference, evaluation, monitoring, trigger evaluation, and controlled retraining boundaries are treated as one coordinated lifecycle rather than as disconnected scripts. This follows the broader MLOps argument that reliable machine learning systems require lifecycle engineering around data, code, models, monitoring, automation, artifact traceability, and operational decision points, not only a trained model artifact [13, 14, 15, 16]. The contribution of this thesis is therefore a governed MLOps pipeline prototype that supports mental-health-adjacent monitoring through wearable-sensor-based behavioral-proxy recognition, not a clinical diagnostic or prognosis model.

1.2 Problem Statement

This thesis addresses a problem at the intersection of wearable sensing, mental-health-adjacent behavior recognition, and MLOps.

The first part of the problem is technical. A wearable HAR model trained under controlled conditions may not behave reliably when applied to new sessions, new users, or different preprocessing conditions. Even when the software executes successfully and the input tensor has the expected shape, the effective meaning of the data may have changed because of unit conversion errors, sensor placement variation, sampling irregularities, normalization mismatch, gravity handling, calibration choices, or domain shift [11, 10, 8]. In a wrist-worn monitoring setting, these factors are not minor details. They define the numerical and temporal contract under which the model operates.

The second part of the problem is operational. In continuous or deployment-style wearable monitoring, labels are usually unavailable at inference time. Direct metrics such

as accuracy, recall, macro F1, or confusion matrices cannot be computed immediately on incoming sessions. The system must therefore rely on proxy signals such as confidence, uncertainty, temporal stability, dominant-class concentration, and drift indicators. However, such proxy signals indicate operational risk rather than verified correctness. Modern neural networks can be overconfident, and confidence alone should not be interpreted as reliable proof of prediction quality [17, 18].

The third part of the problem is governance. If a monitoring signal becomes suspicious, the system still needs a safe decision process. Blind automatic retraining from unlabeled, shifted, or collapsed predictions can amplify errors instead of improving the model. This is especially problematic in a mental-health-adjacent setting, where model outputs may later influence interpretation, reporting, or human review. The pipeline must therefore separate monitoring severity from model mutation: a risky batch may require review without automatically triggering retraining.

The central problem is therefore how to operate such a classifier inside a reproducible pipeline that preserves its preprocessing contract, records monitoring evidence, and routes risky sessions to review instead of automatic retraining.

1.3 Research Objectives and Questions

The work is organized around five practical research objectives:

1. to design and implement an offline-first MLOps pipeline that supports mental-health-adjacent monitoring through wearable-sensor-based behavioral-proxy recognition;
2. to define a reproducible data and model contract for recognizing anxiety-related behavioral proxies from wrist-worn IMU data;
3. to investigate train–serve mismatch and preprocessing sensitivity under deployment-style wearable conditions;
4. to evaluate monitoring and trigger-policy logic under label-scarce inference conditions;
5. to establish clear scope boundaries around clinical validity, automated diagnosis, unrestricted deployment, and automatic retraining.

These objectives lead to the following research questions:

1. **RQ1:** Can an offline-first MLOps pipeline support mental-health-adjacent wearable monitoring by detecting operational risk in behavioral-proxy recognition when labels are unavailable at inference time?
2. **RQ2:** How do preprocessing and train–serve contract choices affect recognition behavior for wrist-worn anxiety-related activity signals?
3. **RQ3:** Can a governed trigger policy support safer monitoring and retraining decisions than unrestricted automatic retraining?

4. **RQ4:** To what extent are observed collapse or degradation patterns caused by model-contract or model-choice issues rather than by input data alone?

The purpose of these questions is not to claim a clinically validated mental health monitoring system. Rather, they define a bounded engineering evaluation of whether a wearable sensor model can be embedded in a reproducible, monitored, and governance-aware MLOps lifecycle.

1.4 Contributions

This thesis makes five main contributions.

1. It develops an offline-first, stage-based MLOps pipeline for wearable-sensor-based mental-health-adjacent behavior monitoring. The pipeline connects data ingestion, validation, transformation, inference, evaluation, monitoring, trigger evaluation, and governed retraining boundaries into one reproducible workflow.

2. It defines and documents a reproducible runtime contract for wrist-worn IMU-based HAR, including unit handling, sampling rate, window size, stride, channel order, normalization policy, model input shape, and fixed label order.

3. It treats train–serve consistency as a runtime-contract problem: unit handling, scaler use, channel order, windowing, and label decoding must remain tied to the promoted model.

4. It implements and evaluates a monitoring and trigger-governance layer that interprets post-inference risk signals and decides whether a session should be logged, reviewed, or queued without allowing automatic mutation from unlabeled data.

5. It provides a bounded mental-health-adjacent framing for wearable HAR. The system recognizes observable anxiety-related behavioral proxies from sensor data, but it does not claim diagnosis, prognosis, treatment recommendation, or population-level clinical validity.

The contributions are, in aggregate, an engineering and systems contribution around wearable mental-health-adjacent monitoring. The novelty lies not in proposing a new neural architecture, but in building and evaluating a reproducible, monitored, and governed lifecycle around a wearable HAR model.

1.5 Scope and Reproducibility Boundary

To keep all claims aligned with executable code and available evidence, this thesis fixes its defended scope to the implemented offline-first runtime path. In this thesis, offline-first means that the complete pipeline can be executed and inspected on a local machine using

versioned code, configuration files, model artifacts, and stored input data. It does not mean that the system is deployed on-device on the wearable, clinically integrated into a care pathway, or operated as a real-time cloud service; these remain outside the present scope and are revisited as directions for future work in Chapter 8, Section 8.4.

- **Core operational path:** stages 1–7, covering ingestion, validation, transformation, inference, evaluation, monitoring, and trigger evaluation.
- **Governed mutation path:** stages 8–10, covering retraining, model registration, and baseline update. These stages exist as a conditional mutation path and are not executed automatically in the defended runtime.
- **Optional supplementary analysis:** calibration and additional drift-audit methods are treated as experimental or future-work extensions rather than as default runtime requirements.

1.6 Thesis Structure

The remainder of this thesis is organized as follows. Chapter 2 reviews the background and related work needed to position the thesis, including wearable mental-health monitoring, anxiety-related behavioral proxies, wearable HAR, train–serve shift, and MLOps governance. Chapter 3 presents the offline-first system architecture and explains the separation between the seven-stage operational assessment path and the conditional mutation path. Chapter 4 defines the data sources, preprocessing decisions, windowing logic, and runtime model contract. Chapter 5 explains how the architecture and model contract are implemented in the repository pipeline. Chapter 6 describes the monitoring, drift, trigger-policy, and retraining-governance logic. Chapter 7 reports the experimental evidence, including preprocessing sensitivity, trigger-policy evaluation, model-contract diagnostics, and negative results. Chapter 8 discusses the findings, answers the research questions, states limitations, and outlines future work.

Chapter 2

Background and Related Work

2.1 Mental Health Monitoring and Wearable Sensing

Mental health monitoring is difficult to perform continuously because many established assessment methods are episodic, retrospective, and dependent on active participation. Clinical interviews, questionnaires, and self-reports remain essential for diagnosis and care, but they usually capture a limited point in time rather than the continuous behavioral variation that occurs in everyday life. This limitation has motivated growing interest in wearable and mobile sensing as a complementary source of objective, timestamped data for mental-health-adjacent monitoring [1, 2, 19, 20, 21, 22].

Wearable devices are relevant in this context because they can collect signals during daily activity with relatively low user burden. Depending on the device, these signals may include motion, heart rate, sleep-related features, physiological variability, or other passive sensing streams. In this thesis, the focus is specifically on wrist-worn inertial measurement unit (IMU) data, especially accelerometer and gyroscope signals. Accelerometers capture linear acceleration and movement intensity, while gyroscopes capture angular velocity and rotational wrist motion. Together, these signals provide a practical basis for recognizing repeated hand and wrist movements that may be behaviorally relevant in anxiety- or stress-related contexts [5, 6].

The mental-health relevance of wearable sensing must be framed carefully. Wearable data can support longitudinal observation and may reveal behavioral or physiological patterns that are difficult to capture through retrospective reports alone, but sensor data do not directly provide a clinical diagnosis. A smartwatch cannot infer anxiety, depression, or distress simply from motion signals without clinical context and validation. Therefore, this thesis treats wearable sensing as a support mechanism for mental-health-adjacent monitoring rather than as a diagnostic instrument.

In line with the title of this thesis, the phrase continuous mental health monitoring is used here as the broader application context. The implemented technical system does not infer a clinical mental-health state directly. Instead, it focuses on wearable-sensor-based recognition of observable wrist-motion behaviors that may later support mental-health-related interpretation under human or clinical review.

This distinction shapes the thesis design. Instead of attempting to infer an internal mental state directly, the system operationalizes the monitoring problem through observable behaviors. Repetitive behaviors such as nail biting, hair pulling, hand scratching, hand tapping, and rubbing can appear in anxiety- or stress-related contexts and can also be represented as wrist-motion patterns. This makes the problem technically suitable for sensor-based Human Activity Recognition (HAR), while the broader application motivation remains continuous mental-health-adjacent monitoring.

The literature also shows that wearable mental-health monitoring is heterogeneous. Some studies focus on physiological anxiety or stress detection, some on mobile passive sensing, some on longitudinal symptom prediction, and others on behavior recognition from wearable motion signals [1, 2, 23, 19, 24]. This heterogeneity is one reason why the scope of the present thesis is deliberately bounded. The thesis does not claim a clinically validated mental-health monitoring system. It studies how wearable sensor data can be processed, monitored, and governed in an offline-first MLOps pipeline when the observable task is anxiety-related behavior recognition.

Figure 2.1 summarizes this bounded framing.

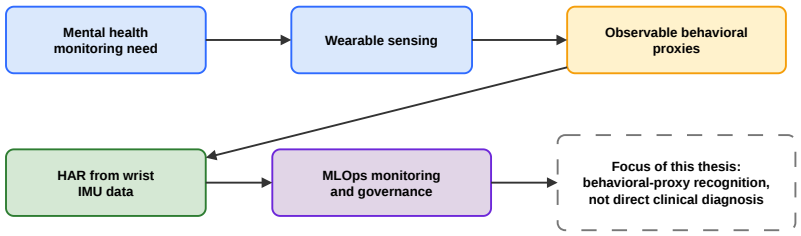


Figure 2.1: Conceptual framing of wearable mental-health-adjacent monitoring in this thesis. The system uses wearable sensing to recognize observable behavioral proxies through HAR, rather than directly diagnosing mental health conditions.

2.2 Anxiety-Related Behavioral Proxies and Human Activity Recognition

The previous section established wearable sensing as a possible source of continuous mental-health-adjacent information. The next question is how this broad monitoring idea can be translated into a concrete machine learning task. In this thesis, the task is not direct mental-state inference. Instead, the monitoring problem is operationalized through observable anxiety-related behavioral proxies. A behavioral proxy is an externally observable action that serves as an indirect, measurable stand-in for an internal state that cannot be sensed directly; in this thesis these proxies are short, repetitive wrist and hand movements that may appear in anxiety- or stress-related contexts, but they are not clinical labels by themselves.

Examples of such behaviors include nail biting, hair pulling, hand scratching, hand tapping, rubbing, and similar repetitive body-focused movements. These behaviors are relevant to this thesis because they are expressed through short, repeated wrist and hand

motions. A wrist-worn IMU can capture part of this motion pattern through acceleration and angular velocity signals. Therefore, the technical problem can be framed as sensor-based Human Activity Recognition (HAR), where temporal windows of wearable sensor data are assigned to predefined activity classes [5, 6, 25].

Recognizing a behavior is not the same as diagnosing anxiety. A model may classify a window as nail biting or hand tapping, but that prediction does not establish the user's mental state, the clinical meaning of the behavior, or the reason why the behavior occurred. The same observable activity can appear in different contexts and can have different interpretations across individuals. For this reason, the behavioral labels used in this thesis are treated as activity labels, not as diagnostic labels.

Prior work supports the feasibility of recognizing anxiety-related or anxiety-displaying activities from wrist-worn motion signals. The ADAMSense study showed that activities such as nail biting, hair pulling, scratching, and tapping can be modeled from wearable sensor data using supervised learning [7, 26]. The Garmin-based study used as a foundation for this thesis further shows that anxiety-related activities can be studied with commercial smartwatch sensor data and sequence-aware deep learning models [8]. These works justify the use of HAR as the recognition layer for the present thesis.

However, anxiety-related HAR is more difficult than coarse activity recognition such as walking, sitting, or standing. Many of the target activities are fine-grained, short, repetitive, and partially overlapping in their wrist-motion signatures. For example, hand scratching and rubbing may share similar gross movement intensity, while nail biting and hair pulling may depend on subtle wrist orientation and transition patterns. Recognition performance can therefore depend strongly on windowing, sensor placement, sampling rate, preprocessing, user movement style, and class definition.

This motivates the need for a pipeline view rather than a model-only view. If anxiety-related behaviors are represented as HAR classes, then the full runtime contract becomes important: the same sensor channels, unit conventions, preprocessing decisions, window size, stride, label order, and model input shape must be preserved during inference, the stage at which the trained model is applied to new sensor windows to produce predictions. A mismatch in any of these elements can change the meaning of the input window before the model makes a prediction. Therefore, the thesis treats behavioral-proxy recognition as both a HAR problem and an operational MLOps problem.

HAR serves as the technical bridge between wearable sensing and mental-health-adjacent monitoring in this thesis. The behavioral-proxy framing keeps the scope scientifically bounded: the system recognizes observable wrist-movement activities that may be relevant for later interpretation, but it does not infer clinical mental states directly. This distinction is central to the rest of the thesis because it explains why the contribution focuses on reproducible sensing, inference, monitoring, and governance rather than on clinical diagnosis.

2.3 Deep Learning for Wearable Sensor-Based HAR

Wearable sensor-based HAR is commonly formulated as a multivariate time-series classification problem. Instead of classifying individual sensor samples, the continuous IMU stream is divided into temporal windows, and each window is assigned an activity label. This window-based representation is important because human activities are expressed through motion patterns over time rather than through isolated acceleration or gyroscope values. Windowing also makes the learning problem tractable: it converts an open-ended sensor stream into fixed-shape, individually labeled examples that a model can batch, compare, and score consistently. Because the window length, stride, and channel order define the exact tensor the model receives, they are not only preprocessing choices but part of the runtime contract; if inference uses a different window length or stride than training, the model is given inputs with a different temporal meaning even when the raw file format looks unchanged. The choice of sampling rate, window length, overlap, and channel order therefore directly shapes the information available to the model [5, 6, 25].

Traditional HAR pipelines often used hand-crafted temporal and spectral features followed by classical classifiers. Designing such features by hand is labor-intensive and dataset-specific, and it can miss the subtle, fine-grained motion cues that separate visually similar wrist behaviors such as scratching and rubbing. Deep learning changed this workflow by allowing models to learn hierarchical representations directly from raw or lightly processed sensor windows, which reduces the dependence on manual feature engineering and has become the dominant approach in recent sensor-based HAR [27, 28]. This is especially useful for wearable IMU data, where discriminative information may appear as local peaks, periodic patterns, temporal transitions, or combinations of acceleration and angular velocity across multiple axes [29, 30, 31].

Convolutional neural networks (CNNs) are useful in this setting because they can learn local temporal patterns from sensor windows. In one-dimensional sensor signals, 1D convolutions can capture short motion motifs such as repeated tapping, rubbing, or wrist rotations. Recurrent models, especially Long Short-Term Memory (LSTM) networks, are designed to model temporal dependencies across sequences [32]. Bidirectional LSTMs extend this idea by processing the sequence in both forward and backward directions, allowing the model to use temporal context from both sides of a window [33].

Hybrid CNN–LSTM architectures combine these strengths. The convolutional part extracts local motion features, while the recurrent part models their temporal ordering. Such combinations have been widely used in sensor-based HAR because they preserve both local movement structure and longer temporal dependencies [27, 28, 34, 35]. In time-series classification more generally, 1D-CNN models have also been shown to provide strong baselines when applied directly to raw temporal signals [36].

Figure 2.2 summarizes the deep-learning view of sensor-based HAR used as the model basis in this thesis.

J. Wang et al./Pattern Recognition Letters 119 (2019) 3–11

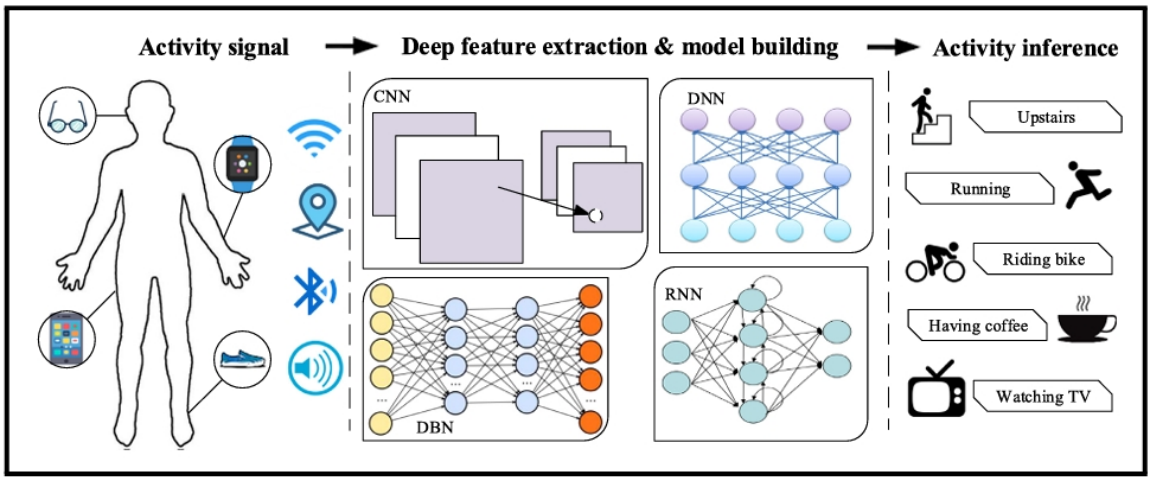


Figure 2.2: Sensor-based activity recognition using deep-learning approaches. The figure illustrates how activity signals from wearable or mobile sensors can be processed through deep feature extraction and model building before activity inference. The general processing-stage structure is adapted from Wang et al. [29]; sensor types, stage labels, and annotations were modified for the wrist-IMU HAR setting of this thesis.

This model family is particularly relevant for anxiety-related wrist behavior recognition. Activities such as nail biting, hair pulling, hand scratching, and hand tapping are not always separable by movement intensity alone. They may require the model to capture short repeated motion motifs, rotational transitions, and temporal context within the window. Prior work on ADAMSense used deep sequence modeling for anxiety-displaying activities from wearable motion sensors [7]. The Garmin-based study used as a foundation for this thesis similarly applies a 1D-CNN-BiLSTM architecture to commercial smartwatch sensor data for anxiety-related activity recognition [8].

Because the thesis later evaluates both within-distribution and user-held-out behavior, the model family must also be considered together with the evaluation protocol. A CNN–LSTM model may perform strongly under random window-level validation while still generalizing poorly to an unseen user if user-specific motion patterns leak across the split. This motivates the stricter subject-independent evaluation discussion in the following section.

However, deep learning does not remove the need for careful pipeline design. A deep model can learn useful representations only under the assumptions present during training. If the inference pipeline changes the units, sampling rate, channel order, scaler policy, window construction, or label mapping, the model may receive data that no longer matches its learned input contract. Therefore, in this thesis, the 1D-CNN-BiLSTM model is treated not as an isolated artifact but as one component of a larger runtime contract. The model, preprocessing configuration, scaler, baseline, and label order must remain consistent for the predictions to be meaningful.

The role of deep learning in this thesis is therefore bounded. The thesis does not claim to propose a new neural architecture. Instead, it uses a defensible HAR model family as

the predictive component inside a reproducible MLOps pipeline. The main research focus is how such a model is operationalized, monitored, and governed under deployment-style wearable conditions where labels may be unavailable, users may differ from the training population, and train–serve assumptions may be fragile.

2.4 Datasets, Wearables, and Domain Shift

Datasets strongly shape what a wearable HAR model can learn and how its results should be interpreted. Many established HAR datasets are collected under controlled or semi-controlled conditions, where participants perform predefined activities, sensors are placed according to a protocol, and labels are available for evaluation. Examples such as early accelerometer-based HAR datasets and benchmark datasets like UCI HAR and PAMAP2 helped standardize window-based activity recognition and made model comparison easier [25, 37, 38, 39]. However, such datasets often differ from the conditions expected in continuous wearable monitoring, where users move naturally, sessions may be noisy, labels may be incomplete, and sensor placement can vary.

For anxiety-related behavior recognition, the dataset problem becomes more specific. The relevant activities are not only general activities such as walking, sitting, or standing, but fine-grained hand- and wrist-centered behaviors such as nail biting, hair pulling, hand scratching, hand tapping, rubbing, and related repetitive motions. The ADAMSense work is important because it directly studies anxiety-displaying activities using wearable motion sensors and provides evidence that such behaviors can be modeled as a supervised HAR task [7]. This makes it a close conceptual foundation for the present thesis.

At the same time, transferring this idea to a commercial smartwatch setting introduces a lab-to-life gap. Research datasets often use controlled collection protocols and may rely on specific sensor devices, placements, sampling rates, and preprocessing conventions. Commercial wearables, by contrast, are designed for everyday use and may expose data through device-specific APIs, sampling behavior, unit conventions, and firmware-dependent characteristics. A model that performs well in one sensor environment may therefore not behave the same way when applied to data from another device or user group. The Garmin-based study used in this thesis context is especially relevant because it moves anxiety-related HAR toward a commercial smartwatch setting. It shows that a 1D-CNN-BiLSTM model can be adapted to Garmin Venu 3 sensor data for anxiety-related activity recognition [8].

This supports the feasibility of the thesis task, but it also highlights the engineering problem: the model is only meaningful when the runtime data preserve the expected sensor channels, units, sampling rate, preprocessing sequence, windowing logic, and label order. Lab-to-life shift can appear in several ways. First, user variability affects how the same behavior is performed. Nail biting, rubbing, or tapping may differ in speed, intensity, duration, and wrist orientation across users. Second, sensor displacement can change the measured signal even when the activity is the same. Wrist-worn devices may rotate, loosen, or shift during use, and such displacement has been shown to affect HAR performance [10]. More broadly, smart devices can differ in sensor characteristics, sampling

behavior, and measurement conditions, so models trained under one device or user setting cannot be assumed to transfer without validation [40]. Third, device and preprocessing differences can create train–serve mismatch. A change in acceleration units, gyroscope units, resampling logic, gravity handling, normalization, or channel order can alter the numerical meaning of the model input.

This also affects evaluation protocol design. In wearable HAR, random or k-fold window-level splits can overestimate deployment performance because windows from the same user or session may appear in both training and validation sets. Such validation measures *within-distribution* learnability: whether the model can learn the labeled data distribution under the chosen preprocessing and windowing assumptions. A large gap between within-distribution and blind-probe accuracy is informative rather than contradictory: it separates what the model can learn about the training population from what it can generalize to an unseen wearer, and it is precisely this gap that motivates the monitoring and governance focus of this thesis [28, 6, 39].

Figure 2.3 illustrates the lab-to-life and train–serve mismatch that motivates the runtime-contract view.

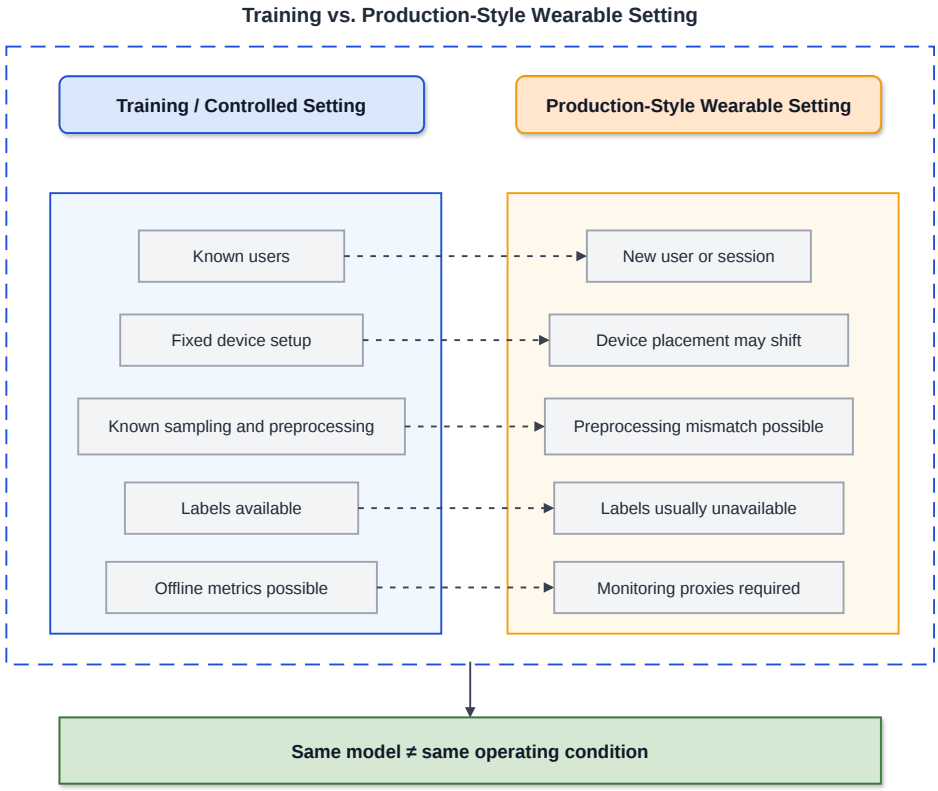


Figure 2.3: Lab-to-life and train–serve mismatch in wearable HAR. A model trained under controlled assumptions may behave differently when user behavior, device placement, preprocessing, or label availability changes during deployment-style inference.

These issues explain why dataset and device provenance are not minor implementation details. In this thesis, the model is not treated as portable by default. It is tied to a runtime contract that includes the data source, sensor columns, unit conversion, scaler

policy, window shape, class order, and monitoring baseline. The purpose of this contract is to reduce hidden mismatch between the training setting and the deployment-style inference setting. The literature on transfer learning, domain adaptation, and sensor displacement reinforces this point. Studies on HAR transfer learning and unsupervised domain adaptation show that models often require adaptation when users, devices, or domains change [11, 41].

However, the present thesis does not claim to solve domain adaptation generally. Instead, it uses the lab-to-life gap as motivation for an MLOps pipeline that can validate inputs, preserve preprocessing assumptions, monitor prediction behavior, and route suspicious sessions to review rather than silently treating every inference batch as reliable. Therefore, the dataset and commercial-wearable context define an important boundary for this thesis. The work is not only about training or selecting a HAR classifier. It is about making the assumptions behind that classifier explicit, reproducible, and inspectable when the model is used on deployment-style wearable sensor data.

2.5 MLOps, Reproducibility, and Offline-First ML Pipelines

The preceding sections show that wearable HAR models are sensitive to sensing context, preprocessing assumptions, user variation, and deployment conditions. These issues cannot be solved by model architecture alone. A trained neural network is only one artifact in a larger machine learning system. The surrounding lifecycle, which spans data ingestion, validation, preprocessing, configuration, model versioning, inference, monitoring, and update control, determines whether the model can be reproduced, inspected, and safely operated after training.

Before reviewing the monitoring and lifecycle literature in detail, it is useful to define what *MLOps* means in this thesis and why it is distinct from model-centric machine learning. A model-centric workflow often treats the trained checkpoint and its offline validation metrics as the main output. An MLOps workflow treats the model as one versioned artifact inside a larger system that also includes validated data, configuration files, preprocessing assumptions, runtime contracts, monitoring outputs, and update policies.

MLOps is commonly described as the intersection of machine learning, data engineering, and DevOps. Machine learning contributes the model and its training procedure; data engineering contributes versioned data, validation, and feature pipelines; and DevOps contributes automation, integration, deployment, and monitoring practices [42, 14]. This lifecycle perspective is important for the present thesis because the main risk is not only whether the 1D-CNN-BiLSTM model can learn the labeled Garmin data. The larger question is whether the trained model can be executed, inspected, monitored, and governed when data arrive from new users or deployment-style sessions.

MLOps provides the lifecycle perspective needed for this problem. Amershi et al. show that machine learning systems create engineering challenges at the interfaces between data, code, models, validation, deployment, and feedback [13]. Kreuzberger et al. define MLOps

Table 2.1: Model-centric machine learning versus an MLOps lifecycle view.

Aspect	**Model-centric ML**	**MLOps lifecycle view**
Goal	Train an accurate model.	Operate a reliable and inspectable ML system.
Main artifact	Model checkpoint.	Versioned data, code, configuration, model, scaler, baseline, and monitoring artifacts.
Validation	Offline metrics such as accuracy or macro-F1.	Offline metrics plus runtime-contract, data-quality, and deployment checks.
Deployment	Often manual, implicit, or notebook-driven.	Versioned and reproducible runtime contract.
Monitoring	Often absent after evaluation.	Confidence, drift, collapse, temporal, and data-quality signals.
Retraining	Ad hoc or experiment-driven.	Review-gated or policy-controlled update path.
Reproducibility	Dependent on scripts, notebooks, or local state.	Configured, versioned, and auditable pipeline execution.

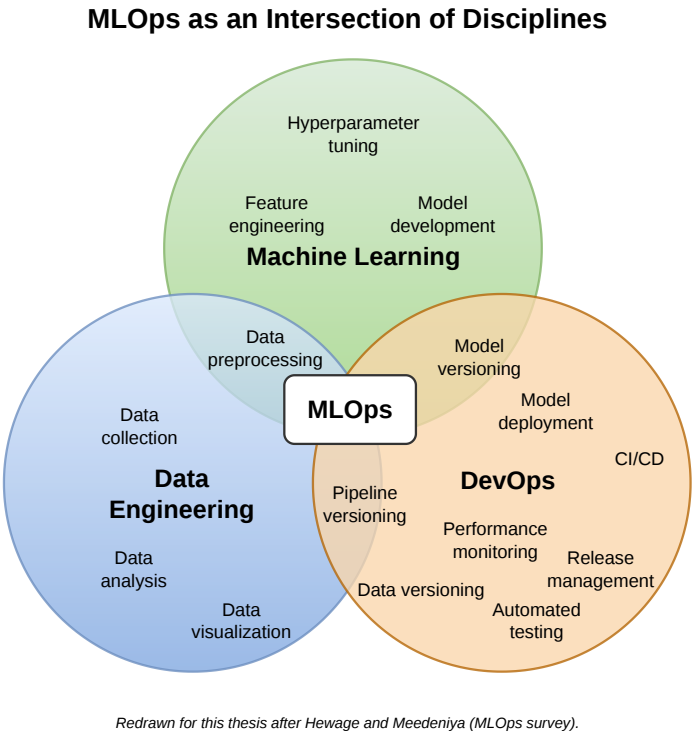


Figure 2.4: MLOps as the intersection of machine learning, data engineering, and DevOps. The three-intersection Venn structure is adapted from Hewage and Meedeniya [42]; labels and positioning were revised to reflect the offline-first setting of this thesis.

as an engineering discipline that combines machine learning, DevOps, and data engineering to support the reliable development, deployment, monitoring, and maintenance of ML systems [14]. Paleyes et al. similarly emphasize that many deployment failures arise from operational and organizational challenges rather than from model training alone [15]. This view is directly relevant to the present thesis because the target contribution is not only a classifier, but a governed wearable-sensor pipeline around that classifier.

A practical MLOps system must make assumptions explicit. In wearable HAR, assumptions are embedded not only in the model weights but also in the data schema, sensor units, channel order, sampling rate, scaler policy, windowing logic, label order, and monitoring baseline. If any of these elements changes silently, the deployed model may receive inputs that no longer match its training conditions. This is a form of hidden technical debt: the system appears to run, but its behavior may no longer be trustworthy because important dependencies are implicit [43]. For this reason, this thesis treats the runtime contract as a first-class MLOps artifact rather than as an implementation detail.

Reproducibility is another central requirement. Reproducible ML systems need more than source code. They require traceable data inputs, configuration files, trained model artifacts, preprocessing parameters, evaluation outputs, and execution metadata. Prior work on ML reproducibility and data management highlights that experiments become difficult to verify when data versions, preprocessing decisions, and execution environments are not recorded clearly [44, 45, 46]. In the context of this thesis, reproducibility means that a pipeline run can be repeated or inspected later with enough information to understand which data, model, scaler, baseline, thresholds, and configuration were used.

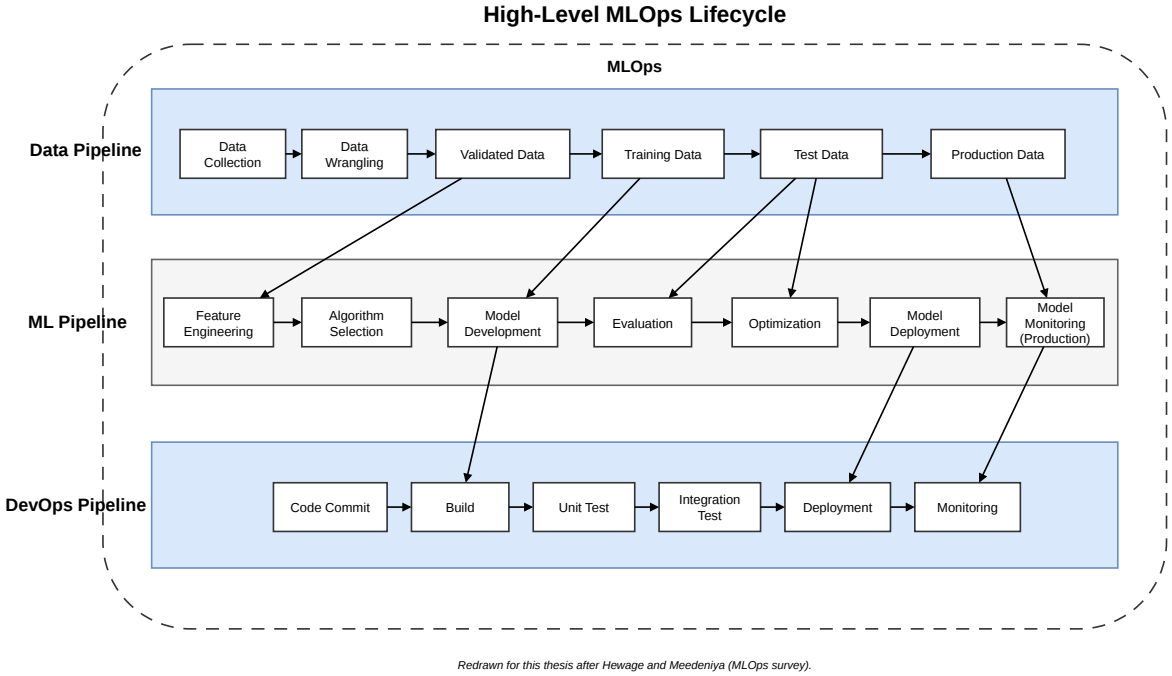


Figure 2.5: High-level MLOps lifecycle across data, machine-learning, and DevOps activities. The lifecycle stage sequence is adapted from Hewage and Meedeniya [42]; stage groupings and annotations were revised to match the offline-first pipeline scope of this thesis.

Tooling supports this lifecycle but does not replace the lifecycle design. MLflow is commonly used for experiment tracking and model lifecycle management [47], while DVC supports versioning of data and model artifacts alongside Git-based code workflows [48]. Containerization and local execution environments can also improve repeatability by reducing differences between development and runtime environments. Practical MLOps literature emphasizes automation, reusable pipelines, audit trails, monitoring, and drift observation as recurring elements of operational ML systems [16]. In this thesis, these ideas are adapted to an offline-first setting rather than to a full cloud production deployment.

The term offline-first is used deliberately. It does not mean that the model runs on the wearable device, nor does it mean that the system is a real-time clinical service. Instead, offline-first means that the complete pipeline can be executed and inspected on a local machine using versioned code, configuration files, model artifacts, and stored wearable sensor inputs. This is appropriate for a thesis setting because it prioritizes reproducibility, transparency, and evidence generation over cloud-scale automation. It also avoids overstating the system as a deployed medical product.

This offline-first framing should not be interpreted as a weaker form of MLOps. Rather, it is a deliberate reproducibility strategy for the thesis setting: every run must be inspectable through local code, configuration files, model artifacts, preprocessing parameters, and run-scoped evidence before stronger deployment claims can be made. The pipeline can process recorded wearable sessions, validate input structure, apply the expected preprocessing contract, generate predictions, evaluate results when labels are available, monitor behavior when labels are absent, and store artifacts for later inspection. This creates a reproducible bridge between experimental HAR research and deployment-style monitoring concerns.

In this thesis, MLOps is the mechanism that connects wearable sensing, behavioral-proxy recognition, monitoring, and retraining governance into one auditable lifecycle. The next sections build on this idea by reviewing monitoring, drift detection, confidence limitations, and the governance risks that arise when model updates are considered under label-scarce conditions.

2.6 Monitoring, Drift, and Confidence Limits

When a wearable HAR model is evaluated on a labeled test set, its behavior can be summarized with metrics such as accuracy, precision, recall, macro F1, and confusion matrices. In deployment-style wearable monitoring, this assumption often breaks down because ground-truth labels are usually unavailable when a new session arrives. The system may still generate predictions, but it cannot immediately verify whether those predictions are correct. This creates a central operational problem: the pipeline needs monitoring signals that can indicate risk even when direct performance metrics cannot be computed.

Monitoring in this thesis therefore sits inside the pipeline's control flow rather than beside it as a reporting layer; its outputs feed the trigger decision directly (Chapter 6). The goal is to observe whether an incoming session behaves similarly to the reference

conditions under which the model and monitoring baseline were defined. Relevant signals include prediction confidence, dominant-class concentration, temporal transition behavior, and statistical drift from baseline sensor distributions. Each signal captures a different type of operational risk, and none of them is sufficient alone.

Confidence is a useful but limited monitoring signal. A high softmax probability may suggest that the model is decisive, but it does not guarantee that the prediction is correct. Modern neural networks can be poorly calibrated, meaning that predicted confidence and empirical correctness may not align [17, 49]. This limitation is especially important in label-scarce inference settings because a model may produce high-confidence predictions even when the input distribution has shifted. For this reason, confidence is interpreted in this thesis as an operational signal, not as proof of correctness.

Uncertainty estimation provides a related but separate perspective. Bayesian approximations such as Monte Carlo dropout can be used to estimate predictive instability by observing variation across stochastic forward passes [18]. In wearable HAR, such uncertainty methods can help identify windows or sessions where the model response is less stable. However, uncertainty estimates are also not ground truth. They can support review decisions, but they cannot replace labeled evaluation or clinical interpretation. In the defended pipeline, uncertainty is therefore treated as a possible extension and supporting concept, while the core monitoring logic relies on implemented confidence, output-distribution, temporal, and drift signals.

Another important monitoring signal is output-distribution behavior. In a healthy multi-class HAR setting, predictions may vary over time depending on the activities present in the session. A suspicious failure mode is dominant-class collapse, where the model assigns most windows to a single class. Such collapse can occur because of model weakness, preprocessing mismatch, domain shift, or a genuine activity distribution that is highly imbalanced [50, 51]. The monitoring system therefore cannot treat dominance alone as proof of failure, but a high dominant-class share becomes a strong risk signal when combined with low transition rates, abnormal confidence, or input drift.

Drift detection focuses on whether incoming data differ from a reference distribution. General concept-drift literature distinguishes changes in the relationship between input data and labels from changes in the input distribution itself [50, 52]. In wearable sensing, drift may appear through changes in user behavior, sensor placement, device characteristics, sampling quality, or preprocessing. Because labels are often unavailable at inference time, this thesis focuses primarily on observable input and output signals rather than verified concept drift.

Statistical distance measures can support this type of monitoring. Maximum Mean Discrepancy (MMD), for example, provides a kernel-based way to compare whether two samples appear to come from the same distribution [53]. In a wearable pipeline, such methods can be used to compare incoming sensor windows or derived features against a training or baseline reference. However, statistical drift should be interpreted carefully. A detected distribution change does not automatically mean the model is wrong, and absence of detected drift does not guarantee correctness.

Out-of-distribution and misclassification-detection literature makes a related point.

Methods based on confidence, energy scores, or other model outputs can help identify unusual or potentially unreliable predictions, but they are imperfect and task-dependent [51, 54]. In smartwatch-based HAR, out-of-distribution behavior may arise from unseen activities, unusual motion patterns, sensor displacement, or corrupted preprocessing. For this reason, monitoring should combine several weak signals rather than rely on a single universal detector.

For the present thesis, the key monitoring principle is separation between observation and action. Monitoring signals can indicate that a session is healthy, suspicious, or severe enough to require review. They do not automatically authorize retraining or model replacement. This distinction is essential because label-scarce drift signals can be ambiguous. A high drift score may indicate a genuine new user pattern, a preprocessing error, a device issue, or a real domain shift. The correct response is therefore not always to update the model, but often to preserve artifacts, inspect evidence, and route the case to review.

This section motivates the monitoring design used later in the thesis. The defended pipeline design is motivated by confidence-related signals, output concentration, temporal behavior, and baseline drift indicators. These signals are intentionally conservative: they are designed to surface operational risk and support governance, not to claim verified prediction quality in the absence of labels.

2.7 Retraining Governance, Ethical Boundaries, and Research Gap

Monitoring becomes useful only when the system defines what should happen after a risk signal is detected. In a deployment-style wearable monitoring setting, suspicious confidence patterns, dominant-class collapse, drift indicators, or unusual temporal behavior may suggest that the model is no longer operating under familiar conditions. However, such signals do not automatically prove that the model is wrong, nor do they prove that retraining would improve the system. This makes governance a central part of the MLOps problem.

Automatic retraining is especially risky when labels are missing or unreliable. If a system retrains directly on its own predictions, errors can be reinforced through confirmation bias. Pseudo-labeling can be useful in semi-supervised learning, but it can also amplify incorrect predictions when the initial model is overconfident or when the target data are shifted [55, 56]. In wearable HAR, this risk is practical rather than theoretical: a collapsed model may label most windows as one class, and retraining on those outputs could make the collapse more stable instead of correcting it.

Continual-learning and domain-adaptation methods show that model updates can be technically possible under changing conditions, but they also introduce new risks. Updating a model for a new domain or user can degrade previously learned behavior, a problem commonly discussed as catastrophic forgetting [57, 58]. Test-time adaptation methods can modify model behavior without labels, but they still require careful assumptions about

incoming data quality and target-domain structure [59]. For a mental-health-adjacent wearable system, the safer default is therefore not automatic mutation, but controlled review.

This thesis separates monitoring severity from retraining permission. A session may be classified as healthy, warning-level, or critical from an operational point of view, but the trigger policy does not automatically promote that status into model retraining. Instead, suspicious sessions are routed toward review, quality assurance, label collection, or further analysis. Retraining, model registration, and baseline update are treated as governed mutation stages rather than as automatic consequences of drift detection. This design reflects the idea that an ML pipeline should preserve evidence before changing the model that produced it.

The ethical boundary is also important. Wearable mental-health-adjacent data are sensitive because they may reveal behavior patterns, routines, stress-related signals, or clinically relevant context. Even when the technical task is HAR, the application framing can influence how outputs are interpreted. For this reason, the thesis avoids diagnostic claims and treats predictions as behavioral activity outputs rather than mental-health conclusions. A prediction such as nail biting or hand tapping may be relevant for later interpretation, but it does not by itself establish anxiety, stress, disorder severity, or treatment need.

This boundary also affects system design. The pipeline should make its uncertainty and limitations visible rather than hide them behind a single prediction. In health-adjacent ML, a conservative system that routes uncertain or shifted cases to review is more defensible than a system that silently updates itself from weak evidence. The system creates traceable artifacts and monitoring signals to make human review more informed, not to displace it.

The related-work gap can now be stated clearly. Wearable HAR literature provides strong foundations for sensing, windowing, and deep activity recognition. Mental-health wearable literature motivates the use of continuous sensing for anxiety- or stress-related monitoring, while also showing that clinical interpretation requires caution. Domain-shift, device-heterogeneity, and adaptation literature explains why wearable models may degrade across users, devices, and conditions. MLOps literature explains the need for lifecycle engineering, reproducibility, monitoring, and governance. However, these strands are often treated separately.

The gap addressed in this thesis is the lack of a bounded, offline-first MLOps lifecycle for wearable-sensor-based mental-health-adjacent behavior recognition. The thesis does not claim a new neural architecture, a universal domain-adaptation method, or a clinically validated mental-health monitoring product. Instead, it contributes a reproducible and inspectable pipeline in which the wearable HAR model, preprocessing contract, monitoring baseline, trigger policy, and retraining boundary are treated as one governed system. This positioning leads directly to the system architecture presented in Chapter 3.

Table 2.2: Synthesis of related work and the remaining gap addressed by this thesis.

Area	**Representative work and coverage**	**Gap for this thesis**
Wearable mental-health monitoring	Abd-alrazaq et al. and Gomes et al. review wearable sensing for mental-health-related monitoring, including anxiety and stress contexts [1, 2].	Motivate wearable sensing, but not an executable HAR pipeline with contract checks, monitoring evidence, and review-gated updates.
Anxiety-related behavioral proxies	Khan et al. show that anxiety-displaying activities can be recognized from wearable motion signals [7].	Establish supervised behavior recognition, but not deployment monitoring, artifact lineage, or no-auto-retrain governance.
Wearable HAR	Lara and Labrador, Bulling et al., and Bao and Intille provide foundations for wearable sensing, windowing, and activity recognition [5, 6, 25].	Lara & Labrador and Bulling et al. establish sensing pipelines and windowing protocols but do not address mental-health-adjacent use, deployment monitoring, or governance under label-scarce conditions.
Deep HAR	Ordóñez and Roggen, Hammerla et al., and related deep HAR work show CNN/LSTM-based sequence modeling for activity recognition [27, 28, 29].	Ordóñez & Roggen and Hammerla et al. advance CNN/LSTM architectures for activity recognition but do not address retraining governance, runtime-contract enforcement, or monitoring under label-scarce deployment conditions.
Domain shift, device heterogeneity, and user-held-out evaluation	Transfer-learning, domain-adaptation, sensor-displacement, and device-heterogeneity studies show that HAR models are sensitive to users, devices, placement, and evaluation protocol [11, 41, 10, 40].	Explain cross-user and cross-device degradation, but rarely connect held-out-user evidence to runtime-contract promotion, thresholds, and trigger governance.
MLOps and reproducibility	Amershi et al., Kreuzberger et al., and Paleyes et al. motivate lifecycle engineering, deployment discipline, and reproducibility for ML systems [13, 14, 15].	Define MLOps lifecycle concerns broadly, but do not bind model, scaler, label order, baseline, and trigger policy into one offline-first runtime contract.
Monitoring and retraining governance	Calibration, uncertainty, pseudo-labeling, and confirmation-bias literature show why confidence and model updates must be handled carefully [17, 18, 55, 56].	Motivate calibration and cautious updates, but not a combined confidence, temporal, dominance, and drift policy for no-auto-retrain wearable HAR.

Chapter 3

Offline-First MLOps System Architecture

Building on the MLOps lifecycle framing developed in Sections 2.5–2.7, this chapter presents the offline-first system architecture that operationalizes the thesis contribution. The architecture is not a software wrapper around a trained HAR model. It defines how recorded wearable sensor sessions are ingested, validated, transformed, inferred, monitored, and routed to governance decisions while model mutation remains explicitly separated from ordinary assessment. Throughout this thesis, *mutation* denotes any operation that changes the deployed model or its baseline, such as retraining, model registration, or baseline promotion, whereas *assessment* only observes a recorded session and reports on it without altering the active model. The chapter therefore establishes the system-level rules that Chapters 4, 5, and 6 instantiate in data, code, and monitoring logic.

3.1 Architectural Goals and Constraints

As established in Chapter 2, this thesis adopts an MLOps lifecycle perspective but bounds it to an offline-first, label-scarce, governance-gated setting. Standard production-oriented MLOps often assumes cloud infrastructure, continuous delivery, online monitoring, and feedback-driven retraining [13, 14, 15, 42]. This thesis uses a different operating envelope: local execution, recorded wearable sessions, run-scoped artifacts, manual promotion, and review-gated mutation. Table 3.1 summarizes this architectural contrast.

The architecture is shaped by four practical goals. First, the system must be reproducible on a local machine from versioned code, configuration files, input data, and model artifacts [44, 45]. Second, stage boundaries must be explicit so that intermediate outputs, monitoring signals, and final trigger decisions can be inspected after execution. Third, the runtime must support deployment-style wearable sessions where labels may be unavailable at inference time. Fourth, the architecture must separate risk observation from model mutation, which keeps hidden dependencies between data, preprocessing, and model code from accumulating silently [43].

These goals are especially important in wearable HAR because apparently small im-

Table 3.1: Comparison between standard production-oriented MLOps and the offline-first MLOps setting used in this thesis.

Aspect	**Standard production-oriented MLOps**	**This thesis: offline-first MLOps**
Execution venue	Cloud services, managed platforms, or orchestration infrastructure.	Single-machine, file-based, session-bounded execution.
Deployment cadence	Continuous delivery, staging/production environments, traffic splitting, or live service updates.	Manual promotion of versioned runtime-contract folders.
Monitoring substrate	Online monitoring over live traffic or continuous data streams.	Batch or session-level monitoring after each recorded wearable inference run.
Retraining trigger	May be triggered by metric drift, delayed feedback, or automated lifecycle policies.	Review-gated; labels, quality checks, and validation evidence are required before mutation.
Train–serve binding	Often distributed across model registry, serving infrastructure, and preprocessing services.	Explicit runtime-contract folder tying model, scaler, label order, monitoring baseline, metadata, and evidence together.
Label availability	May use delayed feedback, shadow evaluation, or online ground truth.	Labels are usually unavailable at inference time; labeled evaluation is handled separately.
Scope of claim	May target deployed production service behavior.	Targets reproducible thesis evidence, local execution, and governed wearable-HAR assessment.

plementation decisions can change the numerical meaning of the model input. Unit conversion, channel order, scaler policy, window size, label order, and monitoring thresholds are therefore architectural concerns, not only preprocessing details. The defended system is intentionally bounded: it is an offline-first thesis prototype, not a cloud production service, a Kubernetes deployment, an on-device wearable application, or a real-time clinical product.

3.2 Pipeline Stages and Execution Views

The architecture is organized into two linked execution views. The first view is the operational assessment path. This path processes a recorded wearable session, validates its structure, transforms the data into the expected model input format, runs inference, computes evaluation metrics when labels are available, performs monitoring, and produces a trigger decision. This path corresponds to Stages 1–7 and is executed for ordinary assessment runs.

The second view is the governed mutation path. This path covers retraining, model registration, and baseline update. It corresponds to Stages 8–10 and is not entered automatically during normal inference. Instead, it requires evidence, review, and an explicit decision to change the active runtime contract. Calibration is treated separately as an optional extension because it can support uncertainty analysis, but it is not required for the minimal defended runtime.

This separation is central to the thesis. The operational path can observe risk and preserve evidence without changing the deployed model. The mutation path can replace or update the model only after the required governance conditions are satisfied. This makes the system conservative by design: it can detect that a session may be unreliable without assuming that automatic retraining is the correct response [14, 15]. Figure 3.1 instantiates the generic MLOps lifecycle introduced in Figure 2.5 for the offline-first wearable HAR pipeline developed in this thesis.

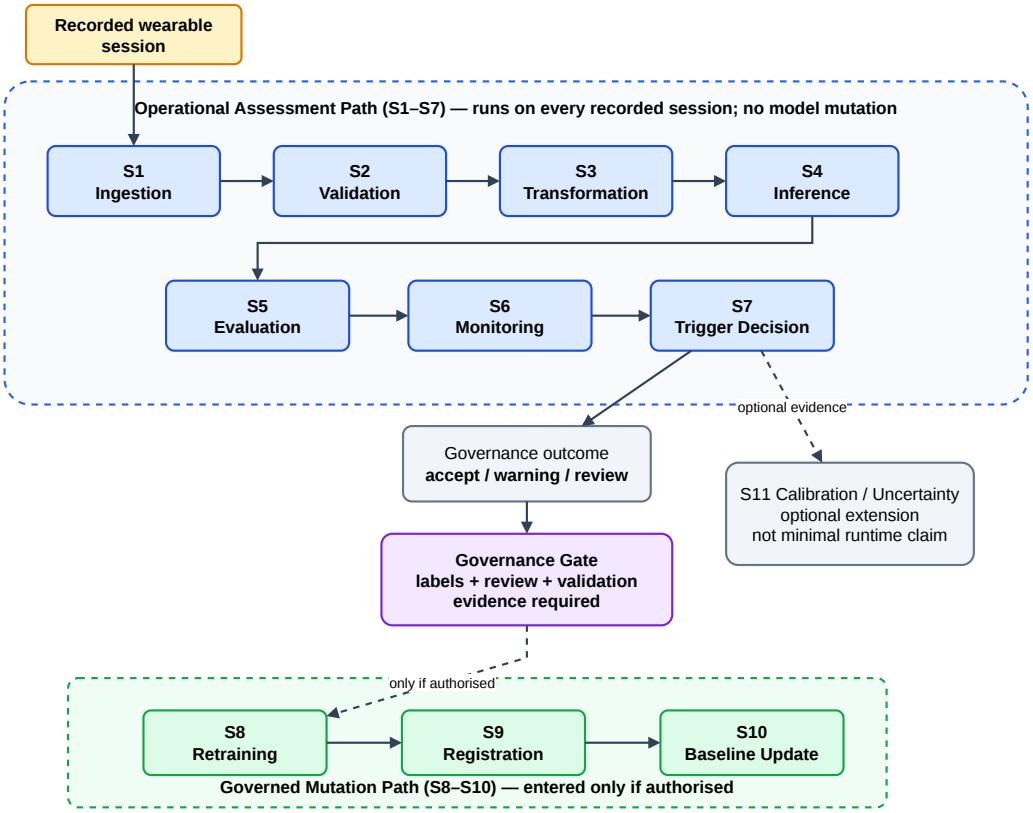


Figure 3.1: Offline-first MLOps architecture of the defended pipeline. Stages 1–7 form the operational assessment path and can run on each recorded session without mutating the active model. Stages 8–10 form the governed mutation path and are entered only when review, labels, and validation evidence justify a model or baseline update. Calibration is treated as an optional extension rather than as part of the minimal runtime claim.

The deep-learning model that drives Stages 3 and 4 of this pipeline is a 1D-CNN-BiLSTM. Figure 3.2 shows the concrete model path: the six-channel IMU stream is segmented into fixed windows, passed through a 1D-CNN for local motion-feature extraction and a BiLSTM for temporal-context modeling, and classified by a softmax output layer.

3.3 The Operational Assessment Path: Stages 1–7

The operational assessment path consists of seven stages. Stage 1 ingests wearable sensor data and brings the input into the expected file and column structure. Stage 2 validates

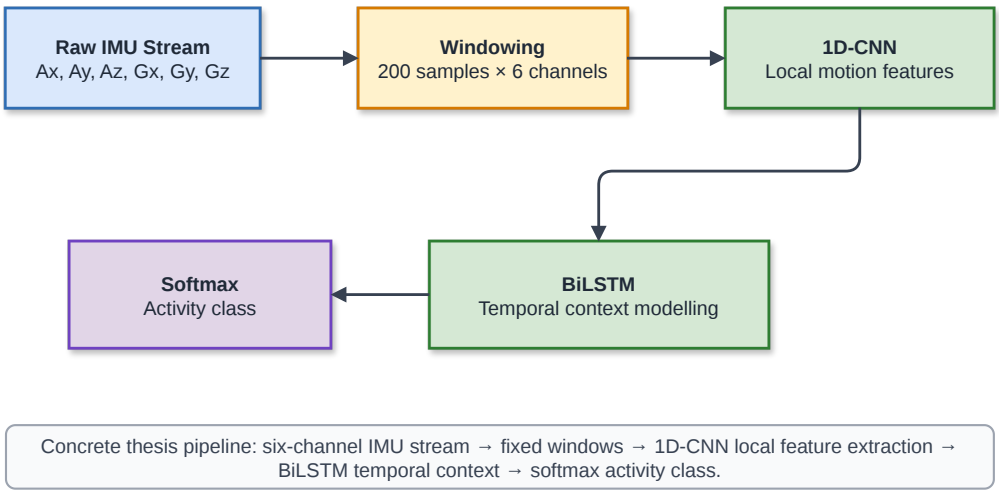


Figure 3.2: Deep-learning pipeline adopted in this thesis. The six-channel IMU stream (Ax, Ay, Az, Gx, Gy, Gz) is segmented into fixed windows, passed through a 1D-CNN for local motion-feature extraction and a BiLSTM for temporal-context modeling, and classified by a softmax layer. This instantiates the generic deep-HAR model family introduced in Figure 2.2.

the data against schema, unit, timestamp, and completeness expectations. Stage 3 applies the transformation logic required by the runtime contract, including unit handling, channel ordering, scaling policy, and window construction. Stage 4 performs model inference and records class probabilities. Stage 5 evaluates predictions when ground-truth labels are available. Stage 6 monitors the run using confidence-related signals, temporal behavior, dominant-class concentration, and distribution-shift indicators. Stage 7 combines these signals into a governance outcome. In practical terms, the later trigger policy uses a collapse guard to catch one-class output concentration, signal voting to require more than one warning before escalation, and a cooldown gate to prevent repeated review requests from the same unresolved condition.

None of these seven stages mutates the active model. They may generate warnings, reports, evaluation files, monitoring summaries, and trigger decisions, but they do not retrain, register, or promote a new model. This preserves a clean separation between observation and action [13, 16]. It also makes negative or unexpected results easier to interpret because the system state remains fixed during the assessment run.

This design also supports preprocessing and contract-sensitivity experiments. The same recorded session can be processed under different controlled configuration settings while the model artifact remains fixed. Such a run can show whether a change in preprocessing, normalization, unit handling, or baseline selection changes monitoring severity. A single alert is not proof of model failure; the architecture preserves the evidence so that a later review can determine whether retraining is warranted.

Table 3.2: Summary of the architecture stages and whether they are allowed to mutate the active model.

Path	Stage	Role and typical output	Mutates model?
Assessment	S1	Ingest wearable session data and standardize input access; produces ingestion metadata.	No
Assessment	S2	Validate schema, columns, units, timestamps, and basic quality checks; produces validation report.	No
Assessment	S3	Apply runtime transformation, scaling policy, and windowing; produces transformed windows.	No
Assessment	S4	Run model inference and store class probabilities; produces prediction outputs.	No
Assessment	S5	Compute evaluation metrics when labels are available; produces evaluation report.	No
Assessment	S6	Monitor confidence, temporal behavior, collapse risk, and drift; produces monitoring report.	No
Assessment	S7	Convert monitoring signals into a governance decision; produces trigger decision.	No
Mutation	S8	Retrain or fine-tune a candidate model under controlled conditions; produces candidate artifacts.	No (produces candidate)
Mutation	S9	Register candidate model with version and provenance metadata; produces registry evidence.	No (registers candidate)
Mutation	S10	Promote compatible baseline and runtime package after review; produces updated contract.	Conditional (promotes)
Optional	S11	Calibration or uncertainty analysis when activated; produces supplementary evidence. Not implemented in the defended pipeline; architectural placeholder for future extension.	No in defended runtime

Within this assessment path, Stages 3 and 4 segment the raw six-channel IMU stream into fixed windows and apply the 1D-CNN-BiLSTM model shown in Figure 3.2. The convolutional layers extract local motion features from each window, and the BiLSTM layer models temporal context before softmax classification.

3.4 The Governed Mutation Path: Stages 8–10

The governed mutation path contains the stages that can change the operational state of the system. Stage 8 performs retraining or fine-tuning on a candidate dataset. Stage 9 registers the candidate model together with its provenance, configuration, and evaluation evidence. Stage 10 updates the active baseline and runtime package only if the candidate satisfies the required contract and review conditions.

This path is deliberately separated from ordinary inference because monitoring signals are not training labels. A critical drift signal, dominant-class warning, or low-confidence run can indicate operational risk, but it does not prove that retraining would improve the system. In label-scarce wearable monitoring, automatic mutation can be unsafe because the model may reinforce its own errors if it is retrained from unreliable predictions [55, 56]. For this reason, the mutation path requires explicit governance rather than automatic

execution.

The governance gate has three practical functions. First, it protects the active runtime from being replaced by a candidate that only improves one specific failure case observed during debugging. Second, it requires the reviewer to inspect the available evidence, including labels, preprocessing assumptions, evaluation metrics, monitoring reports, and contract compatibility. Third, it preserves rollback capability: if a candidate fails validation or later proves unsuitable, the previous runtime package remains available as a known reference point. In this design, retraining is not treated as the natural consequence of a monitoring alert. It is treated as a controlled intervention that must be justified.

A candidate model should therefore not be promoted as a single checkpoint file. It must enter the system as a complete runtime package, including preprocessing assumptions, scaler, label order, monitoring baseline, integrity metadata, and evaluation evidence. This makes the update process reviewable and reversible, and it prevents silent changes to only one part of the inference contract.

3.5 Artifact Lineage and Runtime Contract

The strongest architectural rule in this thesis is that the model is not the deployment unit; the runtime contract is. A model file alone is insufficient because its predictions are meaningful only when the input data are transformed in the same way as during training and interpreted with the same label order, scaler policy, and monitoring assumptions. The runtime contract therefore combines the model artifact, scaler, channel order, unit policy, window shape, label mapping, monitoring baseline, configuration snapshot, integrity metadata, and validation evidence.

This contract view addresses a common source of hidden technical debt in ML systems: dependencies between data, preprocessing, model code, and evaluation assumptions may remain implicit until the system fails [43]. In this thesis, those dependencies are made explicit. Each pipeline run should be traceable from input file to transformed tensor, prediction output, monitoring report, and trigger decision. Reproducibility therefore depends not only on preserving code, but also on preserving the artifact set that defines the model's operating conditions [44, 45, 46].

3.5.1 Runtime Contract as a Same-Folder Invariant

To prevent silent mismatch, the active model, scaler, label order, and monitoring baseline are treated as a same-folder runtime contract. The system should not allow the model to be loaded from one folder while the scaler or baseline is silently taken from another historical experiment. If a newer model is promoted, its compatible scaler, label order, baseline, metadata, and evidence must be promoted with it. If a configured scaler or baseline path points outside the active model folder, the runtime should either correct the path to the matching contract artifact or fail fast when no compatible artifact exists.

This invariant is an architectural rule, not only a coding convenience. It prevents a

common failure mode in wearable HAR: the pipeline may execute successfully, but the model may receive numerically inconsistent input because normalization, unit handling, or baseline statistics belong to another training context. In a health-adjacent monitoring setting, such a mismatch is especially dangerous because the system may still produce confident predictions. The same-folder invariant therefore makes contract coherence inspectable before inference results are interpreted.

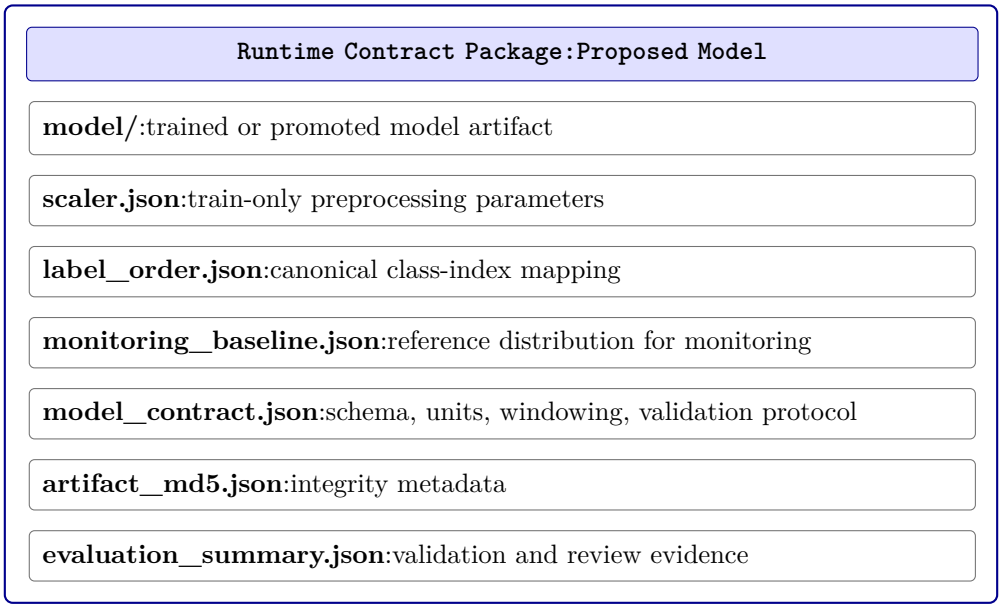


Figure 3.3: Runtime contract package for the promoted runtime of the proposed model. The model file is only one component of the deployable unit; the scaler, label order, monitoring baseline, contract metadata, integrity hashes, and evaluation evidence must be promoted together to preserve train–serve consistency.

Table 3.3: Runtime-contract consistency rules for the same-folder invariant.

Condition	**Pipeline behavior**	**Log or governance signal**
Scaler, label order, and baseline are found inside the active model contract folder.	Use the artifacts as the coherent runtime package.	Runtime contract accepted.
A configured scaler or baseline path points outside the active model folder, but a matching artifact exists inside the active folder.	Override the stale path with the matching same-folder artifact.	Warning that mismatch was corrected.
A configured scaler or baseline artifact points outside the active model folder and no compatible same-folder artifact exists.	Fail fast before inference or promotion.	Error with rollback or contract-repair instruction.

The runtime contract also supports cross-environment reproducibility. Training and inference may occur in different Python, TensorFlow, or Keras environments. If model

artifacts are saved only in a format tied to one environment, a technically correct model may become difficult to load elsewhere. For this reason, the architecture treats portable artifact format as part of the reproducibility layer. A candidate model should be stored in a form that can be loaded by the defended inference environment, with legacy or training-environment formats retained only as supporting evidence or rollback material.

3.6 Scope Boundary

The offline-first boundary is technical. Offline-first means that the complete pipeline can run locally on stored wearable sessions using versioned code, configuration files, model artifacts, and run-scoped evidence. It does not mean that the model runs directly on the wearable device. It also does not mean that the system is a real-time cloud service or a Kubernetes-based deployment. Distributed orchestration is outside the defended runtime boundary because it would add infrastructure complexity without directly strengthening the thesis evidence about preprocessing contracts, monitoring, and governance.

The mental-health boundary is separate. In line with the fixed thesis title, continuous mental health monitoring is used as the application context, not as a claim of clinical diagnosis or prognosis. The system recognizes observable wrist-motion activities that may be relevant for mental-health-adjacent interpretation, but it does not diagnose anxiety, stress, depression, or any other clinical state. Activity predictions are behavioral outputs, not clinical conclusions. This boundary is important because the architecture is designed to make risk and uncertainty visible rather than to replace clinical interpretation.

These scope boundaries keep the contribution focused and verifiable. The contribution is an offline-first, reproducible, and governed MLOps architecture for wearable HAR under a mental-health-adjacent monitoring context. It is not a clinical deployment, not a medical device, not an on-device smartwatch application, and not an unrestricted automatic retraining system.

3.7 Chapter Summary

This chapter defined the system architecture that supports the thesis contribution: an operational assessment path for reproducible inference and monitoring, a governed mutation path for controlled model updates, and a runtime-contract layer that treats the model, scaler, baseline, label order, metadata, and evidence as one coherent deployment unit. Chapter 4 next defines the data, preprocessing, windowing, and model assumptions that this architecture must preserve. Chapter 5 then explains how the architecture is realized in the repository implementation. Chapter 6 returns to the monitoring signals and trigger logic that drive the governance boundary.

Chapter 4

Data Sources, Preprocessing, and Model Contract

4.1 Data Sources and Evidence Roles

This chapter defines the data sources, preprocessing assumptions, label space, model family, and runtime data contract used by the thesis pipeline. The purpose is not yet to describe the software implementation in detail; that is handled in Chapter 5. Instead, this chapter specifies the data and model assumptions that the offline-first architecture from Chapter 3 must preserve.

The thesis uses multiple data sources, but each source has a different evidence role. The source-domain anxiety-related activity data provide the supervised foundation for recognizing anxiety-displaying behaviors. The Garmin-labeled user data provide the main commercial-wearable adaptation setting. The self-recorded User 7 session is kept outside training, scaler fitting, hyperparameter selection, and model selection, and is used as a blind external probe. Separate raw unlabeled Garmin sessions are used to demonstrate deployment-style pipeline execution where labels are unavailable at inference time.

Table 4.1: Data sources, evidence roles, and data scope. Row and window counts describe preprocessing scope; performance metrics are reported in Chapter 7.

Source	**Labels**	**Rows**	**Windows**	**Evidence role**
ADAMSense [7]	Labeled	709,582	Source-domain	Supervised source-domain pretraining for anxiety-related or anxiety-displaying activity recognition.
Garmin users 1–6	Labeled	385,326	3,755 (200-sample, 100-stride)	Fine-tuning, within-distribution validation, class-balance inspection, and user-held-out evaluation design.
User 7 blind probe	Labeled	99,017	989	Blind external probe; excluded from training, scaler fitting, hyperparameter selection, and model selection.
25 unlabeled Garmin sessions	Unlabeled	varies by session	25 sessions	Monitoring, confidence, drift, and trigger-policy demonstration at scale (Chapter 7); no direct accuracy claims.

These roles are enforced throughout the thesis: labeled sources support accuracy and macro-F1 claims, while the unlabeled sessions support only monitoring and trigger-policy evidence. The at-scale monitoring evaluation in Chapter 7 covers the 25 sessions that processed successfully under the standard ingestion path.

Table 4.2: Per-source channel statistics (mean ± standard deviation) for the six IMU channels. Accelerometer values are in m/s^2 after unit normalization; gyroscope values are in deg/s. These differences are descriptive evidence of train–serve distribution shift across data sources, not model-performance evidence.

	Accelerometer (m/s^2)			**Gyroscope (deg/s)**		
Source	Ax	Ay	Az	Gx	Gy	Gz
ADAMSense	$+2.22 \pm 5.69$	$+0.57 \pm 4.89$	-4.49 ± 3.82	-0.91 ± 48.46	$+1.73 \pm 23.69$	$+0.08 \pm 20.97$
Garmin 1–6	$+3.22 \pm 6.57$	$+1.28 \pm 4.35$	-3.53 ± 3.24	$+0.60 \pm 49.93$	$+0.23 \pm 14.81$	$+0.09 \pm 14.17$
User 7	$+3.65 \pm 5.73$	$+2.33 \pm 4.18$	-4.08 ± 3.76	$+0.72 \pm 50.53$	$+0.55 \pm 25.83$	$+0.00 \pm 29.54$
Unlabeled	$+0.76 \pm 6.42$	$+4.23 \pm 4.09$	-3.48 ± 3.84	$+0.47 \pm 62.62$	-0.16 ± 32.71	$+0.04 \pm 55.58$

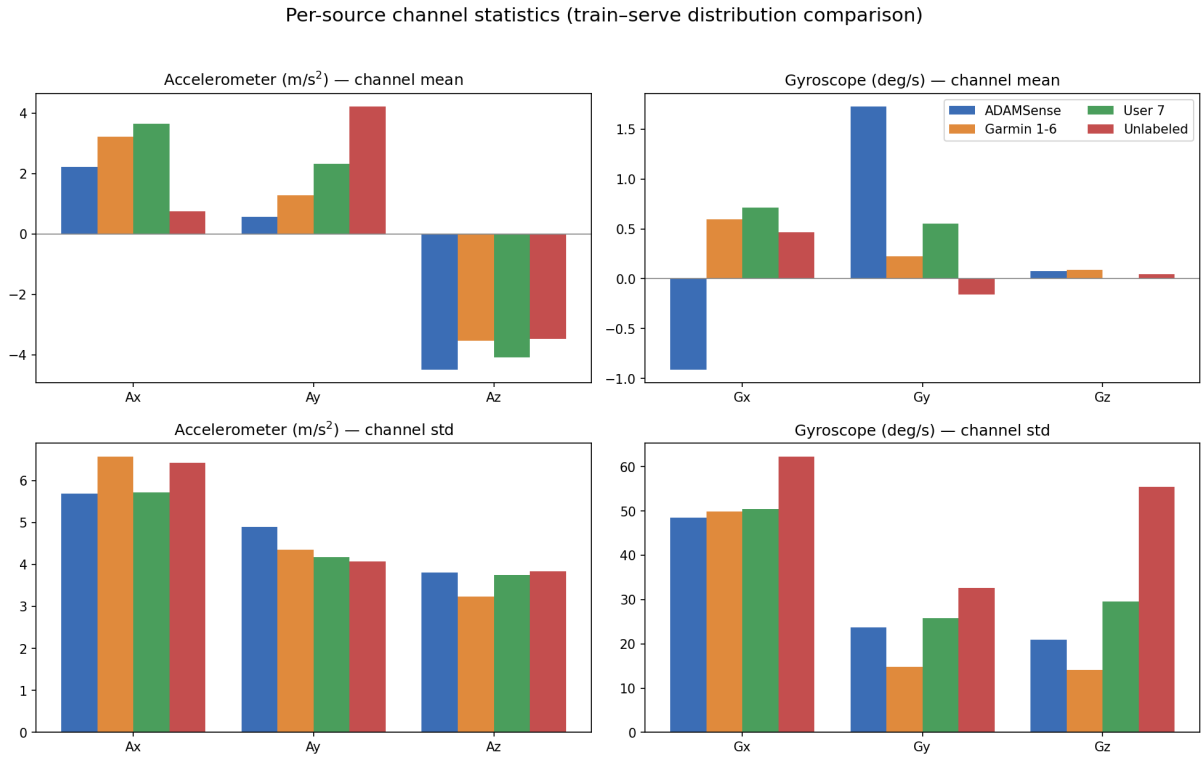


Figure 4.1: Per-source channel mean and standard deviation for the six IMU channels after unit normalization. Differences across ADAMSense, Garmin users 1–6, User 7, and the unlabeled Garmin sessions illustrate cross-source variation relevant to train–serve distribution shift.

Even at the level of unit-harmonized, unscaled channel statistics, the four sources differ in ways that matter for deployment. The unlabeled deployment-style sessions show a markedly lower Ax mean (+0.76 vs. +3.22 m/s^2 for Garmin users 1–6) and a much larger gyroscope spread (for example, Gz standard deviation 55.58 vs. 14.17 deg/s). These

descriptive differences motivate the train–serve contract and the monitoring layer defended later, since the model must operate despite such distribution shift.

Figure 4.2 compares the accelerometer and gyroscope magnitude distributions across the four sources; the differing spreads and tails illustrate the cross-source distribution shift that the runtime contract and monitoring layer must tolerate.

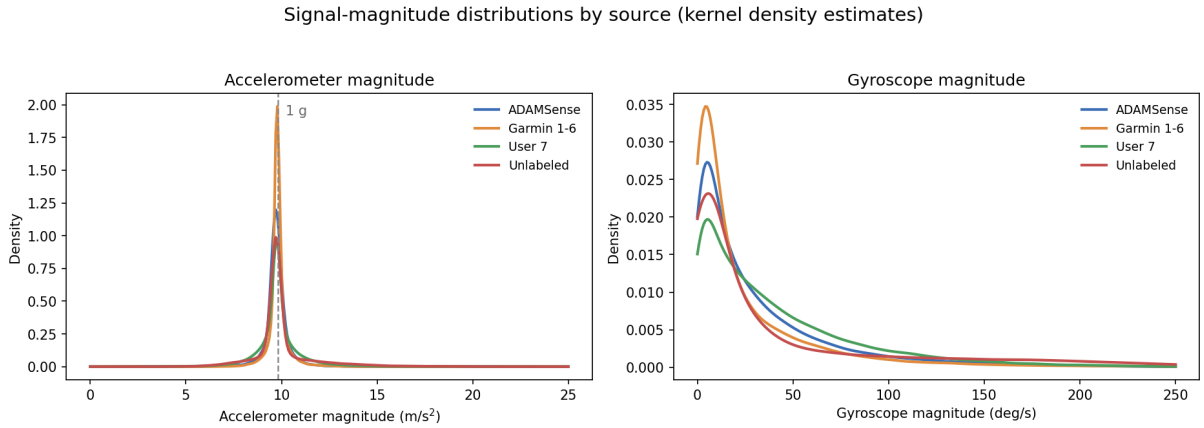


Figure 4.2: Accelerometer and gyroscope signal-magnitude distributions by source (kernel density estimates). The accelerometer magnitude concentrates near gravity for all sources, while the peak heights and gyroscope tails differ, illustrating cross-source distribution shift.

The Garmin users 1–6 recordings originate from the supervisor's published data-collection study [8], collected with participant consent, while the User 7 session and the unlabeled deployment-style sessions were recorded by the author on the author's own device; no third-party personal data is introduced by this thesis. Wrist-worn HAR is sensitive to individual movement style, watch placement, and recording context [40, 10], which is why provenance matters for interpreting the cross-user results in Chapter 7. The corpus is usable for controlled model development and contract verification, but it remains small and user-limited; broad population-level generalization is not claimed.

The activity classes are also imbalanced. Figure 4.3 shows the per-class window counts for Garmin users 1–6, which vary widely across the eleven activities. This imbalance is the reason macro-F1 is reported alongside accuracy and balanced class weights are used during training, as discussed in Chapters 5 and 7.

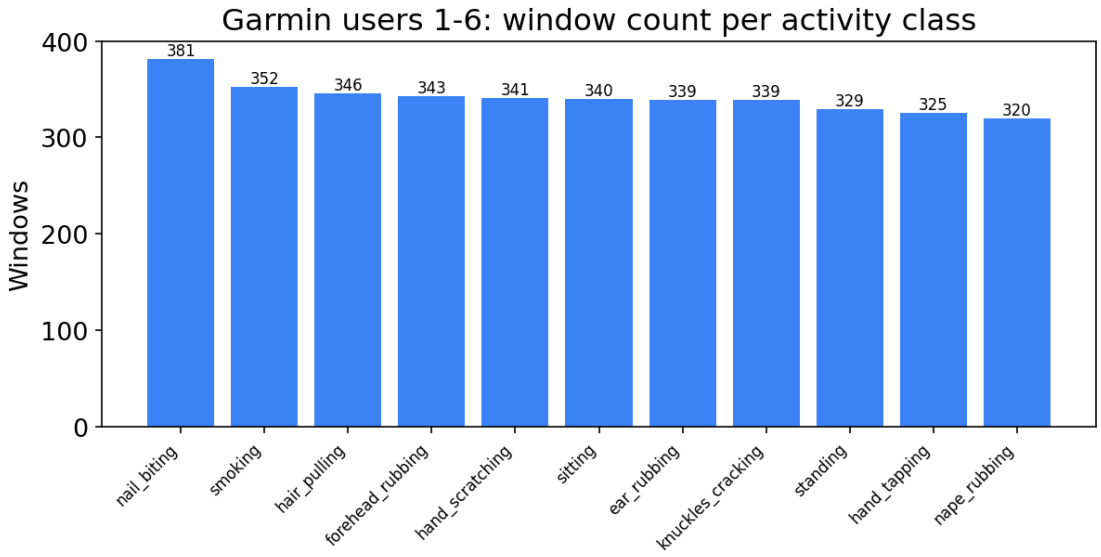


Figure 4.3: Window count per activity class for Garmin users 1–6 (3,755 windows total). The classes are imbalanced, which motivates macro-F1 reporting and balanced class weighting.

4.2 Sensor Schema, Units, and Label Space

The runtime pipeline is built around a fixed six-channel inertial measurement unit schema. The canonical feature order is:

$$Ax, Ay, Az, Gx, Gy, Gz.$$

Here, Ax, Ay, and Az denote the three accelerometer axes, and Gx, Gy, and Gz denote the three gyroscope axes. The accelerometer channels are represented in $\mathrm{m/s^2}$, and the gyroscope channels are represented in deg/s. This schema is treated as part of the model contract rather than as a flexible implementation detail. Any change in column order, units, or channel count changes the meaning of the input tensor and can invalidate inference.

The raw data sources do not all arrive in the same unit convention. In particular, some accelerometer recordings are represented in milli-g rather than $\mathrm{m/s^2}$. Therefore, accelerometer unit conversion is handled before scaling and before window construction. The conversion constant used in the pipeline is:

$$1 \text{ milli-g} = 0.00980665 \text{ m/s}^2.$$

Unit conventions differ by source: the raw Garmin replay files are stored in milli-g and converted as above, whereas the source-domain ADAMSense accelerometer values are recorded in g and harmonized separately into the common $\mathrm{m/s^2}$ convention. This harmonization is applied before any cross-source statistic is computed, so that the channel comparison in Table 4.2 reflects genuine distribution differences rather than a unit mismatch.

Figure 4.4 summarizes the preprocessing contract from raw wearable data to model-ready tensors.

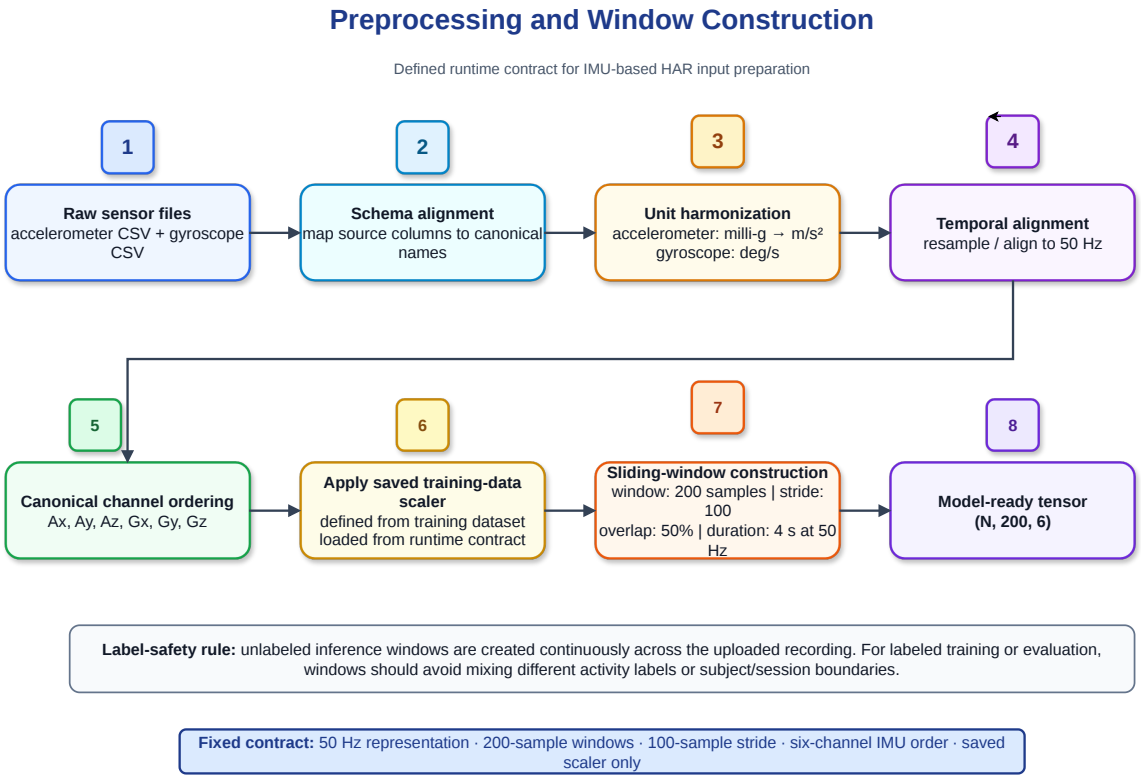


Figure 4.4: Preprocessing contract from raw wearable recordings to model-ready tensors. Unit conversion, canonical column ordering, scaler application, and window construction are treated as contract steps rather than optional preprocessing choices.

The proposed model uses only the six IMU channels Ax, Ay, Az, Gx, Gy, Gz. It does not use an additional magnitude feature channel. Any reference to magnitude-scaling in the training workflow refers to training-time data augmentation, not to an extra model input feature.

The target label space contains eleven activity classes. The numeric order is fixed because the final softmax layer emits probabilities in this exact order. Table 4.3 defines the mapping used by the thesis pipeline.

The label order is therefore not a cosmetic metadata field. If the label file, training code, inference code, and reporting code use different orders, the model can output numerically valid probabilities that are semantically assigned to the wrong classes. For this reason, the fixed label order is part of the runtime contract and is promoted together with the trained model and preprocessing artifacts.

4.3 Windowing and Tensor Representation

The model operates on fixed-length time windows. After unit normalization and column ordering, the continuous six-channel time series is transformed into overlapping windows of

Table 4.3: Fixed activity-label order used by the proposed model's runtime contract.

Index	Activity label
0	`ear_rubbing`
1	`forehead_rubbing`
2	`hair_pulling`
3	`hand_scratching`
4	`hand_tapping`
5	`knuckles_cracking`
6	`nail_biting`
7	`nape_rubbing`
8	`sitting`
9	`smoking`
10	`standing`

200 samples, following the standard sliding-window formulation of sensor-based HAR [25, 6]. At the thesis sampling rate of 50 Hz, this corresponds to a four-second input segment. The baseline stride is 100 samples, corresponding to a two-second shift between adjacent windows.

Each model input is therefore represented as:

$$\mathbf{X} \in \mathbb{R}^{200\times 6}.$$

The first dimension represents time, and the second dimension represents the fixed sensor-channel order. This time-major representation is used consistently across model training, validation, and inference.

Table 4.4: Windowing and tensor contract used by the proposed model.

Contract element	Value / rule
Sampling rate	50 Hz after preprocessing and resampling.
Window length	200 samples, corresponding to four seconds at 50 Hz.
Stride	100 samples, corresponding to two seconds at 50 Hz.
Input tensor shape	$(200, 6)$, with time as the first dimension and sensor channels as the second dimension.
Channel order	Ax, Ay, Az, Gx, Gy, Gz.
Boundary rule	Windows are constructed within user and activity segments; a window never crosses a labeled activity boundary or a participant boundary, which avoids mixed-label windows at transitions.

The four-second window is an engineering compromise. It is long enough to contain short repetitive wrist movements such as rubbing, tapping, scratching, and biting, while remaining short enough to sustain the two-second prediction cadence of the 50%-overlap windowing. Compared with the ADAMSense study, which reported its strongest configuration with longer 12-second windows and a magnitude feature, this thesis uses a shorter four-second window and a fixed six-channel IMU input [7]. This choice reduces temporal

context but better matches the short-cadence offline wearable pipeline evaluated in this thesis; the windowing ablation of Section 7.5.3 quantifies the trade.

Tensor shape alone does not guarantee temporal equivalence. A shape of $(200, 6)$ states that the model receives 200 time steps and six channels, but it does not prove that two datasets share the same temporal scale, unit convention, or sampling behavior. The runtime contract therefore requires the correct upstream assumptions as well: unit conversion, sampling rate, channel order, scaler usage, and label order.

4.4 Model Family and Proposed Architecture

The thesis uses a one-dimensional convolutional and bidirectional recurrent neural-network family for sensor-based HAR. This model family is suitable for multichannel time-series classification because convolutional layers can extract short local motion patterns, while recurrent layers can model temporal dependencies across the input window [27, 28, 60]. Long Short-Term Memory networks were introduced to address vanishing-gradient limitations in recurrent sequence learning [32], and bidirectional recurrent layers process the sequence in both temporal directions [33].

The proposed model keeps this general 1D-CNN–BiLSTM structure. Its preprocessing and windowing configuration was selected among the candidates evaluated during development (the windowing ablation of Section 7.5.3 reports the surviving sweep), and the resulting runtime was then frozen and promoted under that configuration. The architecture is treated as part of the experimental model family, while the exact deployed model is treated as part of the runtime contract. Model D1 is the internal artifact name used in the repository; in the thesis text, the same runtime is referred to as the proposed model. Figure 4.5 summarizes the block-level design.

Table 4.5: Block-level summary of the proposed model's architecture.

Block	**Role**	**Contract relevance**
Input layer	Accepts $(200, 6)$ IMU windows	Fixes the expected temporal length and six-channel sensor schema.
1D convolutional feature extraction	Learns local temporal motion patterns	Encodes short-range wrist-motion signatures from the fixed input channels.
Pooling / regularization	Reduces temporal resolution and limits overfitting	Supports more stable representation learning on limited user-level data.
Bidirectional LSTM layers	Models temporal dependencies across the window	Allows the classifier to use sequence context from both directions within the fixed offline window.
Dense classification head	Maps learned representation to class probabilities	Produces an eleven-class softmax vector in the fixed label order from Table 4.3.

The proposed model contains 967,499 parameters (966,347 trainable, 1,152 non-trainable), as verified from the build notebook output and recorded in `model_param_count.json` in the runtime contract folder. These counts describe architectural scale

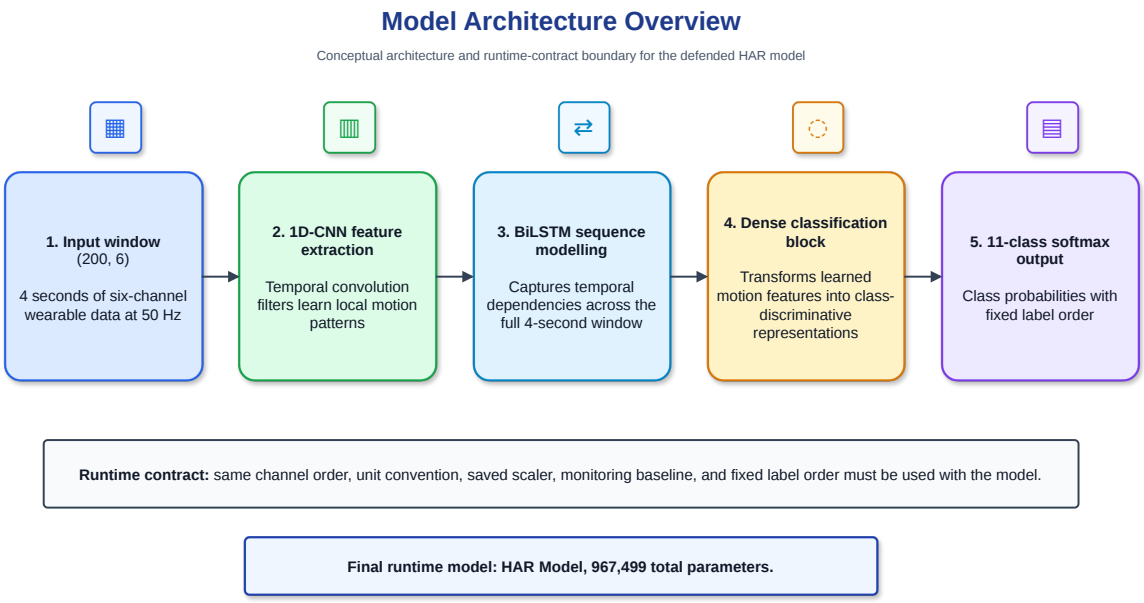


Figure 4.5: Block-level architecture of the proposed model. The model receives a $(200, 6)$ IMU window, extracts local temporal patterns using one-dimensional convolutional layers, models sequence context with bidirectional LSTM layers, and produces an eleven-class softmax output.

only and are not performance claims. Note that `model_contract.json` also records a 94.41% pre-augmentation hold-out accuracy from the build notebook; this uses a single 10% stratified split executed during training and is distinct from the rigorous post-hoc five-fold cross-validation result of 93.61% reported in Chapter 7. Recognition results, cross-validation scores, and blind-probe behavior are reported later in Chapter 7.

The architecture is intentionally not presented as a clinical detector. It is a sensor-based activity-recognition model for behaviors that may be relevant to anxiety-related monitoring. The model predicts activity classes from wearable IMU windows; any interpretation beyond activity recognition must be handled carefully and is outside the direct model output.

4.5 Runtime Data and Model Contract

The central operational assumption of this chapter is that the trained model is not deployable as an isolated file. The model only has a valid meaning together with its preprocessing rules, scaler, label order, input shape, and monitoring baseline. This thesis therefore treats the deployed unit as a runtime contract rather than as a standalone neural-network checkpoint.

Formally, the deployed unit is the runtime contract

$$\mathcal{C} = (M, S, L, B, \Omega),$$

where M is the frozen model, S the train-only scaler, L the fixed label order, B the monitor-

ing baseline, and Ω the preprocessing and windowing metadata (unit convention, channel order, window size, and stride). The scaler S applies the per-channel standardization

$$x'_{i,j} = \frac{x_{i,j} - \mu_j}{\sigma_j}, \qquad j \in \{Ax, Ay, Az, Gx, Gy, Gz\},$$

where the mean μ_j and standard deviation σ_j are fitted only on the Garmin training data (users 1–6) and are never recomputed from User 7 or production sessions. Reusing the saved μ_j and σ_j is what preserves the train–serve feature space; fitting a fresh scaler on an incoming batch would silently change the input distribution the model was trained on.

Table 4.6: Runtime data and model contract for the thesis pipeline.

Contract item	**Required rule**
Model checkpoint	The promoted model file is used for inference. It must not be swapped independently from the scaler, label order, and monitoring baseline.
Input shape	Every inference batch must contain windows of shape $(200, 6)$.
Channel order	The six input channels must be ordered as Ax, Ay, Az, Gx, Gy, Gz.
Accelerometer unit	Accelerometer values must be represented in $\mathrm{m/s^2}$ before scaler transformation. Milli-g inputs must be converted using 1 milli-g $= 0.00980665$ $\mathrm{m/s^2}$.
Gyroscope unit	Gyroscope values are represented in deg/s.
Scaler	The saved Garmin train-only scaler is applied at inference. A new scaler must not be fitted on production or probe data.
Windowing	Windows use 200 samples and 100-sample stride after preprocessing. Labeled windows must not cross activity boundaries.
Label order	Softmax outputs are decoded using the fixed eleven-class label order in Table 4.3.
Monitoring baseline	Drift and confidence checks are interpreted relative to the promoted baseline that belongs to the same runtime contract.
Evidence boundary	Labeled data can support supervised metrics. Unlabeled deployment-style data can support monitoring and trigger-policy evidence, but not direct accuracy claims.

This contract avoids a common deployment failure mode in which a model checkpoint is updated without also updating the preprocessing artifacts that give the input tensor its meaning. For example, applying a different scaler, reordering the sensor channels, or decoding the softmax vector with a different label list may not produce a runtime error, but it can silently change the semantics of the prediction. In the thesis pipeline, this is treated as an MLOps problem rather than a purely modeling problem.

The same contract also protects the interpretation of monitoring. A drift alert is meaningful only if it is computed in the same feature space and with the same preprocessing assumptions as the model input. Therefore, the scaler, monitoring baseline, model checkpoint, and label-order file must be promoted together. This same-folder invariant is developed as an implementation rule in Chapter 5.

4.6 Chapter Summary

This chapter defined the data and model assumptions that the offline-first MLOps pipeline must preserve. The data sources were separated by evidence role, spanning source-domain supervised pretraining data, labeled Garmin development data, blind labeled User 7 probe data, and unlabeled deployment-style Garmin data, and the chapter then fixed the six-channel IMU schema, unit assumptions, label order, windowing procedure, and model family. The central point is that the proposed model is not only a neural-network checkpoint but part of a runtime contract that also binds unit conversion, channel order, scaler usage, label decoding, window construction, and monitoring baselines into a single deployable unit. These assumptions are what make the later evaluation and monitoring results interpretable. Performance metrics are intentionally deferred to Chapter 7, while this chapter establishes the evidence boundaries and deployment contract needed to interpret those results safely.

Chapter 5

Pipeline Implementation

This chapter explains how the architecture from Chapter 3 and the data–model contract from Chapter 4 are realized in the repository implementation. The aim is not to provide a code listing, but to make the implementation auditable. Every architectural rule from Chapter 3 and every contract element from Chapter 4 maps to a specific Python module, configuration source, runtime package, or run-scoped artifact in the repository.

5.1 Repository Layout and Execution Entry Point

The repository separates source code, configuration, runtime contract packages, execution utilities, and run-scoped evidence into distinct areas. This separation supports the offline-first claim: the pipeline can be inspected and executed from local files without relying on a managed cloud platform. Runtime model packages are kept together with their companion artifacts (the scaler, label order, monitoring baseline, contract metadata, and evaluation evidence), so that each promoted runtime contract is a self-contained, inspectable unit. Preprocessing and monitoring thresholds are externalized as configuration rather than hidden inside notebooks, which makes them part of the reproducibility boundary.

The main scripted execution path is concentrated in a single entry point that assembles the layered configuration, resolves the active runtime contract, and invokes the stage-oriented pipeline. Concentrating execution this way keeps the thesis evidence reproducible from one command, rather than dependent on disconnected notebooks or ad-hoc scripts. A runbook of all reproducibility commands is provided in Appendix A.

5.2 Technology and Design Rationale

The implementation choices in this chapter are deliberate rather than incidental; the main choices below each map to an offline-first, contract-driven goal defined in Chapters 3 and 4. This section states the main technology and design decisions together with their rationale, so that the implementation reads as a set of engineering choices rather than as defaults; the broader problem of selecting an open-source MLOps tool stack is itself

an active research topic [61, 42]. A detailed tool and version reference is collected in the technology-stack appendix.

Table 5.1: Main technology and design choices and their rationale.

Choice	**Why it is used**
Python with TensorFlow/Keras	The HAR model is trained and frozen in Keras, so keeping training and serving in the same framework avoids a re-implementation gap between the training notebook and the runtime path.
Persisted scikit-learn `StandardScaler` (train-only)	A saved, train-only scaler guarantees train–serve parity. Recomputing a scaler on each inference batch would leak batch statistics and silently change the feature space the model was trained on.
YAML configuration layering	Preprocessing flags, thresholds, and stage activation are externalized so that controlled variation is reproducible without editing code, making configuration part of the reproducibility boundary rather than a hidden constant.
Stage-oriented components	Bounded, single-responsibility stages are independently testable and each emits an inspectable artifact, which is what makes the run-scoped evidence trail and contract checks possible.
FastAPI serving layer	A lightweight, same-language entry point consumes the identical runtime contract as the batch path, so the API does not become a second preprocessing path with different assumptions.
DVC for data and model versioning	Large data and model artifacts are versioned alongside Git-tracked code without bloating the repository, supporting the artifact-lineage and reproducibility claims [48].
Docker for environment control	Containerization pins dependency versions and provides a repeatable execution boundary for local and CI runs; it is used as a reproducibility aid, not as a cloud-deployment claim.
`pytest` contract tests	Automated tests guard the runtime contract (model, scaler, baseline, label order, API/batch parity) against silent drift as the code changes.
Offline-first, single-machine execution	Local, file-based execution prioritizes reproducibility, inspectability, and auditable evidence over cloud-scale automation, matching the defended Level 1 scope (Section 5.6.3).

Every design choice in this section keeps the runtime contract explicit and the evidence inspectable, reflecting the MLOps principle that reliable systems depend on the surrounding lifecycle rather than the model alone [13, 14, 43]. Tools are selected for reproducibility and contract safety in an offline-first setting, not for production-scale serving or orchestration, which are outside the defended scope.

5.3 Configuration Layering

The pipeline uses layered configuration rather than hard-coded stage behavior. This is necessary because the same codebase is used for labeled evaluation, deployment-style inference, monitoring, API execution, and controlled diagnostic runs. The configuration layer defines which model package is active, which scaler and baseline are used, which preprocessing path is applied, and which stages are enabled.

This layering supports reproducibility and controlled variation. A normal thesis run uses the proposed model's default runtime contract and the standard preprocessing settings

Table 5.2: Configuration source precedence in the pipeline implementation.

Configuration source	**Typical responsibility**	**Precedence role**
Code-level defaults	Defines default runtime model paths, contract artifact paths, and fallback constants.	Baseline source of truth when no override is provided.
Declarative pipeline configuration	Defines stage settings, preprocessing flags, thresholds, and pipeline-level behavior.	Main declarative configuration for ordinary thesis runs.
Runtime overrides / CLI arguments	Select input files, output locations, optional stage flags, or diagnostic alternatives.	Override YAML values for a specific execution.
Environment variables	Override selected deployment-facing paths such as model, scaler, or baseline locations.	Used for API, smoke-test, or rollback scenarios without editing repository code.

defined in the layered configuration. A diagnostic run can then change a specific flag, such as a preprocessing option or an alternative model path, without editing the underlying implementation. This keeps experimental variation explicit and reviewable.

Configuration changes must not silently break the runtime contract. A changed model path must remain compatible with the corresponding scaler, label order, and monitoring baseline. A changed preprocessing flag must be interpreted as a new runtime condition unless the model and baseline were prepared under the same assumption. For this reason, configuration assembly is part of the system's contract-enforcement mechanism.

5.4 Runtime Contract Implementation and Artifact Loading

The runtime contract defined in Chapter 4 is realized in code through three coordinated mechanisms: a single source of path defaults, a scaler-loading utility that validates artifact provenance, and a configuration-build step that binds the model artifact to its accompanying scaler and monitoring baseline. The implementation goal is not only to load files, but to ensure that the model receives input prepared under the same assumptions that defined its training and fine-tuning.

The path defaults are concentrated in a single configuration module that references the active runtime contract directory of the proposed model. That directory holds all artifacts that must be interpreted together: the model checkpoint, the train-only scaler, the label-order file, the monitoring baseline, the contract metadata, the integrity record, and the evaluation evidence. Earlier runtime candidates are retained through separate constants so that rollback or diagnostic comparison remains possible without restructuring the configuration layer.

Before the inference path consumes the scaler, a provenance check confirms that it declares train-only origin and carries the expected six-channel layout. This guards against

the case where a syntactically valid but contractually incompatible scaler is loaded into the inference path. The scaler is loaded from the promoted runtime contract and is never fitted on User 7 or deployment-style sessions; this preserves the train–serve feature space defined during Garmin users 1–6 training.

The API layer consumes the runtime contract through the same default constants as the batch path, all of which can be overridden by environment variables when needed. Environment overrides allow ad-hoc rollback or side-by-side comparison, while the unmodified defaults keep the pipeline aligned with the defended contract.

5.4.1 Same-Folder Invariant Enforcement

Chapter 3 introduced the same-folder invariant as an architectural rule: the model, scaler, label order, and monitoring baseline must live in one runtime contract folder and must be promoted together. The implementation of this rule is centralized in the configuration-assembly step, which is invoked at configuration-build time, before preprocessing or inference starts.

The helper resolves the runtime contract folder from the active model path and compares it against the configured scaler path. The three resulting outcomes correspond to the rules stated in Chapter 3, Table 3.3.

If the configured scaler already lives in the active model folder, the configuration is accepted unchanged and the runtime contract is treated as coherent. If the scaler path points outside the active model folder but a compatible scaler exists inside the active folder, the helper corrects the stale path and records a warning. If no compatible same-folder scaler exists, the helper fails fast before inference begins.

The monitoring baseline is aligned through the API and pipeline configuration defaults, which both resolve to the active contract folder, and is validated by the pytest contract tests. The invariant is enforced before any preprocessing or inference artifact is written. A misconfigured run therefore either proceeds under the coherent contract that the operator likely intended, or fails before producing misleading outputs. This implementation does not solve every possible form of contract drift, but it removes a high-risk failure mode: loading a model from one runtime package while scaling inputs with an artifact from another package.

5.5 Stage Execution and Artifact Trail

The pipeline is implemented as a sequence of stage-oriented components. Each stage has a bounded responsibility and writes outputs that can be inspected after execution. The assessment path can therefore be written as the composition

$$R = S_7(S_6(S_5(S_4(S_3(S_2(S_1(x))))))) \,,$$

where x is the input session and R is the run result, including the governance decision. On unlabeled runs S_5 acts as the identity, since evaluation metrics require labels; the composition is otherwise unconditional. This composition mirrors the architectural separation introduced in Chapter 3: Stages 1–7 form the operational assessment path, while Stages 8–10 belong to the governed mutation path and are not part of this default composition.

Table 5.3: Mapping between architecture stages and repository implementation modules.

Path	**Stage**	**Key output artifact and responsibility**
Assessment	S1	Ingestion metadata; standardizes session input access.
Assessment	S2	Validation report covering schema, unit, and timestamp checks.
Assessment	S3	Preprocessing metadata and model-ready transformed windows.
Assessment	S4	Prediction probabilities and class outputs.
Assessment	S5	Evaluation metrics when ground-truth labels are available.
Assessment	S6	Monitoring report with confidence, temporal, drift, and collapse signals.
Assessment	S7	Trigger decision and governance outcome.
Mutation	S8	Candidate model and training artifacts.
Mutation	S9	Model registry or promotion metadata.
Mutation	S10	Updated baseline and runtime-contract package.
Optional	S11	Calibration or uncertainty evidence when enabled.

To make the mapping concrete, it is useful to trace a single execution. When the pipeline entry point is invoked on a recorded session, it first assembles the layered configuration and resolves the active runtime contract of the proposed model. Stage 1 ingests the session and standardizes column access; Stage 2 validates schema, units, and timestamps; Stage 3 converts units where required, applies the saved train-only scaler, and constructs the row-wise windows of shape $(200, 6)$; Stage 4 runs inference and records per-class probabilities; Stage 5 computes evaluation metrics when labels are present; Stage 6 derives the confidence, temporal, drift, and dominant-class signals; and Stage 7 combines them into a single governance decision. On the labeled User 7 external session, for example, this path produced 989 windows and routed the run to `REVIEW` with `auto_retrain=false`, writing the full evidence chain shown in Figure 5.1. No mutation stage is entered unless review, labels, and validation evidence justify it.

5.5.1 Run-Scoped Artifact Trail

Each pipeline execution writes run-scoped artifacts so that the path from input data to trigger decision can be reconstructed after the run. This is the implementation side of the lineage principle discussed in Chapter 3. The pipeline does not rely only on console output; it writes structured files that preserve validation, transformation, inference, monitoring, and trigger evidence.

Run outputs are stored under an `artifacts/<run_id>/` convention and are also summarized in pipeline logs. A run identifier connects the input session, intermediate artifacts, and final decision, as illustrated in Figure 5.1. This makes it possible to inspect a single execution later without rerunning the full pipeline.

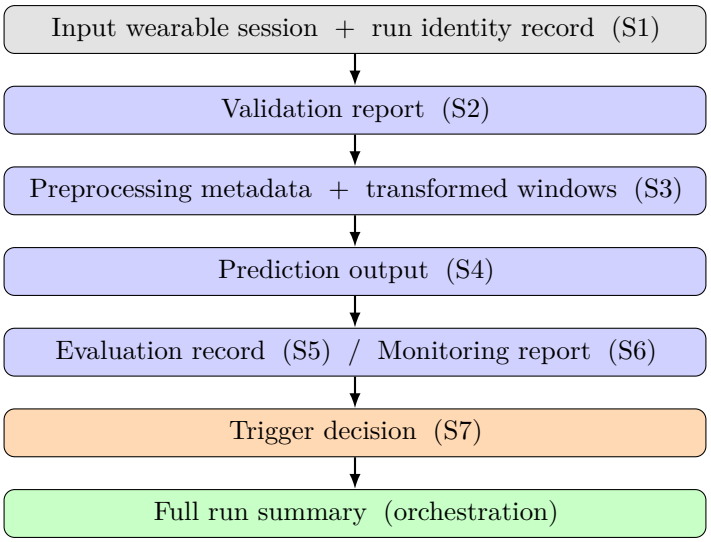


Figure 5.1: Run-scoped artifact trail produced by the offline-first pipeline. Each execution links the input session, validation report, preprocessed windows, prediction output, evaluation or monitoring record, trigger decision, and full run summary into one inspectable evidence chain.

This artifact trail supports debugging and reproducibility. If a run produces unexpected predictions or a severe trigger decision, the stored files allow the reviewer to inspect whether the issue came from input validation, preprocessing, inference, monitoring, or trigger evaluation. The same convention also supports later experimental comparisons because different runs can be compared by their artifacts rather than by informal notes.

5.6 Interfaces, Reproducibility, and Maturity

The repository supports both scripted execution and API-based inference. Scripted execution is used for reproducible thesis runs because it makes input paths, configuration values, and output folders explicit. The API interface provides a serving-style entry point for submitting wearable sensor files and receiving predictions or monitoring outputs.

The API does not redefine the runtime contract. It uses the same model path, scaler path, label order, baseline, and configuration defaults as the batch pipeline. This prevents the API from becoming a parallel inference path with different preprocessing assumptions. When environment variables are used to override paths, they are treated as explicit runtime changes rather than hidden defaults.

The serving interface is a browser-based dashboard. The upload screen presents three modes and a stage strip that makes the pipeline boundary visible before any file is submitted. The strip shows which stages run automatically on the server (Stages 1–3: ingestion, validation, and transformation), which stages produce the visible results (Stages 4–7: inference, evaluation, monitoring, and trigger decision), and which stages are review-gated and require a human action before they execute (Stages 8–10). The three upload modes differ in how the raw recording is supplied. The first mode accepts two separate CSV files, one containing the accelerometer channels and one the gyroscope

channels; the preprocessing pipeline fuses them using the same channel-alignment logic as the batch path. The second mode accepts a single pre-fused activity CSV without a label column, which the dashboard routes as an unlabeled deployment-style session and processes using only the proxy monitoring signals. The third mode accepts the same pre-fused format with an additional ground-truth label column, enabling a labeled evaluation session where accuracy and quality metrics are also computed.

After upload, the results are returned in three panels. The monitoring panel reports the three-layer signal summary: session mean confidence with its uncertain-window percentage, temporal transition rate shown as both a raw session value and a smoothed per-window trace, and per-channel distribution drift score, each with a color-coded severity status. The governance panel reports the trigger decision, the retraining status, and, when applicable, the reason for escalation or cooldown suppression. The prediction panel lists each window's predicted activity class and confidence score. When the upload carries ground-truth labels, the dashboard additionally shows an evaluation card with session-level accuracy, macro-averaged F1, and their temporally smoothed variants, together with a per-class confusion matrix. Appendix E provides annotated screenshots of both a labeled and an unlabeled session walking through each panel in turn.

The API and batch paths are treated as contract-equivalent execution interfaces: both resolve the same promoted runtime package, apply the same scaler and label order, and must therefore remain behaviorally aligned. This alignment is verified as an implementation test rather than asserted as a separate model-performance claim; the contract and upload-mode tests check that both paths consume the identical runtime contract so that the API cannot silently diverge into a different preprocessing path.

Verification and portability are supported by the test suite and the promoted package rather than by ad-hoc scripts. Automated contract tests verify that the active proposed-model package loads correctly, that the model, scaler, baseline, and label order stay aligned, and that the API and batch upload modes behave consistently. In the defended runtime, both the batch pipeline and the FastAPI serving path load the native `.keras` model from the promoted contract folder; the SavedModel-compatible export is kept alongside it for cross-environment portability, not as a separate serving path.

5.6.1 Experiment Tracking Across the Two Execution Paths

Both execution interfaces record evidence to MLflow as two separate experiments, mirroring the contract-equivalent but operationally distinct relationship described above. The batch path logs to the `har-production-pipeline` experiment, where each stage contributes its own metrics so that an entire governed run, from ingestion through the trigger decision, is reconstructable from the tracking record. The serving path logs to the `inference-production` experiment, capturing the focused inference-and-monitoring evidence of a single upload. Keeping the experiments separate prevents interactive serving runs from diluting the batch-run history while still recording both under one local tracking store.

Each run logs a curated set of essential signals rather than raw telemetry. For the

batch pipeline this is one or two metrics per stage: the input shape, the validation-gate result, the window count, inference timing and mean confidence, the temporal, drift, and dominant-class monitoring signals, and the final retraining decision with its alert level. Redundant confidence statistics and constant configuration flags, which either repeat across stages or never vary between runs, are deliberately excluded so the tracking record stays auditable rather than noisy. The serving path logs the analogous proxy set: confidence, drift, the raw and smoothed transition rate, per-activity counts, and, only when the upload carries ground-truth labels, accuracy and macro-F1. MLflow therefore records what the pipeline observed on a specific run, making it possible to compare monitoring severity or trigger outcomes across sessions without rerunning the pipeline [47]. Representative screenshots of the dashboard and of both MLflow experiments are collected in Appendix E.

5.6.2 Reproducibility and Environment Control

The implementation is designed for local reproducibility, following reproducibility practice for scientific machine learning [44, 45, 46]. Configuration files, runtime contract folders, run-scoped artifacts, and tests are kept inside the repository structure so that a run can be reproduced or inspected without a managed production platform. Docker and dependency files support environment consistency, while local execution keeps the defended runtime aligned with the offline-first scope.

A specific reproducibility issue concerns model format portability across environments. Training and conversion may occur in an environment using older TensorFlow/Keras conventions, while thesis execution may occur in a newer environment. To reduce this compatibility risk, the runtime contract retains the native `.keras` artifact used by the defended batch and API paths together with a SavedModel-compatible export for portability, while additional legacy or environment-specific artifacts may be kept as supporting material. This decision belongs to implementation reproducibility: the model package must be loadable by the defended inference environment, not only by the training notebook.

Automated tests provide another reproducibility layer. The defended suite contains 88 tests across 11 files. Its largest families guard exactly the thesis-critical behavior: Stage 6 monitoring (16 tests), Stage 7 trigger policy (14), end-to-end pipeline execution (12), API upload modes (10), and runtime-contract consistency (5). The remaining tests cover ingestion, schema validation, unit conversion, preprocessing, and component-level checks. Contract tests verify that configuration defaults, model paths, scaler paths, API defaults, and runtime assumptions remain aligned, and a continuous integration workflow runs the full suite on every push, so contract regressions surface before they reach the defended branch. These tests are not substitutes for model evaluation, but they reduce the risk that code changes silently break the runtime contract. In an offline-first thesis setting, such checks are valuable because reproducibility depends on local files staying consistent over time.

The roles of the two artifact-management tools are distinct. DVC *is* part of the defended reproducibility mechanism: the raw, processed, and prepared data, the pretrained models, the promoted runtime-contract folder, and the defended evidence run are DVC-tracked, so

they can be restored from content-addressed storage through Git-tracked pointer files [48]. MLflow records what the pipeline observed on a run; DVC or the release bundle supplies the artifacts the pipeline consumed. The proposed model's runtime package also includes an integrity manifest, so the model, scaler, baseline, and label-order files can be verified as one coherent package.

Containerization follows the same contract discipline. Two Docker images are defined: a training image with the full training stack and Stage 1–7 entry point, and a lean inference image that mounts the proposed model's contract folder as a read-only volume. This keeps the frozen assessment path separate from the mutation path described in Chapter 3. Data and models are not baked into either image; the deployment unit remains the runtime-contract folder, and Docker is used as a local reproducibility aid rather than as a Kubernetes-style production-serving claim.

5.6.3 MLOps Maturity Position

The implementation places this thesis most closely at MLOps Level 1: pipeline automation with reproducible batch execution [14, 42]. It exceeds a manual notebook workflow because ingestion, validation, transformation, inference, monitoring, and trigger evaluation are staged, configuration-controlled, and artifact-producing. It remains below Level 2 because retraining, baseline update, deployment, and orchestration are governance-gated rather than automatic.

Table 5.4: MLOps maturity position of the defended thesis pipeline.

Maturity level	**Typical meaning**	**Position in this thesis**
Level 0 / manual ML	Notebook-driven or manually connected scripts.	Exceeded: staged execution, controlled configuration, runtime contracts, and run-scoped artifacts.
Level 1 / pipeline automation	Repeatable pipeline execution with controlled configuration and artifact tracking.	Matched: Stages 1–7 run offline with validation, inference, monitoring, and trigger evidence.
Level 2 / automated training/deployment	Automated retraining, promotion, or production deployment.	Not claimed: retraining and baseline update remain governance-gated and are not automatic.

This positioning matches the offline-first boundary defended in Chapter 3 and motivates the cloud and continuous-deployment directions discussed as future work in Chapter 8.

5.7 Chapter Summary

This chapter mapped the thesis architecture and data–model contract to the repository implementation. It covered configuration layering, runtime-contract loading, the same-folder invariant, stage-to-module correspondence, run-scoped artifacts, API and batch alignment, and the reproducibility controls that support the evidence in Chapter 7. The defended pipeline is a Level 1 MLOps implementation: offline, manually triggered, and reproducible from versioned artifacts without requiring cloud infrastructure or continuous

deployment. Chapter 6 next focuses on the monitoring and trigger-governance logic built on top of this implementation.

Chapter 6

Monitoring and Governance

This chapter describes how the pipeline observes operational risk after inference and how that observation is converted into a governed decision. The architecture in Chapter 3 separated the operational assessment path (Stages 1–7) from the governed mutation path (Stages 8–10). Monitoring (Stage 6) and trigger evaluation (Stage 7) are the part of the assessment path that turns raw predictions into a risk assessment, and they are the mechanism that keeps model mutation explicitly gated rather than automatic. Monitoring is therefore part of the runtime control logic, not an optional layer added after inference.

6.1 Why Monitoring Is Needed Under Label-Scarce Operation

When a model is evaluated on a labeled test set, its behavior can be summarized with accuracy, macro-F1, and a confusion matrix. In deployment-style wearable monitoring this assumption breaks down, because ground-truth labels are usually unavailable when a new session arrives. The system can still generate predictions, but it cannot immediately verify whether those predictions are correct. The pipeline must therefore rely on *proxy* signals that indicate operational risk even when direct performance metrics cannot be computed [62].

The thesis treats these proxy signals as a control layer. Stage 6 observes the run and rates its behavior; Stage 7 converts that behavior into a single governance action. Neither stage mutates the active model. This preserves the central architectural rule that observation and mutation are separated: the system can detect that a session may be unreliable without assuming that automatic retraining is the correct response.

6.2 Post-Inference Monitoring

Stage 6 is warning-oriented and non-mutating. It evaluates three complementary signal layers plus a collapse-risk check. The first layer is confidence and uncertainty, which captures how decisive the model is per window. The second layer is temporal behavior,

which captures how often the predicted class switches between neighboring windows. The third layer is distribution drift, which compares incoming sensor windows against a reference baseline. The monitor also tracks dominant-class concentration, because a model that assigns most windows to a single class is a characteristic failure mode in label-scarce operation. Each signal is rated relative to a conservative operating threshold, summarized in Table 6.1. How these threshold values were chosen, together with the calibration evidence behind them, is set out in Section 6.5.

Table 6.1: Stage 6 monitoring signals and their conservative operating thresholds. None of these signals requires ground-truth labels; each indicates operational risk rather than verified error.

Signal	**What it measures**	**Warning threshold**
Mean confidence	Per-window decisiveness of the softmax output.	below 0.70
Uncertain-window rate	Share of windows whose confidence falls below the uncertainty level.	above 20%
Transition rate	How often the predicted class changes between adjacent windows.	above 35%
Drift (z-score)	Distance of incoming windows from the training-distribution baseline.	above 1.8
Dominant-class share	Fraction of windows assigned to the single most frequent class.	85% warn / 95% critical

The choice of signals reflects distinct failure modes that arise in label-scarce wearable monitoring, each invisible to at least one of the other signals.

Confidence is taken directly from the model's softmax output: for each window the predicted-class probability is its confidence, and a run is summarized by the mean confidence and by the share of windows whose confidence falls below an uncertainty level. This signal is the model's own native output, requires no labels, and is essentially free to compute. It is deliberately not trusted in isolation, because softmax confidence is frequently miscalibrated (Section 6.2.1); its role is to flag low-decisiveness runs and to contribute one vote to the trigger policy, not to certify correctness.

The transition rate measures how often the predicted class changes between adjacent windows. The target behaviors persist over several seconds while the windowing emits a new prediction every two seconds, so genuine activity should produce mostly stable neighboring predictions and frequent switching indicates an unstable output sequence. The transition rate is independent of confidence: a model can be confident on each individual window yet flip between classes from one window to the next, a pattern that a confidence threshold alone would miss.

Drift compares the incoming sensor windows against the training-distribution baseline using a per-channel z-score statistic. A z-score is used at runtime rather than a heavier two-sample test because it is lightweight, fast, and directly comparable across the six channels, which suits the offline-first, per-session setting; a more sensitive kernel-based statistic (MMD) is retained only as an offline audit branch (Section 6.5.2), since the runtime monitor needs to surface a shift for review rather than characterize it precisely. Drift can

detect a change in the input itself (a different device, placement, or unit convention) before that change appears as a collapse in the output [50, 52]. Two drift baselines are active in the system, one per serving path. The API path compares scaled windows against the normalized baseline inside the runtime-contract folder; the batch pipeline's Stage 6 uses a raw-physical-units baseline, matching the physical space in which its windows are audited. In both paths, if the baseline is missing or its feature space does not match the production windows, Stage 6 routes the condition to review rather than fabricating a drift result from incompatible spaces, consistent with the runtime-contract discipline of Chapter 4.

The dominant-class share is the fraction of windows assigned to the single most frequent class. It is tracked because dominant-class collapse is invisible to confidence: a collapsed model can assign almost every window to one class with high confidence. Measuring output concentration directly therefore catches a failure that the other signals would not, and it is the signal most directly tied to the collapse guard in Stage 7 (Section 6.3).

The formal definitions used throughout the chapter follow from this signal set. Let a session produce N windows. For window i the model outputs a softmax vector $\mathbf{p}_i = (p_{i,1}, \ldots, p_{i,K})$ over the $K = 11$ classes; the predicted class is $\hat{y}_i = \arg\max_k p_{i,k}$ and the per-window confidence is $c_i = \max_k p_{i,k}$. The mean confidence and the uncertain-window rate are

$$\bar{c} = \frac{1}{N} \sum_{i=1}^{N} c_i, \qquad U = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}[c_i < \tau_u] \,,$$

where τ_u is the uncertainty level; a run is flagged when $\bar{c} < 0.70$ or $U > 0.20$. A complementary uncertainty measure is the predictive entropy of each window,

$$H_i = -\sum_{k=1}^{K} p_{i,k} \log p_{i,k},$$

which is computed by the optional calibration stage rather than by the defended Stage 6 signal set; it is defined here for completeness because the calibration analysis of Chapter 7 draws on it. The transition rate, which captures temporal stability, is

$$T = \frac{1}{N-1} \sum_{i=2}^{N} \mathbf{1}[\hat{y}_i \neq \hat{y}_{i-1}] \,,$$

flagged when $T > 0.35$. This rate is computed on the raw per-window prediction stream so that it remains a faithful instability signal. In the serving path, the activity sequence returned to the operator is additionally passed through a majority-vote temporal smoothing over neighboring windows, the decision stage of the activity-recognition chain [6, 63], which suppresses isolated single-window flips without altering the monitored value of T. With class counts $n_k = \sum_{i=1}^{N} \mathbf{1}[\hat{y}_i = k]$, the dominant-class share is

$$D = \frac{1}{N} \max_k n_k,$$

with a warning at $D \geq 0.85$ and a critical level at $D \geq 0.95$. Distribution drift standardizes each channel statistic against the training baseline: if channel j has baseline mean μ_j and

standard deviation σ_j and the incoming session has channel mean $\hat{\mu}_j$, the drift score is

$$z = \max_j \frac{|\hat{\mu}_j - \mu_j|}{\sigma_j},$$

flagged when $z > 1.8$. Because z compares per-channel session *means* only, it is intentionally blind to variance and higher-moment shift: a session could match all six channel means while differing in amplitude spread or spectral content and still produce $z \approx 0$. This blind spot is accepted at runtime for the sake of a cheap, interpretable statistic; the offline MMD audit branch exists precisely to cover it. The optional audit branch instead measures the Maximum Mean Discrepancy between a reference sample $X = \{x_i\}_{i=1}^n$ and an incoming sample $Y = \{y_j\}_{j=1}^m$ under a kernel $k(\cdot, \cdot)$,

$$\mathrm{MMD}^2(X, Y) = \frac{1}{n^2} \sum_{i,i'} k(x_i, x_{i'}) + \frac{1}{m^2} \sum_{j,j'} k(y_j, y_{j'}) - \frac{2}{nm} \sum_{i,j} k(x_i, y_j),$$

which is more sensitive but computationally heavier, and is therefore reserved for offline auditing rather than per-session runtime use [53].

6.2.1 Confidence Limits and the Calibration Gap

Confidence is a useful but limited signal. A high softmax probability suggests that the model is decisive, but it does not guarantee that the prediction is correct, because modern neural networks are frequently poorly calibrated [17, 49]. This limitation is not hypothetical in this thesis. On the external User 7 probe, the model produced a mean softmax confidence of approximately 0.86 while its labeled accuracy (reported in Chapter 7) was far lower. The dominant failure mode is therefore *high-confidence misprediction*, not low-confidence uncertainty.

This has a direct design consequence: a single threshold on confidence cannot catch a confident-but-wrong session. If confidence were the only monitored signal, such a run would pass silently. For this reason the monitor combines confidence with temporal, drift, and dominant-class signals, and the trigger policy requires agreement across signals rather than trusting any one of them. Bayesian uncertainty approximations such as Monte Carlo dropout [18] provide a complementary view of predictive instability and are treated as an optional extension rather than as a defended runtime property.

6.3 Trigger Policy and Governance

Stage 7 converts the Stage 6 signals, together with collapse evidence, into a single governance action. The collapse guard is evaluated first: if a collapse condition holds, such as a dominant-class share at or above 95%, or a high-confidence single-class pattern with no temporal variation, the action is review with `should_retrain=false`, regardless of the other signals. If no collapse is detected, the confidence, temporal, and drift signals are scored and aggregated by voting under an explicit no-auto-retrain policy. Any critical signal

escalates to review; two or more warnings route the session to a review and label queue; a single warning is logged for monitoring; otherwise the run is healthy. Consecutive-warning escalation promotes repeated warnings to review without ever setting an automatic trigger, and a cooldown gate is applied last to prevent rapid retraining loops. Table 6.2 summarizes this mapping, and Figure 6.2 shows the overall flow.

The rationale for this design is that no single signal is trustworthy on its own, so the policy is built around agreement rather than any one threshold. Voting lets a confident-but-wrong run still be caught when temporal instability or drift disagrees with the confidence signal, while requiring more than one warning before escalation keeps the false-alarm rate low. The thresholds are deliberately tiered, with monitoring warnings firing earlier and the trigger applying stricter limits, so that risk becomes visible well before any mutation is even considered. Above all the policy is asymmetric: it is easy to raise a review, while by construction no code path converts signal severity into automatic mutation without the label and quality-assurance gate, because retraining from unlabeled, shifted, or collapsed predictions can reinforce the model's own errors. The benefit of the trigger layer is therefore measured not by how often it retrains, but by how reliably it routes genuinely risky sessions to human review while leaving the model unchanged.

Formally, after the collapse guard, let w and c be the numbers of warning- and critical-level signals among confidence, temporal, and drift. The governance action is

$$a = \begin{cases} \texttt{review} & \text{collapse guard holds, or } c \geq 1, \\ \texttt{queue_retrain} & c = 0 \text{ and } w \geq 2, \\ \texttt{monitor} & c = 0 \text{ and } w = 1, \\ \texttt{none} & w = c = 0, \end{cases}$$

with `should_retrain=false` enforced for unlabeled production data in every case, and a final cooldown gate that rewrites a would-be retraining candidate into a queued review.

Table 6.2: Stage 7 trigger decisions under the no-auto-retrain policy. Severity never authorizes mutation on its own; unlabeled production data cannot trigger automatic retraining.

Signal pattern	**Governance action**
Collapse condition holds	`review`; `should_retrain=false` (checked first, overrides voting).
Any critical signal	`review` required; no trigger.
Two or more warnings	`queue_retrain` (review/label queue; `label_required=true`); no immediate mutation.
One warning	`monitor`; no trigger.
No warnings	Healthy; no action.
Cooldown active (final gate)	Decision rewritten to `queue_retrain`; `should_trigger=false`.

Stage 6 and Stage 7 thresholds are intentionally tiered: the monitoring warning thresholds are earlier and looser, while the trigger thresholds are stricter governance limits (for example, the trigger treats confidence below 0.65 as a warning and below 0.50 as critical, drift z-scores above 2.0 and 3.0 as warning and critical, and applies a 24-hour

cooldown). The two stages share signal names but serve different purposes, so their thresholds differ deliberately: Stage 6 is tuned for *sensitivity*, because its warnings only create visibility in session reports and a false warning costs little, while Stage 7 is tuned for *specificity*, because its output feeds the governance decision and a false trigger consumes review capacity. The trigger levels follow a sigma rationale on the training distribution (a drift z of 2.0 corresponds roughly to the 95th percentile and 3.0 to the 99.7th [64]), and the voting and cooldown defaults are justified empirically by the calibration study reported in Section 7.5.2. Figure 6.1 visualizes this tiering for the three voting signals.

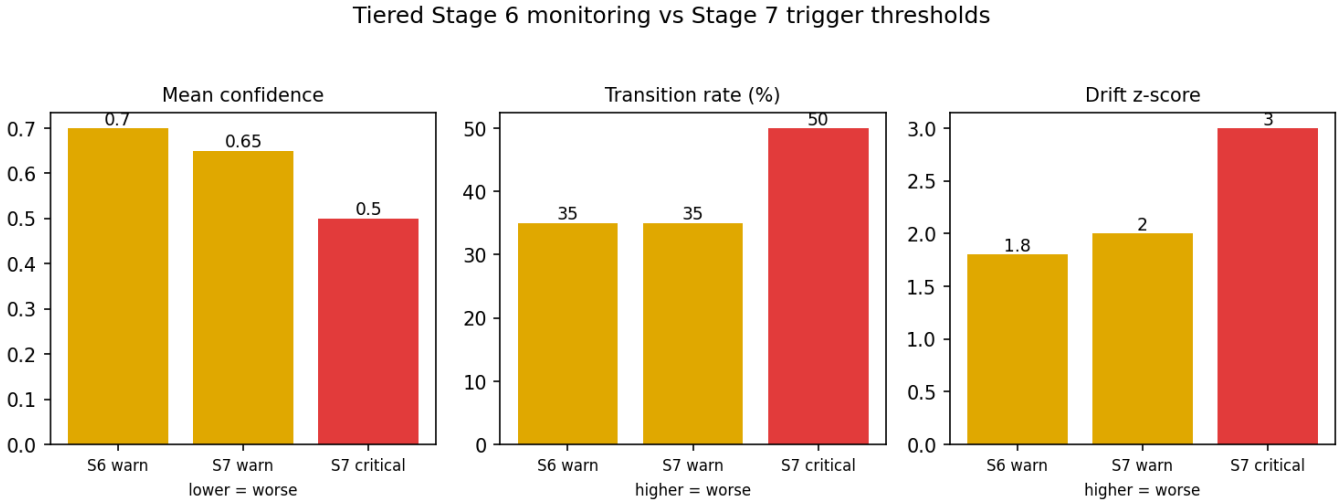


Figure 6.1: Tiered thresholds for the three trigger-voting signals. For each signal, Stage 6 (monitoring) warns earlier and looser than Stage 7, which adds a stricter warning and a critical limit before any governance vote is cast.

The tiering is deliberate and follows the same logic for all three signals. For confidence, Stage 6 warns when the session mean drops below 0.70, while Stage 7 registers a governance warning only below 0.65 and escalates to critical below 0.50. For the temporal transition rate, Stage 6 flags instability at 35%, while Stage 7 adds a critical boundary at 50% before casting a vote. For distribution drift, Stage 6 raises a visibility flag at $z > 1.8$, below the Stage 7 governance warning at $z > 2.0$ and the critical level at $z > 3.0$. The gap between the Stage 6 and Stage 7 warning boundaries is the sensitivity buffer: a run sitting between the two levels is visible in session reports without consuming governance review capacity.

Stage 7 converts the per-layer signals from Stage 6 into a single governance action through a four-step evaluation sequence. A collapse guard is applied first to catch cases where the model output has concentrated on one class regardless of the individual signal levels; the remaining signals are then aggregated by a voting rule across L1–L3; persistent low-level risk is escalated when the same single-warning pattern repeats across consecutive runs; and a cooldown gate prevents duplicate triggers within 24 hours of a recent retrain. The full path, from the Stage 4 softmax output through Stage 6 monitoring to the final governance action, is illustrated in Figure 6.2.

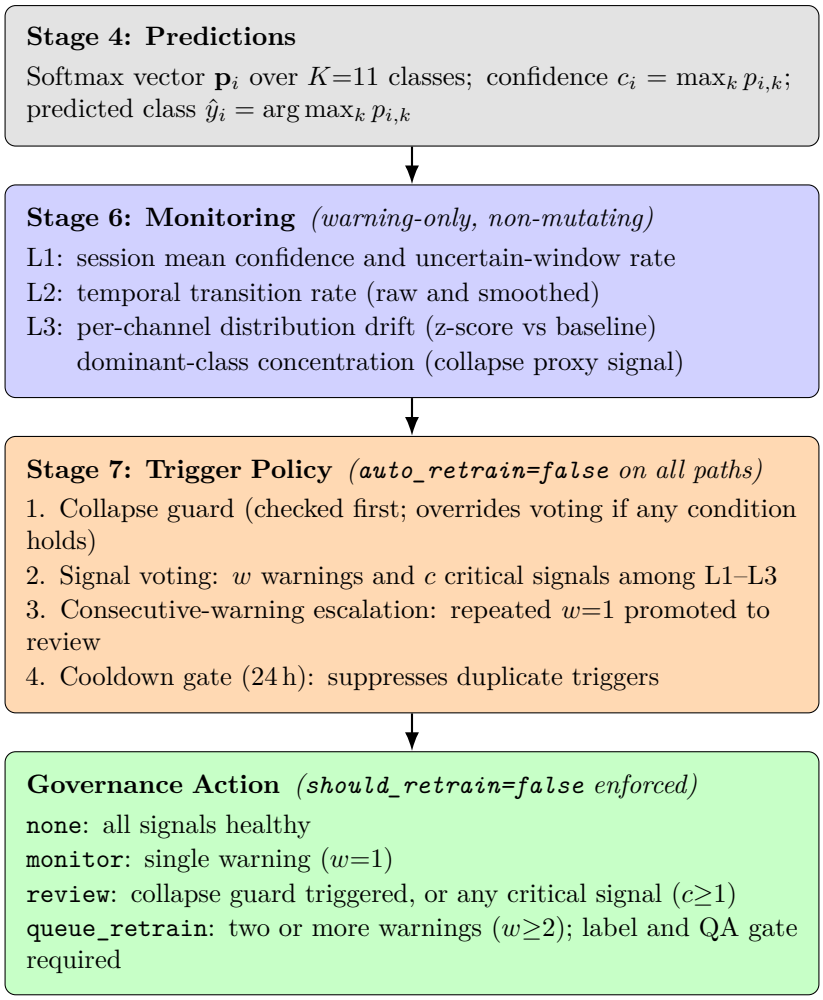


Figure 6.2: Post-inference monitoring and governance flow. Stage 6 evaluates three signal layers and a dominant-class collapse proxy; Stage 7 converts the signals into a governance action through a collapse guard, signal voting, consecutive-warning escalation, and a cooldown gate. Signal severity never authorizes automatic mutation: unlabeled production data routes to review or a label queue only.

Both execution paths write their outputs to MLflow under two nested experiments. The inference child run records the mean prediction confidence, the total window count, and individual prediction counts for each of the eleven activity classes, together with the model content hash and the runtime mode. The pipeline parent run aggregates the signals that drive the trigger decision: mean confidence, dominant-activity share, distribution drift score, temporal transition rate, an overall health flag, the alert severity level, and the retraining decision flag. Separating the two runs lets a reviewer inspect the raw per-class prediction counts from the inference stage independently of the aggregated signals the trigger policy actually consumes. Neither run logs model weights or training data; the only artifact written is the monitoring report, keeping the tracking store lightweight and audit-focused rather than a data archive. Annotated screenshots of both experiments, including the run list, metric tab, and logged artifacts for each path, are provided in Appendix E.

6.3.1 Collapse Guard and Cooldown Semantics

The collapse guard and the cooldown gate are the two mechanisms that keep the policy conservative in practice. The collapse guard is evaluated before the signal voting and returns a review action whenever any of the following conditions holds: an upstream dominance warning is raised; the dominant-class share reaches at least 95%; the dominant-class share reaches at least 90% together with a zero transition rate and high confidence; Stage 4 reports a critical-collapse level; or Stage 6 flags a low-motion collapse risk. In every one of these cases the decision is `review` with `should_retrain=false`, never automatic retraining, because a confidently collapsed single-class output is exactly the pattern that the confidence signal alone can miss.

The cooldown gate is applied last and has precedence over same-cycle escalation. If a decision would otherwise generate a retraining candidate while a recent retrain is still within the cooldown window (24 hours by default), the final decision is rewritten to a queued review with `cooldown_active=true`. The gate exists because consecutive sessions from the same wearer and device tend to share the conditions that raised the first alarm: without a cooldown, every such session would raise a near-duplicate retraining request, flooding the human review queue faster than it can be processed while adding no new information. This is particularly relevant in the offline-first setting, where multiple sessions from the same wearer may be processed in close succession and can reflect the same unresolved condition, such as a sensor placement issue, a domain shift, or an unusual movement style, rather than independent failures. Suppressing the duplicate trigger is safe because the underlying review remains open until a human closes it. The calibration study of Section 7.5.2 quantifies this effect: the 24-hour cooldown reduces trigger volume from 164 to 14 on the recorded history while keeping precision at 1.0. Enforcing the cooldown after the escalation logic ensures that it cannot be overridden later in the same evaluation cycle. The collapse guard and the cooldown gate jointly guarantee that neither a single collapsed batch nor a rapid sequence of back-to-back runs can drive the system into automatic mutation.

6.3.2 Severity Is Not Retraining: The Governance Boundary

A central claim of this thesis is that monitoring severity and model mutation are two separate decisions. A critical monitoring signal indicates observed risk above the warning threshold; it does not prove that the model is wrong, and it does not prove that retraining would improve the system. On both defended evidence runs the governance outcome is `action=review` with `should_retrain=false`: the run is flagged for human review, but no mutation stage is entered. This is a deliberate governance policy, not a missing feature.

The justification is strongest precisely in the label-scarce setting. If the system retrained directly from its own predictions on unlabeled, shifted, or collapsed data, it could reinforce its own errors through confirmation bias [55, 56]. The policy therefore requires that unlabeled production data never triggers automatic retraining regardless of signal severity; a label and quality-assurance gate is required before any retraining candidate is even generated.

6.3.3 Retraining and Baseline Governance

The mutation path (Stages 8–10) is reached only after the governance conditions are satisfied. Stage 8 generates a retraining candidate, Stage 9 registers it with provenance, and Stage 10 promotes it to the active runtime contract. Each step is conditional on review, labels, and validation evidence, and the cooldown gate prevents back-to-back retraining loops by rewriting a would-be trigger into a queued review when a recent retrain is still within the cooldown window. If a candidate fails validation or later proves unsuitable, the previous runtime package remains available as a rollback reference. This keeps mutation reviewable and reversible rather than automatic, consistent with the same-folder runtime-contract rule of Chapter 3.

6.4 Evaluation Metrics: Labeled and Unlabeled Conditions

The metrics available to the system depend on whether labels are present. Under labeled evaluation, the pipeline can compute accuracy, macro-F1, and a confusion matrix, which are reported in Chapter 7. Under unlabeled deployment-style operation, only proxy metrics are available: mean confidence, uncertain-window rate, transition rate, dominant-class share, and drift distance. These proxy metrics indicate operational risk but not verified correctness, in the same spirit as out-of-distribution and misclassification-detection signals [51, 54].

Table 6.3: Labeled evaluation metrics versus unlabeled proxy monitoring metrics. Labeled metrics measure correctness directly; proxy metrics indicate operational risk without ground truth.

Metric	**Labels?**	**What it indicates**
Accuracy	Yes	Overall correctness of predictions.
Macro-F1	Yes	Balanced per-class performance, sensitive to minority classes.
Confusion matrix	Yes	Per-class confusions and where collapse concentrates.
Mean confidence	No	Decisiveness of the output, not its correctness.
Uncertain-window rate	No	Proportion of low-confidence windows.
Transition rate	No	Temporal instability of the predicted sequence.
Dominant-class share	No	Output concentration and collapse risk.
Drift (z-score)	No	Input-distribution shift from the training baseline.

The combination is deliberate because the individual signals answer different questions and none is sufficient alone. A model can be confidently wrong (high confidence, low accuracy), poorly calibrated without obvious drift, drifted without an immediate collapse in the output distribution, or temporally unstable while remaining confident per window. Treating monitoring as a multi-signal layer rather than a single threshold on softmax confidence is therefore a design requirement, not an embellishment.

6.5 Threshold Calibration and Drift Auditing

The monitoring and trigger thresholds follow a hybrid selection strategy. Distribution-shift thresholds are grounded in statistical process-control principles, while confidence, temporal-stability, and dominant-class thresholds were calibrated from the recorded session pool. Neither family constitutes a clinical limit or a population-level statistical guarantee.

The drift thresholds derive from the control-limit convention of statistical process control [64]: deviations of 2σ from a Gaussian reference distribution correspond to the outer 4.55% of expected variation and serve as a warning boundary, while 3σ covers the outer 0.27% and marks a critical boundary. Applying these limits to the per-channel mean-shift score $z = \max_j |\hat{\mu}_j - \mu_j|/\sigma_j$ gives the Stage 7 thresholds of $z > 2.0$ (warning) and $z > 3.0$ (critical), where σ_j is the per-channel training standard deviation [50]. Stage 6 uses the lower boundary $z > 1.8$ because its warnings carry no governance weight and a false alarm costs nothing beyond a log entry; an earlier detection boundary is therefore appropriate for visibility. One caveat applies: because the statistic takes the maximum across six IMU channels, the false-alarm probability is higher than for a single-channel test, so the thresholds should be read as conservative monitoring control limits rather than as formal per-channel statistical tests [62].

6.5.1 Z-Score Severity Levels and Their Statistical Basis

To make the severity mapping explicit, Table 6.4 summarizes the three threshold boundaries and their statistical interpretation. The 2σ and 3σ limits are not arbitrary engineering choices; they are the standard control-chart boundaries established by Page [64] and widely adopted in machine-learning data-drift monitoring [50, 62]. Under a Gaussian reference distribution, a z-score below 1.8 means the incoming channel means fall well within the expected variation of the training population. A z-score between 1.8 and 2.0 is an early visibility flag: the session is near the outer boundary but has not yet crossed the formal 2σ warning limit. A z-score above 2.0 means that fewer than 4.55% of sessions drawn from the same training distribution would be expected to deviate this far, making it a statistically meaningful warning. A z-score above 3.0 means that fewer than 0.27% of training-distribution sessions would reach this level, which is the conventional critical boundary in statistical process control.

This tiered design means that a session can be visible in monitoring reports before it crosses any governance boundary, and only genuinely extreme drift triggers a review action. The statistical process-control origin of the thresholds also makes them interpretable to an external reviewer without additional calibration data: a 3σ drift event is unexpected under any reasonably Gaussian distribution, regardless of the specific domain.

Confidence thresholds are calibrated operating limits, not probability guarantees. Softmax outputs of deep neural networks are frequently poorly calibrated, sometimes with high confidence on incorrect predictions [17, 49]; correctly classified examples tend to have higher maximum softmax probabilities than misclassified or out-of-distribution ones, but the converse does not hold [51]. This is not an abstract concern here: on the external

Table 6.4: Z-score severity levels used in this pipeline and their statistical basis. Thresholds follow the $2\sigma/3\sigma$ control-limit convention of statistical process control [64, 50].

Z-score range	**Severity**	**Meaning**	**Stage action**
$z \leq 1.8$	Healthy	Channel means within the expected training variation.	No flag.
$1.8 < z \leq 2.0$	Early warning	Approaching the 2σ boundary; fewer than 5% of training sessions deviate this far.	Stage 6 visibility flag only; no governance vote.
$2.0 < z \leq 3.0$	Warning (2σ)	Outer 4.55% of the training distribution. Statistically meaningful shift in at least one channel.	Stage 7 casts one warning vote.
$z > 3.0$	Critical (3σ)	Outer 0.27% of the training distribution. Severe distributional departure.	Stage 7 escalates directly to `review`.

User 7 probe, mean confidence was approximately 0.86 while labeled accuracy was far lower. With $K = 11$ classes and a random-guess baseline of $1/K \approx 0.09$, the warning levels of 0.70 (Stage 6) and 0.65 (Stage 7) represent decisiveness floors rather than correctness thresholds, and 0.50 marks the critical level at which mean confidence is barely five times the chance baseline. The uncertain-window rate adds a complementary check: if more than 20% of individual windows fall below the per-window cutoff of 0.50, the session is unreliable even if the mean confidence sits just above the warning level. Across the curated Garmin session pool, healthy in-distribution runs stayed below 10% on this metric, placing the 20% flag well above normal operation.

The temporal transition rate and dominant-class thresholds were both derived from the windowing structure and confirmed on the recorded session pool. At a stride of two seconds, a session with three to four genuine activities produces natural class transitions at roughly 15–25%; the 35% warning marks the rate above which predicted-class switches are more consistent with output instability than genuine behavioral change, which runs counter to the expectation that sustained wrist activities produce stable neighboring predictions [6, 63]. Dominant-class concentration follows the same logic: a healthy model handling one dominant activity might legitimately assign 70–80% of windows to a single class, so the 85% boundary is just above that plausible ceiling. The 95% level is the collapse signature: a model assigning 95 of every 100 windows to one class regardless of motion is not distinguishing eleven activities, and high softmax confidence on such an output is exactly the failure that a confidence threshold alone cannot catch [51]. Both boundaries were confirmed on the session pool: healthy runs stayed below 85%, accuracy-collapsed runs crossed 95%. The 24-hour cooldown is a governance deduplication parameter: replaying the trigger policy over the recorded session history reduced trigger volume from 164 to 14 events while keeping precision at 1.0, confirming that suppressed candidates were genuine duplicates (Section 7.5.2).

All prediction-side thresholds were calibrated against the same session pool that forms the Chapter 7 evaluation, so those results demonstrate the policy within its calibration domain rather than generalizing to unseen deployments. The thresholds are conservative

operating limits: not clinical thresholds, not population-level statistical guarantees, and not validated wearable-HAR constants. Longitudinal recalibration against a larger and more diverse session pool would be needed before any stronger claim could be made, and this gap is revisited as future work in Chapter 8.

6.5.2 Distribution-Sensitive Drift Audit

Beyond the lightweight runtime drift monitor, a distribution-sensitive audit branch based on Maximum Mean Discrepancy (MMD) can compare incoming windows against a reference sample using a kernel-based two-sample statistic [53]. This branch is treated as an offline audit that improves the *visibility* of degraded batches; it is not promoted into the runtime mutation path. A chunked Population Stability Index check is computed in the same spirit in the production pipeline as a secondary drift diagnostic; like the MMD branch, it improves drift visibility but does not feed the Stage 7 trigger vote, which relies on the per-channel z-score. The distinction matters for the governance argument: better drift visibility does not by itself justify retraining, so a more sensitive drift statistic and a mutation decision are evaluated separately rather than conflated. The corresponding experimental evidence is reported in Chapter 7.

6.6 Chapter Summary

This chapter described the monitoring and governance layer that operates on top of the implemented pipeline. Stage 6 rates a run using confidence, temporal, drift, and dominant-class signals against conservative thresholds; Stage 7 converts those signals into a single governance action through a collapse guard, signal voting, escalation, and a cooldown gate. The central principle is that monitoring severity is separated from model mutation: a critical signal routes to review, and unlabeled production data never triggers automatic retraining. The calibration gap observed on the external probe (high confidence at modest accuracy) motivates the multi-signal design, because a single confidence threshold cannot catch a confident-but-wrong session. Chapter 7 evaluates these monitoring and trigger behaviors empirically.

Chapter 7

Experiments and Evaluation

The previous chapters defined the data and model contract (Chapter 4), the implementation (Chapter 5), and the monitoring and governance logic (Chapter 6). This chapter reports the empirical evidence. Its purpose is not to claim a high final accuracy, but to measure what the proposed model actually does under three conditions of increasing difficulty: within-distribution validation, a blind external probe, and label-scarce monitoring. The central observation is a large and consistent gap between within-distribution learnability and blind-probe performance. This gap is reported openly because it is the empirical motivation for the runtime-contract, monitoring, and governance contribution of this thesis.

7.1 Evaluation Setup and Evidence Boundary

All results in this chapter are produced under the fixed runtime contract of the proposed model defined in Chapter 4: a six-channel IMU input of shape $(200, 6)$, a 50 Hz sampling rate, a 200-sample window with a 100-sample stride, the eleven-class label order, and the train-only Garmin scaler. No result in this chapter changes any architecture, hyperparameter, augmentation, threshold, or acceptance decision after observing the held-out or blind data. The provenance of the numbers is also fixed: the model-building results were produced in the version-controlled build notebook that constructs and freezes the runtime contract, while the pipeline of Chapter 5 consumes that frozen contract for inference, evaluation, monitoring, and governance; Appendix C maps every reported number to its artifact.

7.1.1 Evidence Boundary

The data sources are not interchangeable, and each can support only a specific kind of claim. Labeled Garmin users 1–6 support accuracy, macro-F1, and confusion analysis. The blind User 7 session supports a single descriptive external measurement, excluded from training, scaler fitting, hyperparameter selection, and model selection. The unlabeled session supports monitoring, confidence, drift, and trigger-policy evidence, but not direct

accuracy claims, because ground-truth labels are unavailable at inference time. This boundary is enforced throughout the chapter so that no source is used beyond what it can prove.

7.1.2 Fixed Runtime Contract and Evaluation Protocols

The proposed model contains 967,499 parameters and is fine-tuned from an ADAMSense-pretrained base on Garmin users 1–6. The five-fold protocol is a window-level split with the scaler fitted on users 1–6 before splitting; it measures within-distribution learnability and is, by construction, optimistic relative to deployment on a new wearer. It uses the 3,755 contiguous-activity windows from Garmin users 1–6 [28, 6, 39].

Augmentation is applied only at training time and only to the training split of each fold. The three-fold augmentation (time-warping, magnitude scaling, and jitter) expands the training set; it is not an additional input feature and does not change the six-channel input contract from Chapter 4. Balanced class weights are used to counter class imbalance. The fixed random seed is recorded with each run for reproducibility.

7.1.3 Evaluation Metrics

Recognition is summarized with accuracy and macro-averaged F1. Accuracy over N windows with predicted labels $\hat{y}_i$ and true labels y_i is

$$\mathrm{Acc} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}[\hat{y}_i = y_i]\,.$$

For each class k, precision and recall are

$$P_k = \frac{TP_k}{TP_k + FP_k}, \qquad R_k = \frac{TP_k}{TP_k + FN_k},$$

and the per-class F1 and macro-averaged F1 over $K = 11$ classes are

$$F1_k = \frac{2P_kR_k}{P_k + R_k}, \qquad \mathrm{MacroF1} = \frac{1}{K}\sum_{k=1}^{K} F1_k.$$

Macro-F1 is reported alongside accuracy because the eleven activity classes are imbalanced, and accuracy alone can hide minority-class collapse. Calibration is assessed separately in Section 7.3.

7.2 Recognition Performance: Within-Distribution and Blind Probe

Two labeled evaluations were conducted. Under five-fold window-level cross-validation on Garmin users 1–6, the model reaches 93.61% mean accuracy and 0.937 macro-F1. On

the blind User 7 probe, evaluated exactly once after the model was frozen, accuracy falls to 36.96% (macro-F1 0.350), a 57-point gap that is the central empirical finding of this chapter. The subsections below describe each result and what it can, and cannot, be taken to mean.

7.2.1 Within-Distribution Five-Fold Cross-Validation

The per-fold breakdown is given in Table 7.1. Fold-level variance is low: all five folds fall within one percentage point of the 93.61% mean, which confirms the result is not driven by a single favorable split (Appendix D, Figure D.1). This is in the same range as the published Garmin HAR benchmark of 87.0% ± 1.2%, obtained under a comparable five-fold protocol [8]; the augmentation recipe and scaler policy differ between the two studies, so the comparison indicates consistency of magnitude rather than a like-for-like ranking. The result confirms the architecture and preprocessing contract are sound for the labeled training distribution.

This number is reported as a within-distribution measurement and not as a deployment estimate. Because the scaler is fitted on all six users before the windows are split, because windows from the same wearer appear in both training and validation folds, and because adjacent windows overlap by 50% so that windows sharing half their raw samples can land in different folds, the protocol measures how well the model fits the known data rather than how well it transfers to a new person. The next subsection makes this distinction explicit.

Table 7.1: Five-fold window-level cross-validation on Garmin users 1–6 (751 windows per fold).

Fold	**Accuracy**	**Macro-F1**	**Windows**
1	93.34%	0.934	751
2	94.41%	0.945	751
3	94.14%	0.942	751
4	93.48%	0.935	751
5	92.68%	0.928	751
Mean / Total	**93.61%**	**0.937**	3,755

7.2.2 Blind User 7 External Probe

User 7 is a self-recorded session excluded from every model-development decision (training, scaler fitting, hyperparameter selection, and model selection) and evaluated exactly once after the model was frozen. On its 989 windows, accuracy is 36.96% (macro-F1 0.350), with a 95% bootstrap confidence interval of $[33.9\%, 39.8\%]$. This is reported as a single descriptive external measurement, not a selection or validation metric.

The degradation is expected. The session is self-recorded and self-labeled, so watch placement, wrist orientation, and gesture execution style vary relative to the training

population, on top of the natural cross-user distribution shift. The eleven activities here are predominantly fine-contact hand-to-face gestures (nail biting, smoking, hair pulling, ear rubbing, nape rubbing, hand scratching) whose wrist-sensor signatures differ mainly in arm angle, contact force, and timing, all of which are person-specific. A six-user training set cannot cover this variation. Activities with a wearer-stable IMU signature transfer better: hand tapping is recognized at 91% even on User 7, while most fine-contact gestures fail. The cross-user gap, not model capacity, is the binding constraint.

What the result also exposes is that prediction confidence does not track accuracy. Mean confidence on the User 7 session is 0.858 despite accuracy of 36.96%. A model that is wrong the majority of the time while reporting high confidence is precisely the behavior a threshold on softmax confidence would miss, and it is the direct motivation for the multi-signal monitoring design of Section 7.3. Comparison with prior work must be read carefully: the ADAMSense study reports up to 92.16% on a single best-case held-out user using twelve-second windows and an additional magnitude feature [7], a different sensor setup and evaluation design, not directly comparable to a four-second six-channel blind probe on a different device. External cross-user validation is rarely reported in the wearable HAR literature, which is precisely the gap this probe addresses [4].

7.3 Calibration and the High-Confidence Failure Mode

The User 7 result exposes a calibration gap: prediction confidence and empirical correctness do not align. Modern neural networks are frequently poorly calibrated [17, 49], and in the label-scarce wearable setting this is not a minor issue but the dominant operational risk, because a shifted session can still produce high-confidence predictions. The dominant failure mode observed here is therefore high-confidence misprediction rather than low-confidence uncertainty.

Calibration is quantified with the expected calibration error, which compares accuracy and confidence across M confidence bins B_m,

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{N} \left|\mathrm{acc}(B_m) - \mathrm{conf}(B_m)\right|,$$

and visualized with a reliability diagram and confidence summaries in Figure 7.1. The reliability diagram shows the model sitting below the diagonal in the high-confidence bins, the signature of overconfidence. The bootstrap confidence interval reported for User 7 is a window-level bootstrap interval over the windows of a single session; it does not represent population-level uncertainty across users, and it should not be read as a generalization guarantee. Adjacent windows overlap by half, so the bootstrap samples are correlated and the interval is approximate; a block bootstrap over contiguous segments would yield a wider, more conservative interval.

This calibration gap is the direct justification for the multi-signal monitoring design of Chapter 6. A single threshold on softmax confidence would pass a confident-but-wrong

session silently, so the monitor combines confidence with temporal stability, dominant-class concentration, and input drift, and the trigger policy requires agreement across signals rather than trusting any one of them.

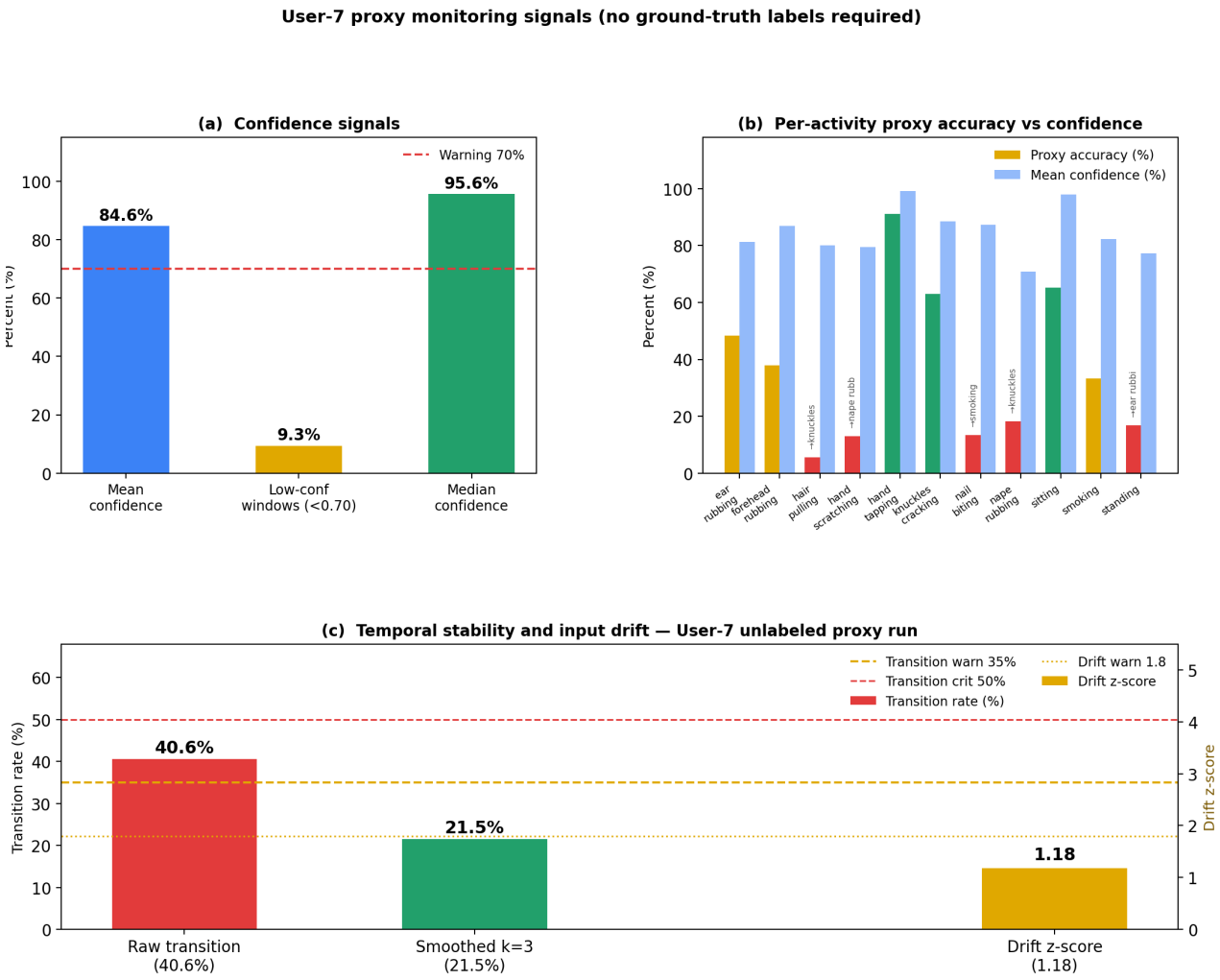


Figure 7.1: User-7 proxy monitoring signals from the unlabeled schedule-aligned run (no ground-truth labels). *(a)* Confidence summary: mean confidence 84.6%, low-confidence window rate 9.3%, and median confidence 95.6%. The 70% warning threshold is not breached by the mean. *(b)* Per-activity proxy accuracy (colored bars) vs. mean softmax confidence (blue, transparent) across all 11 activity segments. Hand tapping and sitting transfer well, while hair pulling, nail biting, and hand scratching collapse despite high confidence. *(c)* Temporal transition rate drops from 40.6% (raw) to 21.5% after majority-vote smoothing (k=3), falling below the 35% warning level. Input drift z-score is 1.18, below the 1.8 warning threshold.

Three key observations follow from these signals. First, mean confidence (84.6%) sits well above the 70% warning threshold even though proxy accuracy is only 36.70%, confirming that softmax magnitude is not a reliable proxy for correctness on this session. Second, the per-activity breakdown in panel (b) shows the error is concentrated: hand tapping achieves high proxy accuracy while gestures with similar hand-to-face trajectories (nail biting, hair pulling, hand scratching) collapse. Third, panel (c) shows that raw temporal instability is substantially reduced by smoothing, which means many prediction

flips are isolated single-window errors rather than sustained confusions. The drift z-score of 1.18 indicates the input distribution is shifted relative to the training baseline but not to a critical degree.

A per-class confusion matrix for User 7 (Figure 7.2) shows that the errors are structured rather than random. Hand tapping stands out as the reliably recognized activity, with 91% of its windows correctly classified and a mean confidence of 0.99, reflecting a rhythmic motion pattern that is relatively distinctive and stable across wearers. Most other activities produce systematic confusions concentrated on a small set of wrong classes: hair pulling is predicted as knuckles cracking in 48% of its windows; nail biting is predicted as smoking in 51% of its windows; hand scratching goes predominantly to nape rubbing; and standing is confused with ear rubbing. These confusions are not random; they cluster among gestures with similar hand-to-face trajectories, confirming that the model is misled by the specific execution style of User 7 rather than being uniformly confused. A small set of classes therefore absorbs a disproportionate share of the predictions from many input segments. This is the dominant-class concentration pattern that the monitoring layer is designed to surface: when the output is heavily skewed toward one or two wrong classes, confidence stays high while accuracy collapses, and only the dominant-class signal can catch it.



Figure 7.2: Per-class confusion matrix on the blind User 7 probe. Errors concentrate on a small set of gesturally similar wrong classes: hair pulling goes to knuckles cracking, nail biting to smoking, hand scratching to nape rubbing. Hand tapping is the exception, correctly classified at 91%.

A per-class ranking of the User 7 F1 (Appendix D, Figure D.2) confirms this pattern: a few activities, such as hand tapping, are recognized reliably, while several others receive an F1 close to zero. The macro-averaged F1 of 0.350 is therefore the average of a small number of well-recognized classes and many that the model fails on for this external user.

7.4 User-7 Proxy Metrics: What the Pipeline Sees Without Labels

The blind probe result in Section 7.2.2 was obtained with ground-truth labels. In deployment, labels are not available. This section addresses a practical question: can the pipeline reach the same conclusions about session quality using only what it can observe at inference time, namely per-window confidence scores, the predicted class sequence, and

input drift statistics?

To test this, the same User 7 recording was processed a second time as a schedule-aligned unlabeled run. The recording followed a scripted protocol in which one activity was performed per timed segment, so the known activity schedule provides proxy ground truth without requiring manual per-window annotation. Schedule-aligned accuracy is the fraction of windows whose predicted class matches the activity scheduled for that time interval. The labeled blind probe yields 36.96% accuracy (macro-F1 0.350) at mean confidence 0.858, while the schedule-aligned unlabeled proxy yields 36.70% accuracy (macro-F1 0.351) at mean confidence 0.846. The two conditions agree to within 0.3 percentage points on every metric, showing the monitoring signals track ground-truth session quality without requiring manual annotation.

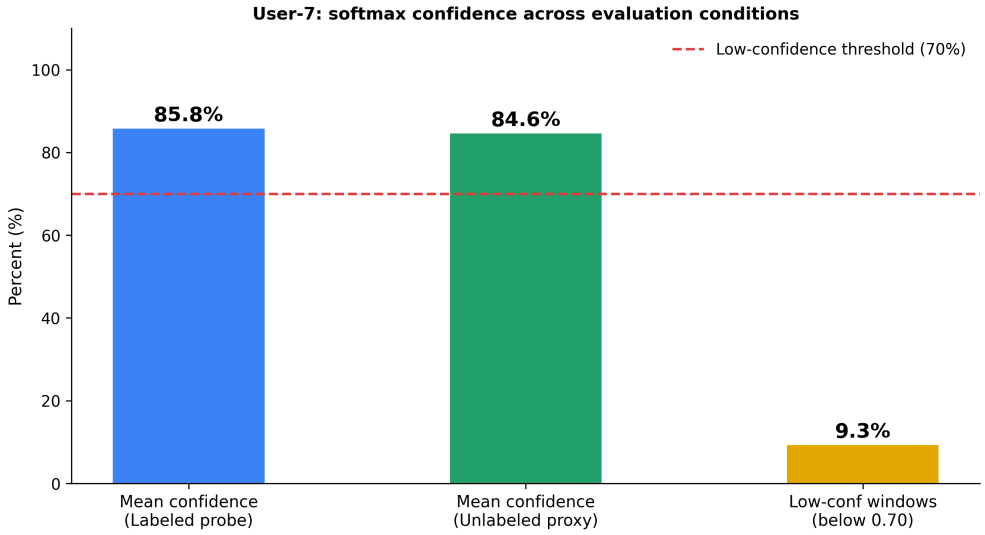


Figure 7.3: Softmax confidence for the User-7 session. The two leftmost bars show mean confidence under the labeled blind probe (85.8%) and the schedule-aligned unlabeled proxy (84.6%), both well above the 70% warning threshold. The rightmost bar shows that only 9.3% of unlabeled windows fall below the 0.70 confidence level, confirming the overconfidence pattern: the model produces high softmax outputs even when its predictions are predominantly wrong.

The agreement matters because the pipeline cannot rely on ground-truth labels in production. The confidence, temporal-transition, and dominant-class signals that Stage 6 computes from unlabeled windows carry the same diagnostic information as the labeled accuracy. Yet confidence alone is not sufficient: mean confidence is 0.846 on the unlabeled run while accuracy is only 36.70%. The model is systematically overconfident, so a threshold on softmax confidence alone would pass this session without any alert. The monitoring design of Chapter 6 addresses this by combining confidence with temporal stability and dominant-class concentration, requiring agreement across signals before escalating.

A further indicator available without labels is the low-confidence window rate. On the User-7 unlabeled run, 9.3% of windows have a softmax peak below 0.70. On the training distribution this rate is near zero. An elevated rate therefore signals, without any label,

that the session lies outside the model's confident operating region. Combined with the elevated temporal transition rate (40.6% raw), the pipeline has multiple label-free signals pointing to the same conclusion the labeled evaluation confirms.

Temporal smoothing offers a partial improvement. Passing per-window predictions through the majority-vote stage [6, 63] (soft vote over k=3 neighboring windows) raises schedule-aligned accuracy from 36.70% to 39.43% and macro-F1 from 0.351 to 0.373, and lowers the temporal transition rate from 40.6% to 21.5%, which falls below the 35% warning threshold (Figure 7.4). Mean confidence (0.846) and the drift z-score are unchanged by smoothing, since it only reorders predicted labels without touching the per-window softmax or input statistics.

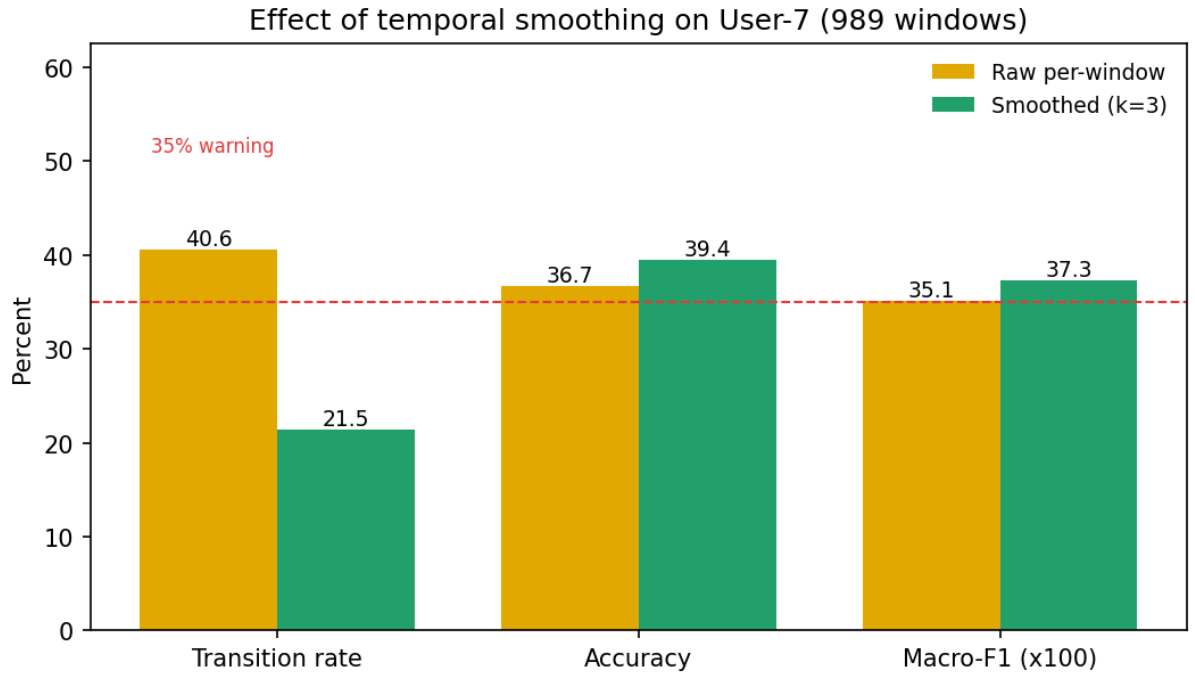


Figure 7.4: Majority-vote temporal smoothing on the User-7 session (989 windows). The transition rate falls from 40.6% to 21.5%, below the 35% warning level, and accuracy and macro-F1 each gain roughly two to three points. Confidence and drift are unaffected.

The accuracy improvement is real but modest, and the reason is structural. Smoothing corrects isolated single-window prediction flips, but the dominant errors on User-7 are sustained confusions across consecutive windows: hair pulling is predicted as knuckles cracking across an entire segment, nail biting as smoking, and hand scratching as nape rubbing. A three-window majority vote cannot resolve a confusion that persists for dozens of consecutive windows. On the training users, where the base predictions are already strong, the same smoothing raises accuracy from 95.7% to 97.9% because the residual errors there are mostly isolated flips. The contrast between the two settings shows that smoothing is effective for jitter but not for systematic cross-user confusion, which requires additional training data or a personalization step.

7.5 Monitoring and Trigger-Policy Evidence

Section 7.4 showed that the pipeline can track session quality without labels on a single reference session. This section extends that to scale: 25 recorded unlabeled sessions processed through Stages 1–7 under the same fixed runtime contract, and the trigger-policy and contract-sensitivity calibration that justifies the deployed defaults. On every defended evidence run the Stage 7 outcome is `action=review` with `should_retrain=false`: the session is flagged for human review, and no model mutation is triggered. This is the intended behavior: a label-scarce monitoring signal indicates operational risk, not verified error, and does not on its own justify changing the deployed model. The preprocessing and threshold sweeps reported alongside these runs are treated as contract-sensitivity evidence, showing that monitoring severity responds to controlled changes in preprocessing while the model artifact remains fixed.

7.5.1 Session-Level Monitoring Outcomes

To test the monitoring and governance layer at scale, the recorded unlabeled sessions in the repository were processed through Stages 1–7 under the same fixed runtime contract. The analysis covers the 25 sessions that processed successfully under the standard per-sample ingestion path; two recordings with unusable layouts (one empty accelerometer file, one stored in a legacy Excel format) are excluded. Figures 7.5–7.8 show each monitoring signal separately. Across the twenty-five sessions, prediction confidence stayed high: the average was 0.83 and no session fell below the 0.70 level, confirming that the overconfidence pattern from Section 7.3 persists in deployment-style data. Temporal transition rates were frequently elevated (mean 40.3%, 21 of 25 above the 35% warning threshold). Distribution drift was mostly contained (mean z-score 1.18) with one session exceeding the critical level. Dominant-class share flagged two sessions, one reaching 99.7%. The governance result is the central finding: the trigger policy routed **23 of the 25 sessions to review**, left one at monitor-only and one healthy, while authorizing automatic retraining on **none** of them.

This pattern is operationally important because the signals do not fail in the same direction. Confidence remains high and therefore contributes little to detection; transition rate surfaces instability in most sessions; drift identifies a smaller number of input-shift cases; and dominant-class share catches rare collapse. The review decision is therefore not a restatement of one chart, but the result of combining partially independent warnings under the no-auto-retrain governance rule.

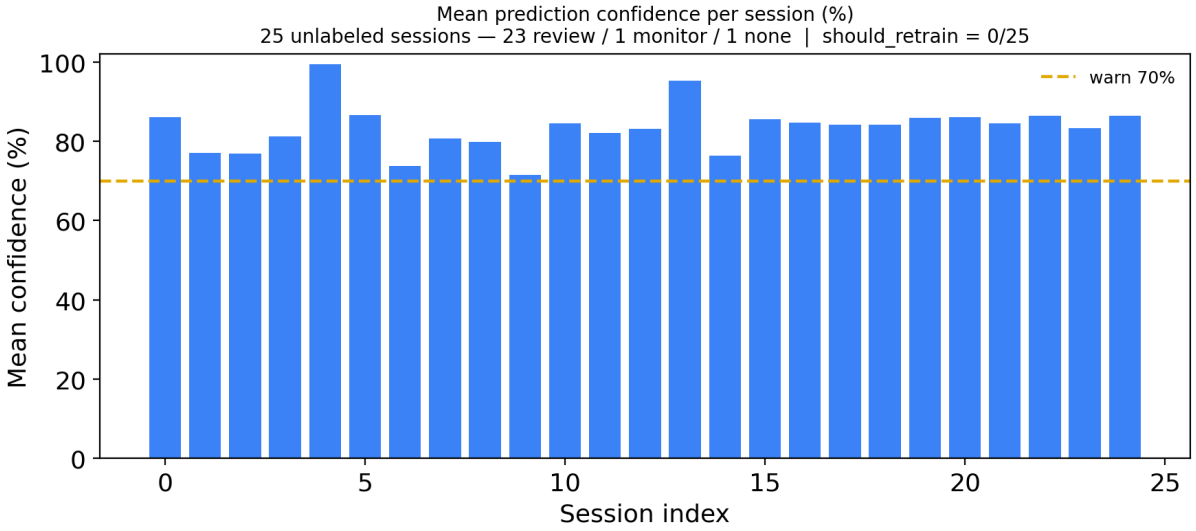


Figure 7.5: Mean prediction confidence per session across all 25 unlabeled sessions. Every session stays above the 70% warning threshold (dashed line), which is the overconfidence pattern: high softmax output persists even on sessions the model handles poorly, so a confidence-only filter would flag nothing.

Confidence alone is therefore not a useful discriminator here. The temporal transition rate tells a different story.

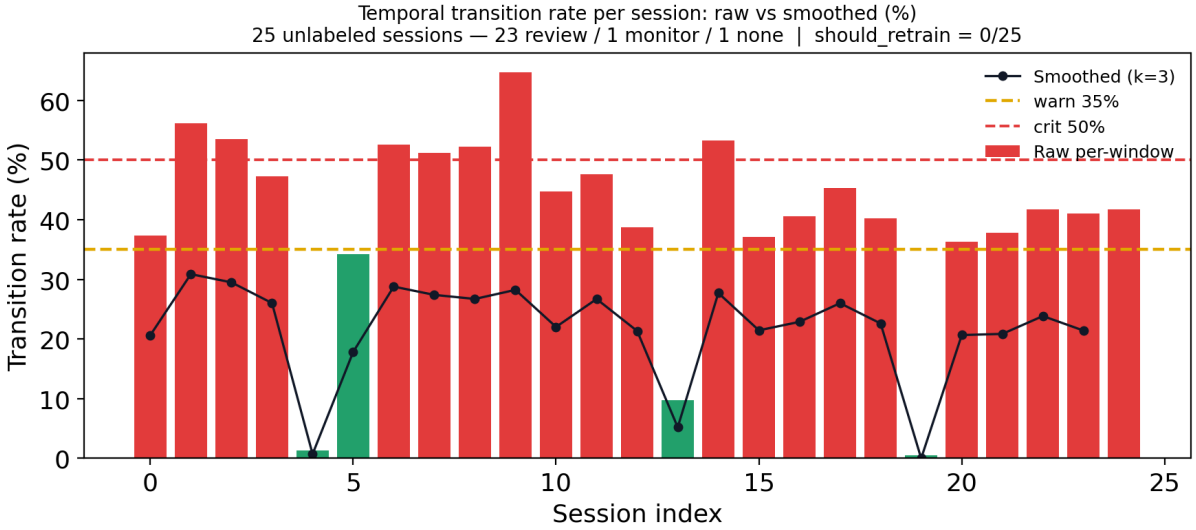


Figure 7.6: Temporal transition rate per session (raw per-window, bars) and after majority-vote smoothing k=3 (line). Twenty-one of twenty-five sessions exceed the 35% warning level on the raw stream; smoothing brings all of them below it, confirming the instability is predominantly single-window jitter. The monitoring system evaluates the raw stream; smoothing is a post-processing step applied afterward.

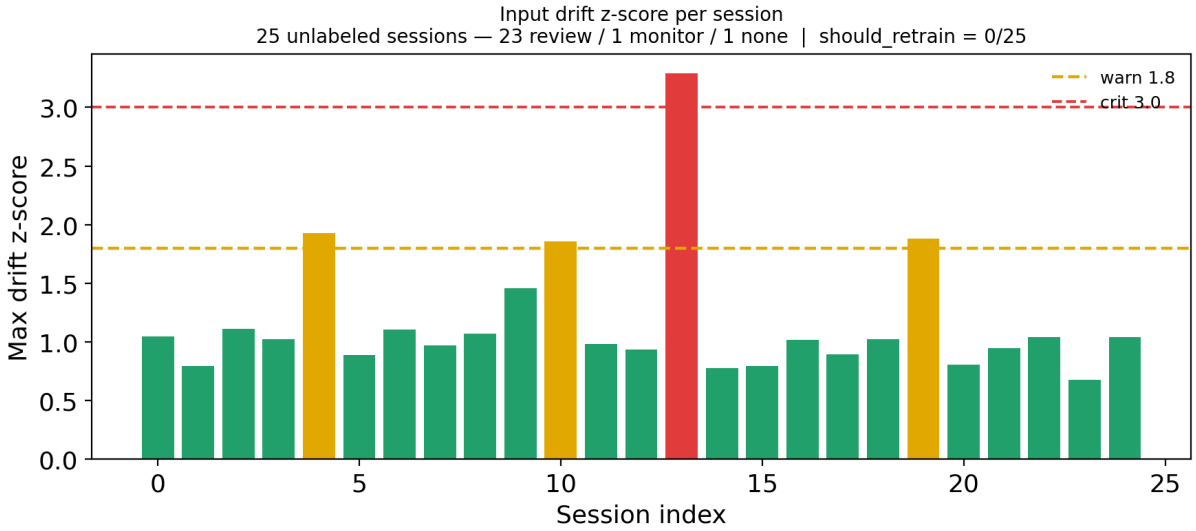


Figure 7.7: Input drift z-score per session. Drift is mostly contained (mean 1.18, below the 1.8 warning threshold) but four sessions reach warning level and one exceeds the critical threshold of 3.0. Drift contributes to the review routing for these sessions independently of the transition signal.

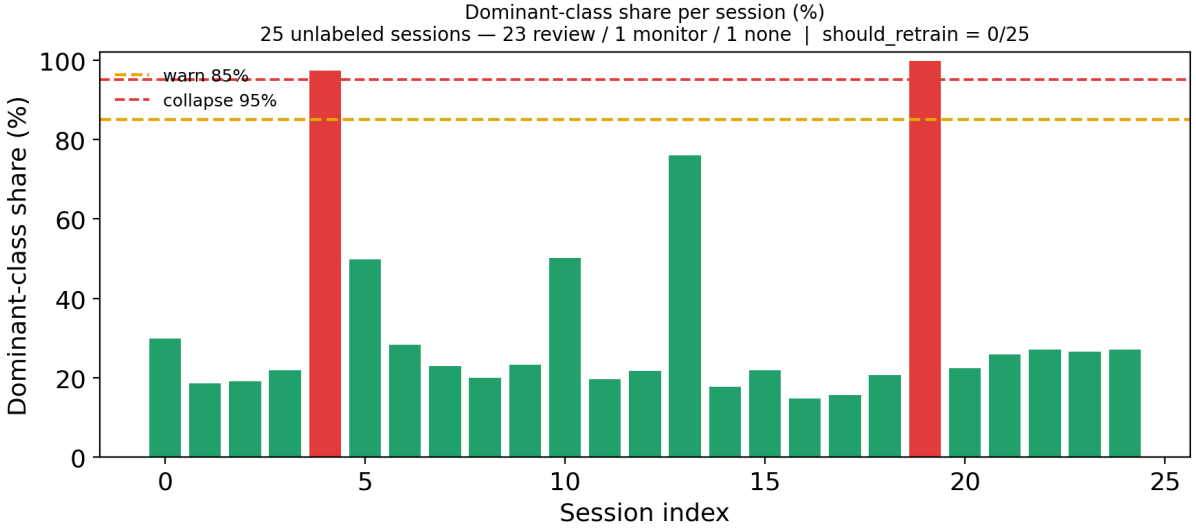


Figure 7.8: Dominant-class share per session, defined as the fraction of windows assigned to the single most predicted class. Two sessions exceed the 85% warning threshold, one reaching 99.7%. This near-total collapse onto a single output class is not detectable from confidence alone; the collapse guard in Stage 6 catches these cases.

Table 7.2 decomposes this outcome by trigger action and signal. Risk was surfaced predominantly by the temporal-transition signal (21 of 25 sessions above the 35% warning level, seven above the 50% critical level), with drift contributing four warning-level excursions including one critical, and the dominant-class signal flagging two sessions (one of which reached a 99.7% dominant share, a collapse-level concentration caught by the collapse guard). Confidence alone would have flagged nothing: every session stayed above the 0.70 warning level, which is the overconfidence pattern of Section 7.3 reappearing in deployment-style data. The `queue_retrain` escalation path never fired on any of the

twenty-five sessions: `should_retrain` was `false` throughout, so every review was reached through critical signals or the collapse guard rather than through warning accumulation; this is itself a finding about how risk manifests in these sessions. No code path converted any severity level into a model mutation, so the no-auto-retrain invariant of Chapter 6 held on real data, not only by construction.

Table 7.2: Decomposition of trigger actions and monitoring-signal incidence across the 25 unlabeled sessions.

Outcome / signal incidence	**Sessions**
Trigger action `review`	23
Trigger action `monitor`	1
Trigger action `none` (healthy)	1
Retraining queued (`should_retrain=true`)	0
Stage-6 severity CRITICAL / WARNING / HEALTHY	5 / 19 / 1
Transition rate above warning (35%) / critical (50%)	21 / 7
Drift z-score above warning (1.8) / critical (3.0)	4 / 1
Dominant-class share above 85% (maximum 99.7%)	2
Mean confidence below the 0.70 warning level	0

One apparent tension is worth addressing. Temporal smoothing (Section 7.4) brings the mean transition rate from 40.2% raw to 21.6% smoothed across the 25 sessions, below the 35% warning level on every session. If smoothing resolves the instability, why did 23 sessions trigger a review? The monitoring system always evaluates the raw prediction stream; smoothing is applied afterward in the serving path as a display aid for clinicians, not as input to the trigger policy. The review actions were based on the raw stream, where 21 of 25 sessions exceeded 35% transition. This separation is deliberate: monitoring the raw stream captures the model's instability as it occurs, whereas monitoring the smoothed stream would suppress the signal the system is designed to surface. Appendix E illustrates these outcomes through the live dashboard.

7.5.2 Trigger-Policy Calibration

The voting policy and cooldown used by Stage 7 were not chosen arbitrarily; they were selected in a development-time calibration study retained here because it quantifies why the governed policy is safer than naive alternatives. In the study, candidate trigger policies were replayed over the recorded monitoring history, and their triggers were scored against sixteen reference risk episodes identified in that history. Table 7.3 reports the outcome. A single-signal policy, which fires whenever any one monitoring signal crosses its threshold, detects every episode but emits 251 triggers at a 13.9% false-alarm rate. Requiring agreement of two of the three voting signals keeps episode recall at 100% while cutting the false-alarm rate to 0.7% and raising precision from 0.841 to 0.988. Adding the 24-hour cooldown of the deployed configuration reduces trigger volume further, to fourteen triggers at perfect precision, at the cost of not re-triggering on repeat episodes inside the cooldown window (episode recall 0.875). This trade is acceptable under the no-auto-retrain design

because a triggered review remains open until a human closes it: suppressing a duplicate trigger does not suppress the underlying review. These numbers are calibration evidence from development, not a deployment benchmark; they are reported because they are the empirical justification for the two-of-three voting and cooldown defaults of Chapter 6.

Table 7.3: Trigger-policy calibration against sixteen reference risk episodes in the recorded monitoring history.

Policy	**Triggers**	**Precision**	**Episode recall**	**False-alarm rate**
Single signal	251	0.841	1.000	13.9%
Two-of-three voting	164	0.988	1.000	0.7%
Two-of-three + 6 h cooldown	42	0.952	1.000	0.7%
Two-of-three + 12 h cooldown	24	0.917	1.000	0.7%
Two-of-three + 24 h cooldown (deployed)	14	1.000	0.875	0.0%

7.5.3 Preprocessing and Contract Sensitivity

The claim that contract choices materially change runtime behavior is supported by a controlled windowing experiment. The same labeled Garmin data are re-windowed under six combinations of window size (128, 200, and 256 samples) and overlap (25% and 50%), and a lightweight surrogate model is refit per configuration under an identical reduced training budget. Because of that reduced budget, the absolute scores are not comparable to the proposed-model results in Section 7.2; the sweep is read for relative differences between configurations under identical data and code.

Table 7.4: Contract-sensitivity ablation over window size and overlap (surrogate refit per configuration; read for relative differences, not absolute performance). The deployed contract uses 200 samples at 50% overlap.

Window	**Overlap**	**Duration**	**Accuracy**	**Macro-F1**	**Flip rate (median)**
128	25%	2.56 s	0.673	0.671	0.282
128	50%	2.56 s	0.666	0.661	0.223
200	25%	4.00 s	0.676	0.676	0.292
200	**50%**	**4.00 s**	**0.688**	**0.685**	**0.240**
256	25%	5.12 s	0.678	0.670	0.304
256	50%	5.12 s	0.735	0.731	0.233

Windowing alone moves surrogate accuracy across an approximately seven-percentage-point range (66.6% to 73.5%) with the architecture, data, and training budget fixed, confirming that preprocessing choices are not cosmetic. The 50% overlap lowers the prediction flip rate at every window size (for the 200-sample window, from 0.292 to 0.240 median), so the overlap choice directly shapes the temporal-stability signal that Stage 6 monitors. Longer windows raise surrogate accuracy but lengthen the decision interval to 5.12 seconds. The deployed contract (200 samples at 50% overlap) balances recognition quality, the four-second window duration, and temporal stability. Together

with the trigger-policy calibration above, this sweep is the quantitative basis for the contract-sensitivity claim of Chapters 3 and 4: window construction and preprocessing change accuracy, confidence, and stability even when everything else is fixed, which is precisely why they are bound into the runtime contract rather than left as free parameters.

7.6 Synthesis and Implications

The four sections of evidence in this chapter tell one consistent story. Within the labeled training distribution the model reaches 93.61% five-fold accuracy, confirming the architecture and preprocessing contract are technically sound. On a blind external session evaluated once after the model was frozen, accuracy falls to 36.96% while mean confidence remains 0.858: the model is wrong most of the time but does not know it. The proxy evaluation in Section 7.4 then shows the pipeline can detect this without labels: schedule-aligned accuracy tracks the labeled result to within 0.3 percentage points, and the transition rate, low-confidence window rate, and dominant-class share all point to the same degraded session quality the labels confirm. Temporal smoothing narrows the gap modestly (to 39.43%) and resolves single-window jitter, but cannot fix the sustained cross-user confusions that drive the bulk of the error. At scale, 23 of 25 unlabeled sessions are correctly routed to review, with no false retrain authorization.

The implication is not that the model is inadequate. Within-distribution accuracy is an insufficient basis for trusting a wearable HAR model in deployment. A model can match a published benchmark on the training population and still be systematically wrong and overconfident on a new wearer. The pipeline is built for this condition: the runtime contract keeps preprocessing and model coupled to prevent silent mismatch, the multi-signal monitor surfaces risk without labels, and the governance boundary routes that risk to human review rather than automatic mutation. The trigger-policy calibration shows two-of-three voting raises precision from 0.841 to 0.988 at full episode recall; the windowing ablation shows preprocessing choices alone move recognition and stability by several points. The experimental result is therefore the governance architecture, not a high final accuracy.

7.7 Chapter Summary

This chapter reported the empirical evidence across three conditions. Five-fold cross-validation on the six labeled users confirms the model learns the training distribution and is consistent in magnitude with the published benchmark [8]. The blind User 7 probe exposes a 57-point accuracy drop accompanied by high overconfidence, and the proxy evaluation in Section 7.4 shows the pipeline detects this degradation without labels: schedule-aligned accuracy, low-confidence window rate, and temporal transition rate all flag the same session the labeled probe confirms is poor. Temporal smoothing partially recovers stability but not the underlying cross-user confusion. At scale, 23 of 25 unlabeled sessions are routed to human review with no automatic retraining, the trigger-policy calibration establishes

the two-of-three voting and cooldown defaults, and the windowing ablation grounds the contract-sensitivity argument. Chapter 8 interprets these findings against the research questions and states the limitations and future directions.

Chapter 8

Discussion and Conclusion

This chapter interprets the evidence from Chapter 7, answers the research questions posed in Chapter 1, states the limitations and threats to validity, and outlines future work. The experimental results can be summarized in one sentence: the proposed model learns the known Garmin distribution well (93.61% five-fold), degrades on a blind external probe (36.96%), and remains systematically overconfident throughout (mean confidence 0.858 on User 7). The discussion below treats this gap not as a failure to be explained away, but as the empirical justification for the runtime-contract, monitoring, and governance contribution of the thesis.

8.1 Findings and Answers to the Research Questions

The central finding is that within-distribution accuracy is an insufficient basis for trusting a wearable HAR model in a mental-health-adjacent setting. A model can match or exceed a published five-fold benchmark and still be unreliable and miscalibrated on a wearer it has not seen. The within-distribution result is legitimate and is reported as such; the unseen-user and blind-probe results are realistic deployment estimates and are reported without inflation. The distance between the two is the result. Table 8.1 answers the four research questions against this evidence.

For RQ1, the strongest evidence is the session-level decomposition of Section 7.5.1: across twenty-five real unlabeled sessions the pipeline routed twenty-three to human review, caught one collapse-level session at a 99.7% dominant share through the collapse guard, and queued zero automatic retrains while the confidence signal alone, the naive safeguard, would have flagged nothing. The answer is bounded in two ways: the signals indicate operational risk rather than verified error, and the thresholds were calibrated on the same session pool (Section 6.5), so the result demonstrates the policy operating as configured rather than threshold generalization.

For RQ2, the windowing ablation of Section 7.5.3 makes the contract claim quantitative: with architecture, data, and training budget fixed, window-size and overlap choices alone move surrogate accuracy across an approximately seven-percentage-point range and change the temporal flip rate that Stage 6 monitors, and the channel-statistics comparison of

Table 8.1: Research questions, the evidence used, and the bounded answer.

RQ	Evidence	Answer (with bound)
RQ1	Unlabeled-session monitoring routed to `review` using confidence, temporal, dominant-class, and drift signals (Section 7.5).	Yes, but bounded: the offline-first pipeline detects operational risk without labels, yet it indicates risk rather than verified correctness.
RQ2	Within-distribution success under the fixed contract plus the contract-sensitivity ablation (Sections 7.2.1, 7.5.3).	Substantially: unit handling, scaler, channel order, windowing, and baseline each materially change recognition and monitoring behavior when altered, so these contract elements must remain bound to the model.
RQ3	No-auto-retrain trigger policy, collapse guard, cooldown, and the trigger-policy calibration (Chapter 6; Section 7.5.2).	Yes: routing severity to review rather than to automatic mutation is safer than unrestricted retraining under label-scarce, shifted, or collapsed conditions.
RQ4	Strong five-fold versus weak blind-probe results, with a 0.858 confidence gap (Section 7.3).	Not a single cause: the contract is sound (within-distribution success makes a gross pipeline break unlikely), so the degradation points to cross-user domain shift and calibration limits, not to input data alone.

Chapter 4 shows the input distributions the contract must hold together. Preprocessing and windowing are therefore semantic components of the model, not interchangeable defaults, which is exactly why the runtime contract binds them.

For RQ3, the governance design is supported both architecturally and empirically. The trigger-policy calibration (Section 7.5.2) shows that requiring two-of-three signal agreement raises trigger precision from 0.841 to 0.988 at full episode recall and cuts the false-alarm rate from 13.9% to 0.7%, and the cooldown reduces duplicate triggers without closing the underlying review. On the session-level evaluation the no-auto-retrain invariant held on all twenty-five sessions. Unrestricted automatic retraining would have acted on label-free, possibly collapsed predictions; the governed policy demonstrably routed the same risk to review instead.

For RQ4, the attribution rests on two observations that hold simultaneously: the channel statistics of Chapter 4 show measurable distribution differences across sources; and the labeled and schedule-aligned User 7 evaluations agree to within 0.3 percentage points (Section 7.5.1), which shows the pipeline measures consistently whether or not labels are present. Together these point to cross-user domain shift and calibration limits as the drivers of degradation, with a gross model–contract defect made unlikely (though not impossible) by the within-distribution result.

8.2 Interpretation and Implications for Wearable Mental-Health-Adjacent MLOps

The within-distribution and blind-probe gap has a clear interpretation. Window-level five-fold cross-validation measures whether the model can fit the labeled distribution; the

blind User 7 probe measures whether it transfers to a genuinely unseen wearer. These are different questions, and the same model and data answer them very differently. The roughly 57-point gap indicates that the binding constraint is cross-user generalization rather than model capacity or preprocessing correctness, consistent with broader evidence that wearable HAR models degrade across users and devices [40, 10, 11]. The calibration gap compounds this: the model is not merely wrong on unseen users, it is wrong while confident, which is the most dangerous behavior for a health-adjacent system because a confident-but-wrong session can pass a naive safeguard silently.

The implication for wearable mental-health-adjacent MLOps is that notebook-level accuracy cannot be the basis for promotion or trust. The system needs a runtime contract that keeps preprocessing and the model coupled, monitoring signals that surface operational risk when labels are unavailable, and a governance boundary that routes risk to human review rather than to automatic model mutation. The experimental evidence therefore validates the architecture of the thesis rather than a high final accuracy. The contribution is not a model that solves anxiety-related HAR for all users; it is a reproducible, monitored, and governed lifecycle that makes the model's limitations visible and keeps its outputs interpretable.

The operational machinery is as much a result as the model numbers. Every defended number maps to a versioned artifact through the run-scoped trail and the integrity-hashed runtime-contract package; the data, the contract, and the defended evidence run are restorable through DVC or the release bundle; a suite of 95 automated tests (dominated by monitoring, trigger, pipeline, and contract checks) runs in continuous integration on every push; the training and serving environments are containerized along the governance boundary; and on twenty-five real sessions the monitoring layer surfaced risk on twenty-three while permitting zero unauthorized mutations. In an area where models are routinely published without their operational surroundings, this auditable surrounding is the thesis's primary engineering claim.

8.3 Limitations and Threats to Validity

The scope of these claims is bounded, and the architectural and mental-health boundaries set in Section 3.6 continue to apply. The following limitations are specific to the evidence in this thesis.

- **Small and narrow user population.** The labeled development set contains six Garmin users and one blind external probe. Cross-user estimates from six users carry high variance, and a single blind probe cannot establish population-level behavior.
- **User 7 data quality and author-as-subject bias.** User 7 is self-recorded and self-labeled by the author, who knew the activity definitions and recording schedule. Both annotation timing uncertainty and a demand-characteristics risk apply: the performed movements may be cleaner or more canonical than spontaneous behavior. The procedural mitigations (exclusion from training, scaler fitting, and

model selection; a single post-freeze evaluation) bound the statistical influence but cannot remove the behavioral one.

- **Threshold–evaluation circularity.** As disclosed in Section 6.5, the monitoring thresholds and trigger policy were calibrated on the same pool of recorded sessions used for the session-level evaluation. That evaluation therefore demonstrates the policy operating as configured rather than threshold generalization to unseen deployments.
- **Retraining constraints.** The fine-tune-from-current retraining path and the candidate-versus-current governance gate were validated as a functional and reversible mechanism, not as a demonstrated accuracy improvement; no held-out retraining dataset was available. The protocol also requires several users to form user-held-out validation folds and cannot fine-tune on a single new wearer.
- **Scope boundaries.** The system recognizes observable wrist-motion activities; it does not diagnose anxiety, stress, or any clinical state, and no clinical validation was performed. Results characterize this dataset and device, not wearable HAR in general. The defended runtime is offline-first and single-machine; on-device and real-time operation are out of scope. Monitoring and trigger thresholds are conservative defaults calibrated against this repository's distribution, not longitudinally validated decision limits.

8.4 Future Work

The most direct limitation of the current evidence is its scale. The labeled development set contains six wearers, and a single blind probe cannot establish population-level behavior. A larger longitudinal dataset with more users, recorded across realistic behavioral variation rather than a scripted protocol, would allow the monitoring thresholds to be calibrated against a representative session population rather than set as conservative defaults.

The cross-user degradation evidenced by the blind User 7 probe motivates principled per-user adaptation at inference time. Test-time adaptation and continual user-specific fine-tuning allow the model to update from a wearer's own activity stream rather than relying solely on cross-user generalization [9, 58]. Such personalization is a natural complement to the governed retraining path that Stages 8–10 already provide.

The model is also systematically overconfident on unseen users: mean softmax confidence 0.858 while macro-F1 is 0.350 on the blind probe. Post-hoc calibration methods such as temperature scaling [17] or stochastic inference via Monte Carlo dropout [18] address this directly by aligning confidence scores with empirical accuracy, which would make the confidence monitoring signal more informative than a threshold against uncalibrated probabilities.

Several distribution-shift and domain-adaptation methods were evaluated during development but excluded from the defended runtime under the no-auto-retrain policy. These include lightweight test-time adaptation strategies [59] and transfer-learning approaches

calibrated for wearable HAR [11, 41]. Reconsidering these once a labeled validation path exists would allow drift signals to guide controlled model adaptation rather than route all risk to human review alone.

The served prediction stream is currently stabilized by a fixed majority-vote temporal smoother, which reduces the temporal transition rate from 40.6% to 21.5% without labels (Section 7.4). Replacing this heuristic post-filter with a learned sequence model such as a hidden Markov model or a conditional random field [63] is the principled refinement: such a model captures the transition structure of real behavioral sequences rather than applying a uniform vote window uniformly across all activities.

The most direct path toward a deployed online service requires no architectural change. The inference service is already containerized (`docker/Dockerfile.inference`: FastAPI served by Uvicorn) and exposes predictions over HTTP, and the runtime-contract package is mounted as a versioned volume whose integrity hash the contract validator checks on every startup. Deploying this to Kubernetes amounts to pushing the inference image to a container registry, provisioning the contract volume as a PersistentVolumeClaim, and defining a Deployment backed by that image together with a Service that routes HTTP traffic to it. This blueprint is cloud-agnostic: the same Deployment, Service, and PersistentVolumeClaim manifests run without code modification on any managed Kubernetes offering, including Amazon Elastic Kubernetes Service (EKS), Google Kubernetes Engine (GKE), or Azure Kubernetes Service (AKS), and a cloud-hosted MLflow Tracking Server or model registry on any of these platforms could replace the local DVC artifact store as the lineage backend for the runtime-contract volume. Horizontal Pod Autoscaling would then scale inference replicas with request load, while the monitoring loop could run as a sidecar container or a periodic CronJob consuming the prediction logs. Governed retraining under this architecture maps naturally to a Kubernetes Job: the trigger policy submits the job when signal agreement and the cooldown gate are satisfied, and the existing review gate still requires a human decision before the candidate model updates the contract volume. Stages 8–10 already provide the governance-gated skeleton that a continuous-delivery loop for model updates would formalize; the engineering step that separates the offline-first prototype from a production-grade service is exposing that skeleton to a scheduler and a container registry.

Before any of these extensions could support mental-health-adjacent interpretation, clinical validation and an explicit human-review workflow would be required. Longitudinal calibration of monitoring thresholds against real clinician assessments, rather than surrogate agreement with a scripted recording protocol, is the minimum before the outputs could be considered responsible outside a research context. These directions are revisited from the scope boundary established in Section 3.6.

8.5 Conclusion

This thesis developed an offline-first MLOps pipeline for wearable-sensor-based, mental-health-adjacent behavioral-proxy recognition, built around a runtime contract, post-inference monitoring, and a governance boundary that separates risk observation from

model mutation. The empirical evaluation of the proposed model showed strong within-distribution learnability (93.61% five-fold), degraded performance on a blind external probe (36.96%), and systematic overconfidence (mean confidence 0.858). The thesis does not claim a universally robust anxiety-related HAR model. Its contribution is to show that a wearable HAR model needs an explicit runtime contract, monitoring, artifact lineage, and governance before its outputs can be responsibly interpreted in a mental-health-adjacent setting, and to demonstrate a reproducible pipeline where every monitoring decision traces to a stored artifact and every retraining action requires human approval.

References

[1] A. Abd-alrazaq, R. AlSaad, S. Aziz, A. Ahmed, K. Denecke, M. Househ, F. Farooq, and J. Sheikh, "Wearable artificial intelligence for detecting anxiety: Systematic review and meta-analysis," *Journal of Medical Internet Research*, vol. 25, p. e48754, 2023.

[2] N. Gomes, M. Pato, A. R. Lourenço, and N. Datia, "A survey on wearable sensors for mental health monitoring," *Sensors*, vol. 23, no. 3, p. 1330, 2023.

[3] U. Oleh, R. Obermaisser, A. Malchulska, and T. Klucken, "A multi-stage, RAG-enhanced pipeline for generating longitudinal, clinically actionable mental health reports from wearable sensor data," in *Advances in Digital Health and Medical Bioengineering II (EHB 2025)*, ser. IFMBE Proceedings, vol. 142. Springer, 2025.

[4] S. Shen, W. Qi, J. Zeng, S. Li, X. Liu, X. Zhu, C. Dong, B. Wang, Y. Shi, J. Yao, B. Wang, X. Lou, S. Gu, P. Li, J. Wang, G. Jiang, and S. Cao, "Passive sensing for mental health monitoring using machine learning with wearables and smartphones: Scoping review," *Journal of Medical Internet Research*, vol. 27, p. e77066, 2025.

[5] Ó. D. Lara and M. A. Labrador, "A survey on human activity recognition using wearable sensors," *IEEE Communications Surveys & Tutorials*, vol. 15, no. 3, pp. 1192–1209, 2013.

[6] A. Bulling, U. Blanke, and B. Schiele, "A tutorial on human activity recognition using body-worn inertial sensors," *ACM Computing Surveys*, vol. 46, no. 3, pp. 33:1–33:33, 2014.

[7] N. S. Khan, M. S. Ghani, and G. Anjum, "ADAM-sense: Anxiety-displaying activities recognition by motion sensors," *Pervasive and Mobile Computing*, vol. 78, p. 101485, 2021.

[8] U. Oleh and R. Obermaisser, "Recognition of anxiety-related activities using 1DC-NNBiLSTM on sensor data from a commercial wearable device," in *The 15th International Conference on Current and Future Trends of Information and Communication Technologies in Healthcare (ICTH 2025)*, ser. Procedia Computer Science, vol. 272. Elsevier, 2025, pp. 351–359.

[9] A. Akbari and R. Jafari, "Personalizing activity recognition models through quantifying different types of uncertainty using wearable sensors," *IEEE Transactions on Biomedical Engineering*, vol. 67, no. 9, pp. 2530–2541, 2020.

[10] H. Wang, X. Wang, C. Lu, M. Yuan, Y. Wang, H. Yu, and H. Li, "Enhancing human activity recognition in wrist-worn sensor data through compensation strategies for sensor displacement," *IEEE Access*, vol. 12, pp. 95 058–95 075, 2024.

[11] S. G. Dhekane and T. Plötz, "Transfer learning in human activity recognition: A survey," *arXiv preprint arXiv:2401.10185*, 2024.

[12] S. Jackson, M. Yaqub, and C.-X. Li, "The agile deployment of machine learning models in healthcare," *Frontiers in Big Data*, vol. 1, p. 7, 2019.

[13] S. Amershi, A. Begel, C. Bird, R. DeLine, H. Gall, E. Kamar, N. Nagappan, B. Nushi, and T. Zimmermann, "Software engineering for machine learning: A case study," in *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 2019, pp. 291–300.

[14] D. Kreuzberger, N. Kühl, and S. Hirschl, "Machine learning operations (MLOps): Overview, definition, and architecture," *IEEE Access*, vol. 11, pp. 31 866–31 879, 2023.

[15] A. Paleyes, R.-G. Urma, and N. D. Lawrence, "Challenges in deploying machine learning: A survey of case studies," *ACM Computing Surveys*, vol. 55, no. 6, pp. 1–29, 2022.

[16] N. Gift and A. Deza, *Practical MLOps: Operationalizing Machine Learning Models*. O'Reilly Media, 2021.

[17] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proceedings of the 34th International Conference on Machine Learning*, vol. 70, 2017, pp. 1321–1330.

[18] Y. Gal and Z. Ghahramani, "Dropout as a Bayesian approximation: Representing model uncertainty in deep learning," in *Proceedings of the 33rd International Conference on Machine Learning*, vol. 48, 2016, pp. 1050–1059.

[19] L. M. Levine, M. Gwak, K. Kärkkäinen, S. Fazeli, B. Zadeh, T. Peris, A. S. Young, and M. Sarrafzadeh, "Anxiety detection leveraging mobile passive sensing," *arXiv preprint arXiv:2008.03810*, 2020.

[20] P. Lago, "A state-of-the-art review of computational models for analyzing longitudinal wearable sensor data in healthcare," *arXiv preprint arXiv:2407.21665*, 2024.

[21] F. Sabry, T. Eltaras, W. Labda, K. Alzoubi, and Q. Malluhi, "Machine learning for healthcare wearable devices: The big picture," *Journal of Healthcare Engineering*, vol. 2022, p. 4653923, 2022.

[22] P. T. Borghare, D. A. Methwani, and A. G. Pathade, "A comprehensive review on harnessing wearable technology for enhanced depression treatment," *Cureus*, vol. 16, no. 8, p. e66173, 2024.

[23] N. C. Jacobson, D. Lekkas, R. Huang, and N. Thomas, "Deep learning paired with wearable passive sensing data predicts deterioration in anxiety disorder symptoms across 17–18 years," *Journal of Affective Disorders*, vol. 282, pp. 104–111, 2021.

[24] A. Pinge, V. Gad, D. Jaisighani, S. Ghosh, and S. Sen, "Detection and monitoring of stress using wearables: A systematic review," *Frontiers in Computer Science*, vol. 6, p. 1478851, 2024.

[25] L. Bao and S. S. Intille, "Activity recognition from user-annotated acceleration data," in *Pervasive Computing*, ser. Lecture Notes in Computer Science, vol. 3001. Springer, 2004, pp. 1–17.

[26] R. K. Nath and H. Thapliyal, "Machine learning based anxiety detection in older adults using wristband sensors and context feature," *arXiv preprint arXiv:2106.03019*, 2021.

[27] F. J. Ordóñez and D. Roggen, "Deep convolutional and LSTM recurrent neural networks for multimodal wearable activity recognition," *Sensors*, vol. 16, no. 1, p. 115, 2016.

[28] N. Y. Hammerla, S. Halloran, and T. Plötz, "Deep, convolutional, and recurrent models for human activity recognition using wearables," in *Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence (IJCAI)*, 2016, pp. 1533–1540.

[29] J. Wang, Y. Chen, S. Hao, X. Peng, and L. Hu, "Deep learning for sensor-based activity recognition: A survey," *Pattern Recognition Letters*, vol. 119, pp. 3–11, 2019.

[30] S. Zhang, Y. Li, S. Zhang, F. Shahabi, S. Xia, Y. Deng, and N. Alshurafa, "Deep learning in human activity recognition with wearable sensors: A review on advances," *Sensors*, vol. 22, no. 4, p. 1476, 2022.

[31] U. Oleh, R. Obermaisser, and A. S. Ahammed, "A review of recent techniques for human activity recognition: Multimodality, reinforcement learning, and language models," *Algorithms*, vol. 17, no. 10, p. 434, 2024.

[32] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[33] M. Schuster and K. K. Paliwal, "Bidirectional recurrent neural networks," *IEEE Transactions on Signal Processing*, vol. 45, no. 11, pp. 2673–2681, 1997.

[34] N. Lyons, A. Santra, and A. Pandey, "Improved deep representation learning for human activity recognition using IMU sensors," in *2021 20th IEEE International Conference on Machine Learning and Applications (ICMLA)*. IEEE, 2021, pp. 326–331.

[35] H. Wang, J. Zhao, J. Li, L. Tian, P. Tu, T. Cao, Y. An, K. Wang, and S. Li, "Wearable sensor-based human activity recognition using hybrid deep learning techniques," *Security and Communication Networks*, vol. 2020, p. 2132138, 2020.

[36] Z. Wang, W. Yan, and T. Oates, "Time series classification from scratch with deep neural networks: A strong baseline," in *2017 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2017, pp. 1578–1585.

[37] D. Anguita, A. Ghio, L. Oneto, X. Parra, and J. L. Reyes-Ortiz, "A public domain dataset for human activity recognition using smartphones," in *Proceedings of the 21st European Symposium on Artificial Neural Networks (ESANN)*, 2013, pp. 437–442.

[38] J. Reyes-Ortiz, D. Anguita, A. Ghio, L. Oneto, and X. Parra, "Human activity recognition using smartphones," UCI Machine Learning Repository, 2013.

[39] A. Reiss and D. Stricker, "Introducing a new benchmarked dataset for activity monitoring," in *2012 16th International Symposium on Wearable Computers*. IEEE, 2012, pp. 108–109.

[40] A. Stisen, H. Blunck, S. Bhattacharya, T. S. Prentow, M. B. Kjærgaard, A. K. Dey, T. Sonne, and M. M. Jensen, "Smart devices are different: Assessing and mitigating mobile sensing heterogeneities for activity recognition," in *Proceedings of the 13th ACM Conference on Embedded Networked Sensor Systems*, ser. SenSys '15. ACM, 2015, pp. 127–140.

[41] A. Chakma, A. Z. M. Faridee, I. Ghosh, and N. Roy, "Domain adaptation for inertial measurement unit-based human activity recognition: A survey," *arXiv preprint arXiv:2304.06489*, 2023.

[42] N. Hewage and D. Meedeniya, "Machine learning operations: A survey on mlops tool support," *arXiv preprint arXiv:2202.10169*, 2022.

[43] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems*, vol. 28, 2015, pp. 2503–2511.

[44] J. Gardner, Y. Yang, R. Baker, and C. Brooks, "Enabling end-to-end machine learning replicability: A case study in educational data mining," in *ICML 2018 Reproducibility in Machine Learning Workshop*, 2018, arXiv:1806.05208.

[45] L. Bahaidarah, E. Hung, A. F. De Melo Oliveira, J. Penumaka, L. Rosario, and A. Trisovic, "Toward reusable science with readable code and reproducibility," *arXiv preprint arXiv:2109.10387*, 2021.

[46] A. R. Munappy, J. Bosch, H. Holmström Olsson, A. Arpteg, and B. Brinne, "Data management challenges for deep learning," in *2019 45th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. IEEE, 2019, pp. 140–147.

[47] M. Zaharia, A. Chen, A. Davidson, A. Ghodsi, S. A. Hong, A. Konwinski, S. Murching, T. Nykodym, P. Ogilvie, M. Parkhe, F. Xie, and C. Zumar, "Accelerating the machine learning lifecycle with MLflow," *IEEE Data Engineering Bulletin*, vol. 41, no. 4, pp. 39–45, 2018.

[48] R. Kuprieiev, S. Bhattarai, D. Petrov, P. Redzyński, and I. Shcheklein, "DVC: Data version control — Git for data and models," Zenodo, 2023, doi:10.5281/zenodo.7976797.

[49] J. Błasiok, P. Gopalan, L. Hu, and P. Nakkiran, "When does optimizing a proper loss yield calibration?" in *Advances in Neural Information Processing Systems*, 2023, arXiv:2305.18764.

[50] J. Gama, I. Zliobaite, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, pp. 44:1–44:37, 2014.

[51] D. Hendrycks and K. Gimpel, "A baseline for detecting misclassified and out-of-distribution examples in neural networks," in *International Conference on Learning Representations*, 2017, arXiv:1610.02136.

[52] G. I. Webb, R. Hyde, H. Cao, H. L. Nguyen, and F. Petitjean, "Characterizing concept drift," *Data Mining and Knowledge Discovery*, vol. 30, pp. 964–994, 2016.

[53] A. Gretton, K. M. Borgwardt, M. J. Rasch, B. Schölkopf, and A. Smola, "A kernel two-sample test," *Journal of Machine Learning Research*, vol. 13, pp. 723–773, 2012.

[54] W. Liu, X. Wang, J. D. Owens, and Y. Li, "Energy-based out-of-distribution detection," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 21 464–21 475.

[55] D.-H. Lee, "Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks," in *ICML 2013 Workshop on Challenges in Representation Learning*, vol. 3, 2013, p. 896.

[56] E. Arazo, D. Ortego, P. Albert, N. E. O'Connor, and K. McGuinness, "Pseudo-labeling and confirmation bias in deep semi-supervised learning," in *2020 International Joint Conference on Neural Networks*. IEEE, 2020, pp. 1–8.

[57] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell, "Overcoming catastrophic forgetting in neural networks," *Proceedings of the National Academy of Sciences*, vol. 114, no. 13, pp. 3521–3526, 2017.

[58] S. Jha, M. Schiemer, F. Zambonelli, and J. Ye, "Continual learning in sensor-based human activity recognition: An empirical benchmark analysis," *Information Sciences*, vol. 575, pp. 1–28, 2021.

[59] D. Wang, E. Shelhamer, S. Liu, B. Olshausen, and T. Darrell, "Tent: Fully test-time adaptation by entropy minimization," in *International Conference on Learning Representations*, 2021, arXiv:2006.10726.

[60] J. Kim, S. Oh, H. Kim, and W. Choi, "Tutorial on time series prediction using 1d-cnn and bilstm: A case example of peak electricity demand and system marginal price prediction," *Engineering Applications of Artificial Intelligence*, vol. 126, p. 106817, 2023.

[61] P. Ruf, M. Madan, C. Reich, and D. Ould-Abdeslam, "Demystifying MLOps and presenting a recipe for the selection of open-source tools," *Applied Sciences*, vol. 11, no. 19, p. 8861, 2021.

[62] A. Bodor, M. Hnida, and D. Najima, "From development to deployment: An approach to MLOps monitoring for machine learning model operationalization," in *2023 14th International Conference on Intelligent Systems: Theories and Applications (SITA)*. IEEE, 2023, pp. 1–8.

[63] J. Lester, T. Choudhury, and G. Borriello, "A practical approach to recognizing physical activities," in *Pervasive Computing (Pervasive 2006)*, ser. Lecture Notes in Computer Science, vol. 3968. Springer, 2006, pp. 1–16.

[64] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, no. 1–2, pp. 100–115, 1954.

Appendix A

Reproducible Commands and Runbook

This appendix lists the commands needed to reproduce the thesis runs. The defended pipeline, tests, experiment tracking, and figure generation run in the `thesis-mlops` conda environment. A separate `tf-gpu` environment was used only to accelerate model building; it is not required for normal pipeline inference.

A.1 Environment setup

```
conda create -n thesis-mlops python=3.11 -y
conda activate thesis-mlops
python -m pip install --upgrade pip
python -m pip install -r config/requirements.txt
```

A.2 Retrieve versioned data and model artifacts

```
dvc pull
```

A.3 Run the assessment pipeline (Stages 1–7)

For a specific raw session, pass the accelerometer and gyroscope CSV files explicitly:

```
python run_pipeline.py --stages ingestion validation transformation \
  inference evaluation monitoring trigger \
  --accel-file data/raw/<session>_accelerometer.csv \
  --gyro-file  data/raw/<session>_gyroscope.csv
```

A.4 Governed retraining (Stages 8–10, review-gated)

```
python run_pipeline.py --retrain --labels data/raw/labels.csv
```

Retraining fine-tunes the current proposed model on labeled data and requires review, labels, and quality-assurance approval before any promotion; it is never triggered automatically.

A.5 Chapter 7 figures

```
python scripts/generate_ch7_figures.py
```

A.6 Tests

```
python -m pytest tests -q
python -m pytest tests/test_runtime_contract_consistency.py -q
python -m pytest tests/test_stage2_validation.py tests/test_stage6_monitoring.py -q
```

A.7 API inference service (local)

```
python -m uvicorn src.api.app:app --host 127.0.0.1 --port 8000
```

Appendix B

Runtime Contract and Artifact Inventory

This appendix documents the deployable proposed-model package. The deployment unit is not the model file alone but the complete runtime contract: the model, its train-only scaler, the fixed label order, the monitoring baseline, the contract metadata, and the integrity record must be promoted together, as argued in Chapters 3–5. All artifacts below reside in the same runtime-contract folder, `models/runtime_contract/model_d1_final_20260528/`.

Table B.1: Contents of the deployed proposed-model runtime contract.

Artifact	Role
`finetuned_1DCNNBILSTM_HAR_model.keras`	Frozen 1D-CNN-BiLSTM model (967,499 parameters), native Keras format.
`finetuned_1DCNNBILSTM_HAR_model_savedmodel/`	Portable SavedModel form for cross-environment loading.
`garmin_train_scaler.json`	Train-only StandardScaler parameters (per-channel mean and scale), fitted on Garmin users 1–6.
`garmin_train_scaler.joblib`	The same scaler in joblib form, used by the inference path.
`garmin_train_scaled_baseline.json`	Monitoring baseline in the scaled feature space; drift reference for the API/runtime-contract monitoring path (the batch pipeline's Stage 6 uses the raw-physical baseline, Chapter 6).
`label_order.json`	Fixed eleven-class label order used to decode the softmax output.
`model_contract.json`	Contract metadata: input shape, window, stride, sampling rate, scaler fit users, and result-file pointers.
`model_param_count.json`	Parameter-count record for the promoted model: 967,499 total parameters, 966,347 trainable, and 1,152 non-trainable.
`artifact_md5.json`	Integrity hashes so the package can be verified as one coherent unit.
`user7_blind_probe_summary.json`	Result of the labeled User 7 blind probe.
`user7_unlabeled_probe_summary.json`	Result of the unlabeled User 7 monitoring probe.

The model expects input of shape $(200, 6)$ at 50 Hz, a 200-sample window with a

100-sample stride, accelerometer channels in m/s^2 and gyroscope channels in deg/s, and the eleven-class label order above. Deployment or probe data are never used to fit the scaler; the saved Garmin train-only scaler is reused to preserve train–serve parity.

Appendix C

Experimental Evidence and Result Artifacts

This appendix maps every quantitative result reported in Chapter 7 to the artifact it was read from, so that each number can be verified at its source. The recognition numbers were produced by the build notebook `notebooks/Model_D1_Final_Build_Runtime_Contract.ipynb`, which writes its results to `notebooks/model_HAR_27may_artifacts/`; this is the same location referenced by the deployed contract metadata.

A note on protocol provenance. The five-fold result uses a stratified window-level split executed in this build notebook. The within-distribution result in this thesis comes from the build notebook, while the pipeline orchestrates inference, monitoring, evaluation, and governed retraining over the resulting runtime contract. The from-scratch architecture in the pipeline training module is a smaller baseline configuration; the deployed proposed model (967,499 parameters) was built and fine-tuned in the notebook, and governed retraining fine-tunes that deployed model rather than training a new architecture from scratch.

The five-fold result is recorded with the explicit note that it measures within-distribution learnability. These provenance notes are the basis for the two-protocol framing used throughout Chapter 7.

Table C.1: Chapter 7 results and their source artifacts.

Result	**Value**	**Source artifact**
Five-fold window-level CV	93.61% / 0.937	`notebooks/model_HAR_27may_artifacts/garmin_5fold_window_cv_results.json`
Blind User 7 probe	36.96% / 0.350 / conf 0.858	`models/runtime_contract/model_d1_final_20260528/user7_blind_probe_summary.json`
Chapter 7 figures (7.1–7.4)	—	`scripts/generate_ch7_figures.py`
Confusion matrix and reliability	—	regenerated from the deployed model on the User 7 session by the script above
Monitoring and trigger outputs (Section 7.5)	`review`	`thesis/evidence/TRIGGER_POLICY_EVAL.csv`; per-run `trigger_decision.json`
Contract-sensitivity sweeps (Sections 7.5.3, 7.5.2)	—	`thesis/evidence/ABLATION_WINDOWING.csv`; `reports/THRESHOLD_CALIBRATION.csv`
Unlabeled monitoring at scale (25 sessions, Figures 7.5–7.8)	23/25 review, 0/25 retrain	`scripts/run_unlabeled_sessions.py` and `scripts/collect_unlabeled_monitoring.py` produce `reports/unlabeled_sessions_monitoring.json` and `.csv` from the per-run artifact trails
User-7 labeled vs unlabeled (Figure 7.3)	36.96% vs 36.70%	`models/runtime_contract/model_d1_final_20260528/user7_blind_probe_summary.json` and `user7_unlabeled_probe_summary.json`
Temporal smoothing effect (Section 7.5.1, Figure 7.4)	T: 40.6%→21.5%; acc. +2.7pp	`scripts/generate_smoothing_figure.py` on the User 7 unlabeled session

Appendix D

Supplementary Result Figures

This appendix collects supplementary figures referenced from Chapter 7. They support the main results but are not central to the argument, so they are placed here to keep the experiments chapter focused.

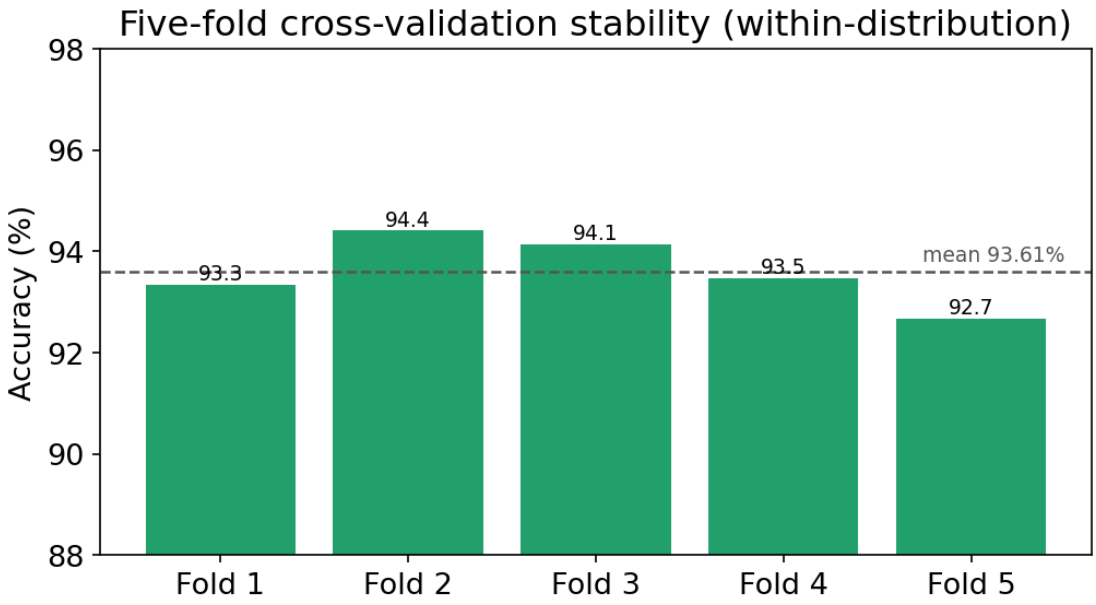


Figure D.1: Per-fold accuracy under five-fold cross-validation on Garmin users 1–6. The low variance across folds shows the within-distribution result is stable and not driven by a single favorable split.

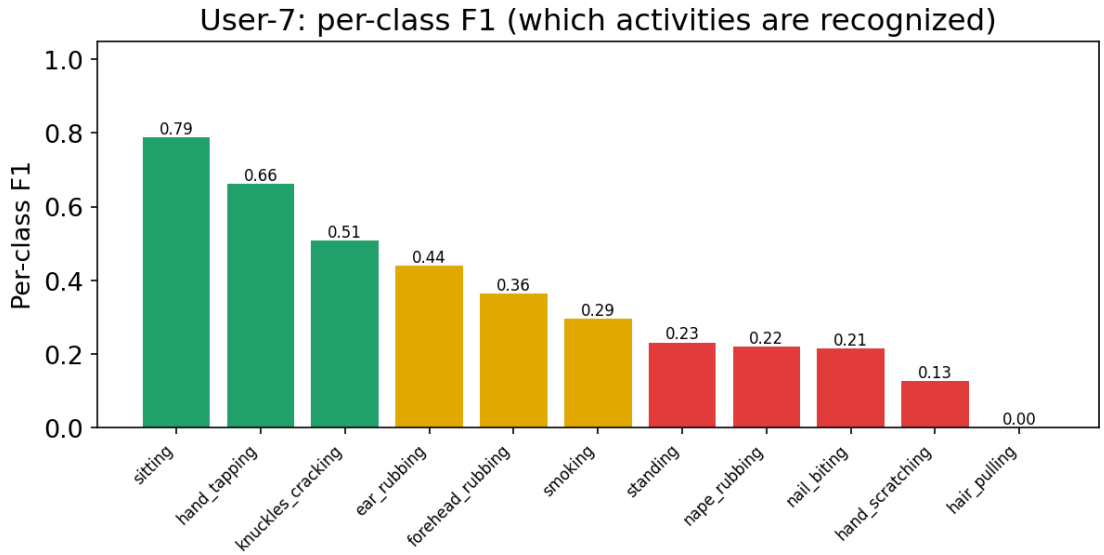


Figure D.2: Per-class F1 on the blind User 7 probe. Recognition is highly uneven across the eleven activities: a few activities such as hand tapping are recognized reliably while several fine-grained behaviors receive an F1 close to zero. The macro-averaged F1 of 0.350 summarizes this spread.

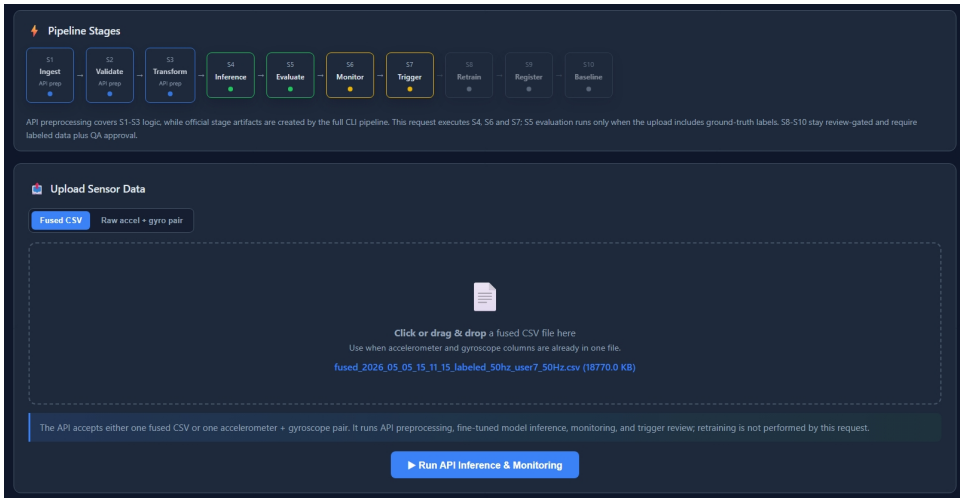
Appendix E

System Walkthrough: Web UI and Experiment Tracking

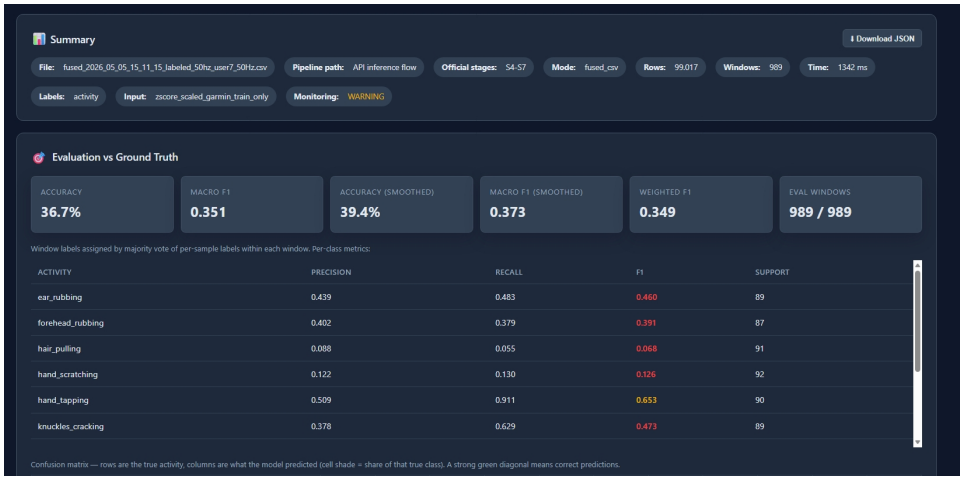
This appendix illustrates the running system referenced from Chapter 5. It walks through the serving path (the FastAPI dashboard) on two contrasting sessions and then through the two MLflow experiments that track the serving and batch paths. The two sessions show both ends of the label-optional design: a labeled session, where the dashboard additionally computes evaluation metrics, and an unlabeled session, where only the proxy monitoring signals are available. The screenshots confirm that the behavior described in Chapters 5–7 operates in the live system; they are presented as evidence of the implementation, not as a separate argument.

E.1 Serving Path on a Labeled Session

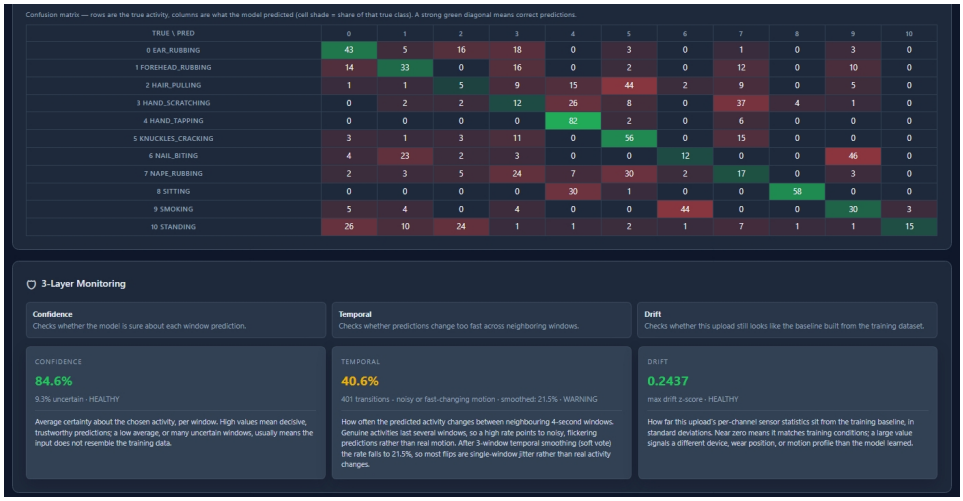
Figure E.1 walks through the serving path on the labeled User 7 session (fused upload mode), from upload to per-window predictions. Because the upload carries ground-truth labels, the dashboard computes an evaluation card (accuracy, macro-F1, and their smoothed variants) in addition to the monitoring signals, and the temporal signal routes the run to review. Key things to observe: the stage strip at the top of the upload screen (S1–S3 preprocessing, S4–S7 result path, S8–S10 review-gated) mirrors the ten-stage architecture of Chapter 5; the evaluation card appears only in this labeled mode and is absent in the unlabeled case below; and the governance decision is `REVIEW` because the raw temporal transition rate (40.6%) exceeds the monitoring threshold even though the smoothed rate (21.5%) is below it, which is the expected separation between the monitoring and serving paths described in Section 7.5.1.



(a) Upload screen with the stage strip (S1–S3 API preprocessing, S4–S7 visible result path, S8–S10 review-gated) and the fused-CSV upload mode.



(b) Run summary (API inference flow, stages S4–S7, fused mode, 989 windows) and the labeled evaluation card with accuracy, macro-F1, and smoothed variants.



(c) Three-layer monitoring (confidence 84.6%, temporal 40.6% with smoothed 21.5%, drift $z = 0.24$) and the per-class confusion matrix.

Figure E.1: Serving path on the labeled User 7 session (continued in Figure E.1). The evaluation card is computed only because the upload carried labels.



(d) Governance decision: routed to `REVIEW` by the temporal signal, with retraining blocked pending labels, a qualifying trigger, and human approval.

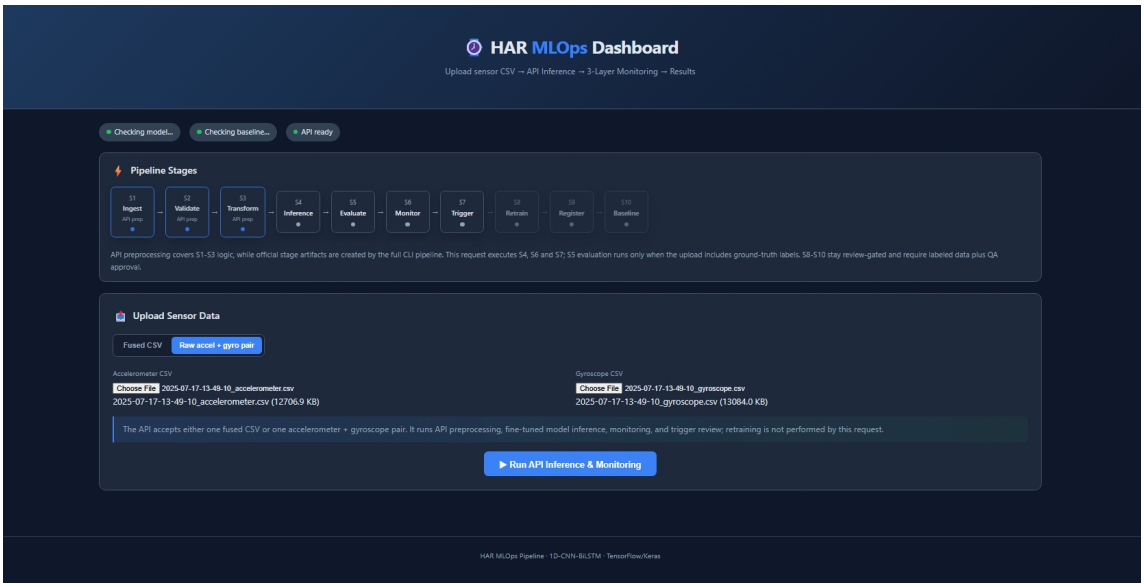


(e) Per-window predictions with confidence, and the footer explaining the windowing and three-layer monitoring contract.

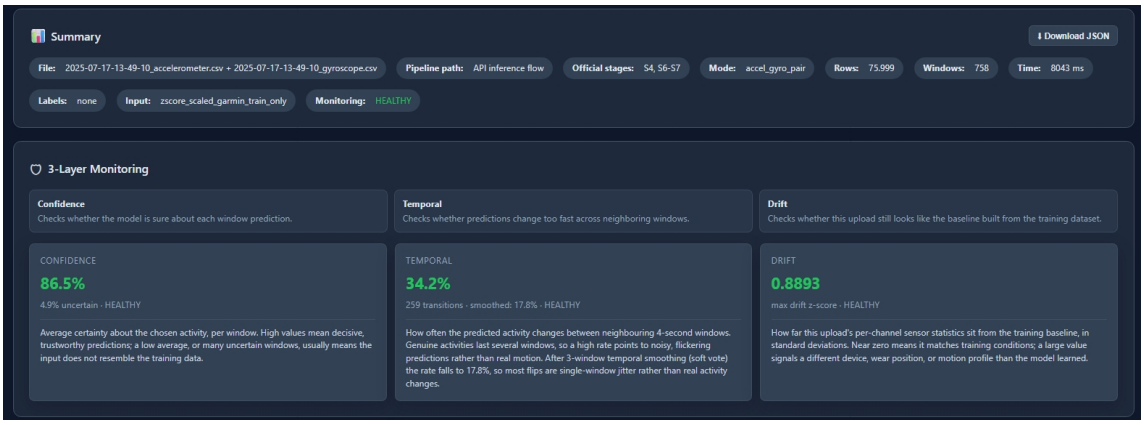
Figure E.1: Serving path on the labeled User 7 session (continued). The temporal signal routes the run to review while retraining stays blocked, the no-auto-retrain invariant of Chapter 6 operating on live data.

E.2 Serving Path on an Unlabeled Session

Figure E.2 shows the same serving path on an unlabeled session uploaded as a raw accelerometer-plus-gyroscope pair. No evaluation card is produced, because no ground truth exists; the pipeline relies entirely on the proxy monitoring signals. Here all three layers are healthy and the governance decision is `NO_ACTION`, the complementary outcome to the labeled session above and the label-free monitoring behavior on which Section 7.5.1 reports at scale. Comparing these two sessions side by side illustrates the label-optional design: the upload interface, preprocessing, inference, monitoring, and governance stages all execute identically; the only difference is that the labeled session additionally activates the evaluation card, and its elevated temporal rate triggers a review while the unlabeled session's lower rate does not.



(a) Upload screen in raw accelerometer-plus-gyroscope mode, with the two-file picker.

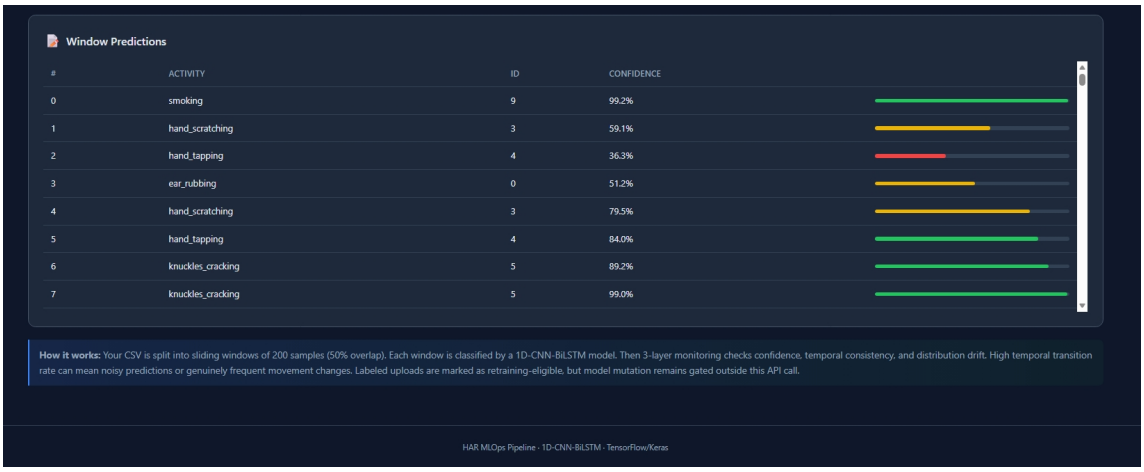


(b) Run summary (`Labels: none`, raw accel+gyro mode, 758 windows) and three-layer monitoring: confidence 86.5%, temporal 34.2% (smoothed 17.8%), drift $z = 0.89$, all healthy.

Figure E.2: Serving path on an unlabeled session (continued in Figure E.2). With no labels, the dashboard produces no evaluation card and depends only on the proxy monitoring signals.



(c) Governance decision: `NO_ACTION`, since all three monitoring layers are healthy; retraining remains blocked by the same gate.



(d) Per-window predictions with confidence for the unlabeled session.

Figure E.2: Serving path on an unlabeled session (continued). All layers are healthy and the trigger returns `NO_ACTION`, the complement to the labeled review case in Figure E.1.

E.3 Experiment Tracking

MLflow records the two execution paths of Section 5.6.1 as two separate experiments, shown in Figure E.3. The remainder of this section tours each experiment in turn, from its run list through a single run's overview, curated metrics and parameters, and logged artifacts. Keeping the serving and batch paths in separate experiments prevents interactive dashboard submissions from diluting the governed batch-run history; each experiment therefore provides a clean, queryable record of its own execution path and makes the audit trail described in Section 5.5.1 navigable without filtering.

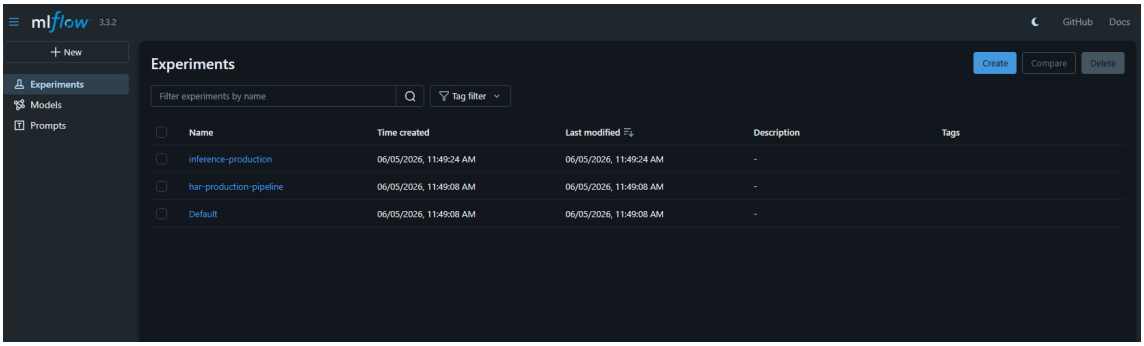
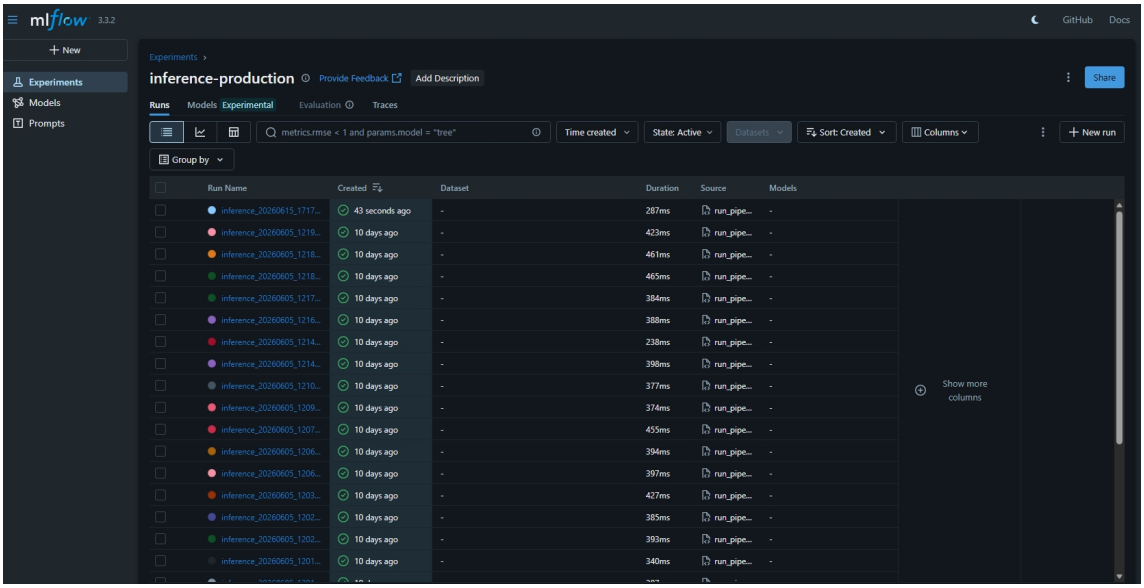


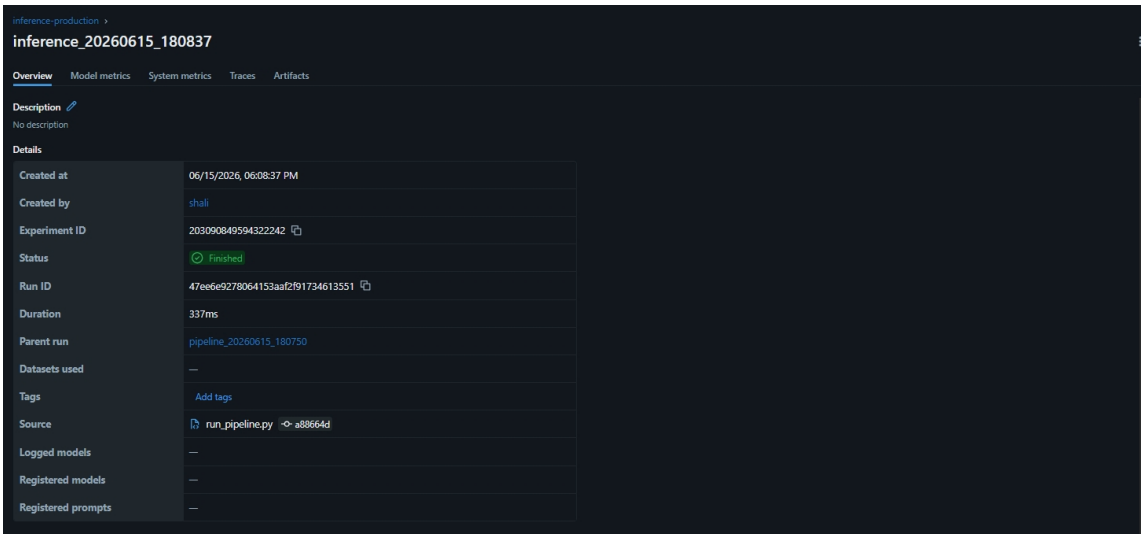
Figure E.3: The two execution paths recorded as separate MLflow experiments: `inference-production` (serving) and `har-production-pipeline` (batch). Keeping them separate prevents interactive serving runs from diluting the governed batch-run history.

E.3.1 Inference-Production Experiment (Serving Path)

The `inference-production` experiment captures runs from two sources: the Stage 4 batch inference logger (`src/components/model_inference.py`), which records the resolved contract (model MD5, scaler input space, runtime mode) and archives the inference report and prediction files; and the WebUI logger (`_log_inference_mlflow()` in `src/api/app.py`), which records the proxy monitoring metrics (mean confidence, per-activity counts, total windows) for dashboard-submitted sessions. Figure E.4 tours a single batch-path run, where both metric and parameter sets are present.

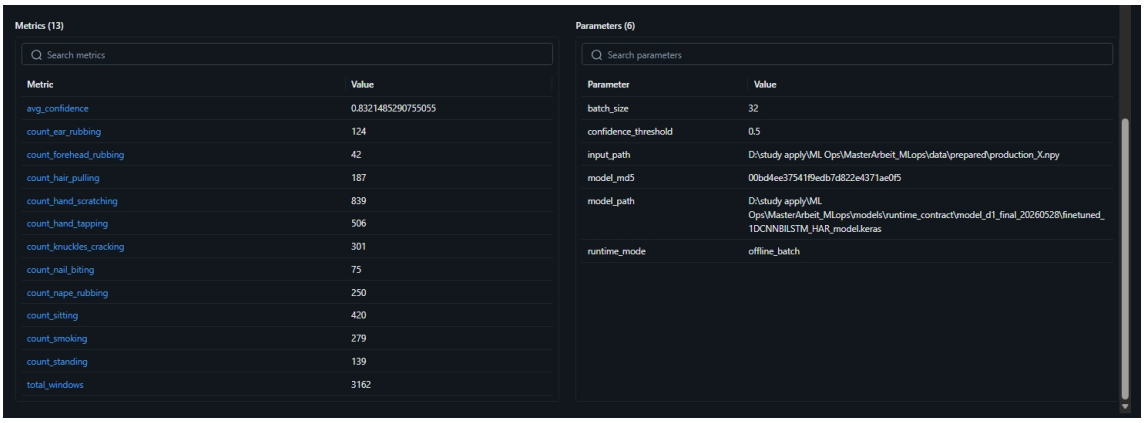


(a) Run list of the `inference-production` experiment (serving-path uploads).

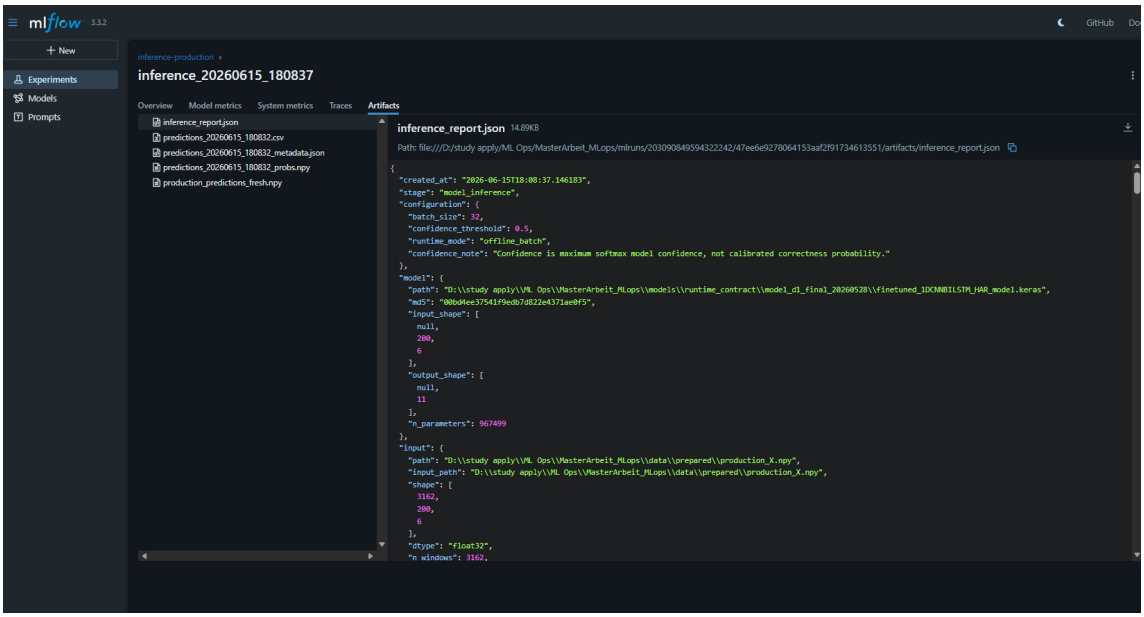


(b) Run overview: run identifier, finished status, source script and commit, and the parent-run link tying serving evidence back to its pipeline run.

Figure E.4: Inference-production experiment (continued in Figure E.4): run list and single-run provenance.



(c) Metrics tab: mean confidence, per-activity counts, and total windows (proxy set). Parameters tab: model MD5, scaler input space, and runtime mode (Stage 4 batch logger).

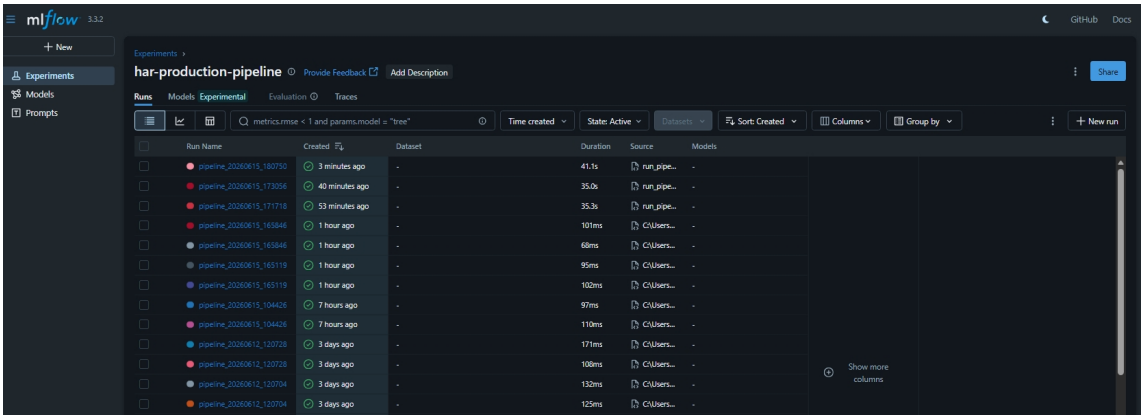


(d) Logged artifacts: the inference report and prediction files archived with the run.

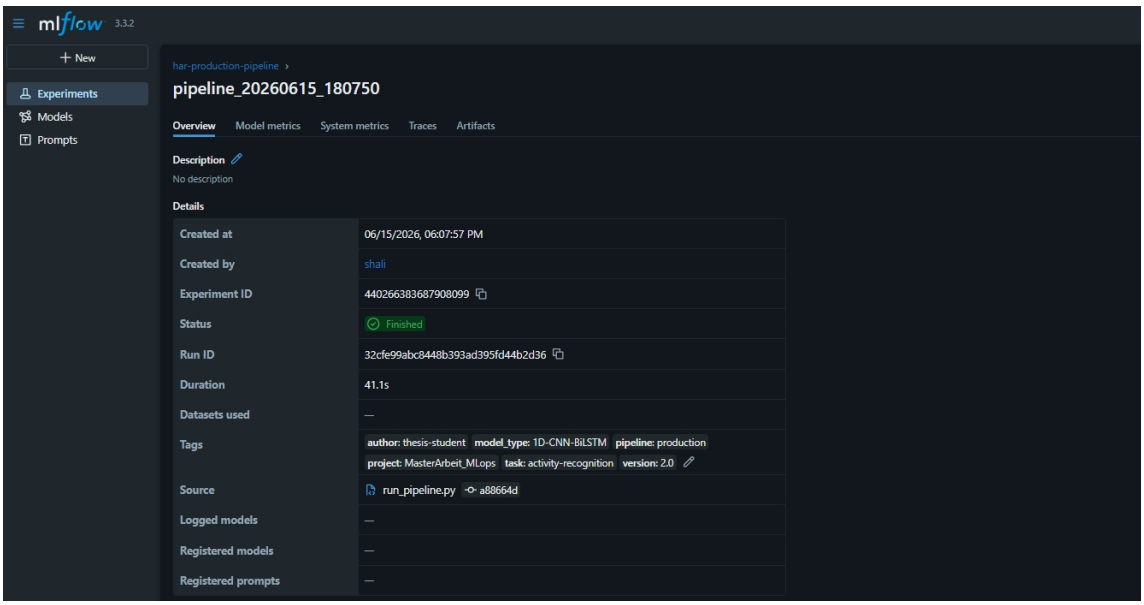
Figure E.4: Inference-production experiment (continued): curated metrics and parameters, and the archived artifacts.

E.3.2 Har-Production-Pipeline Experiment (Batch Path)

Figure E.5 tours a single batch run. The metrics tab (E.5c) shows the curated thirteen stage metrics of Section 5.6.1, one or two per stage, together with run parameters recording that all seven stages completed, the overall status was `SUCCESS`, and every retraining flag was `False`, the no-auto-retrain outcome captured as data. The artifacts tab exposes `run_info.json`, which records each stage's status and details and is the run-scoped artifact trail of Section 5.5.1.

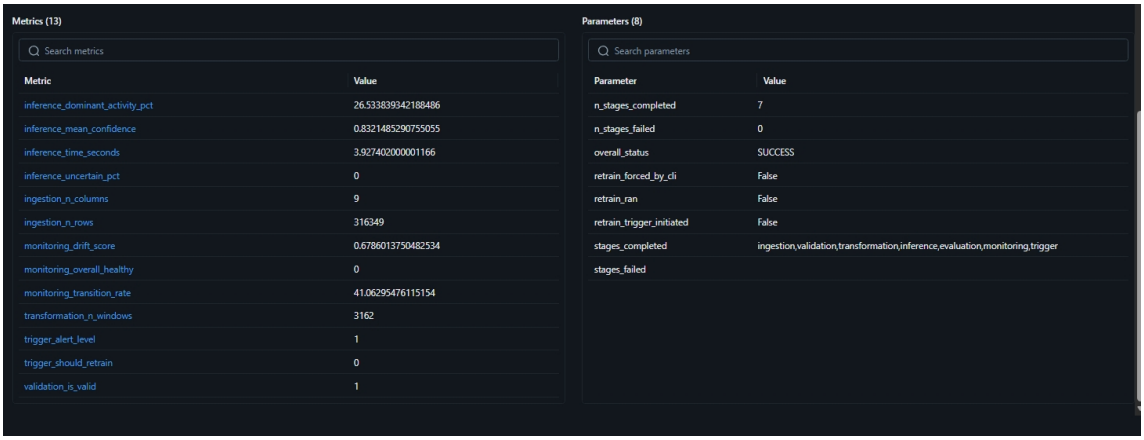


(a) Run list of the `har-production-pipeline` experiment (governed batch runs).

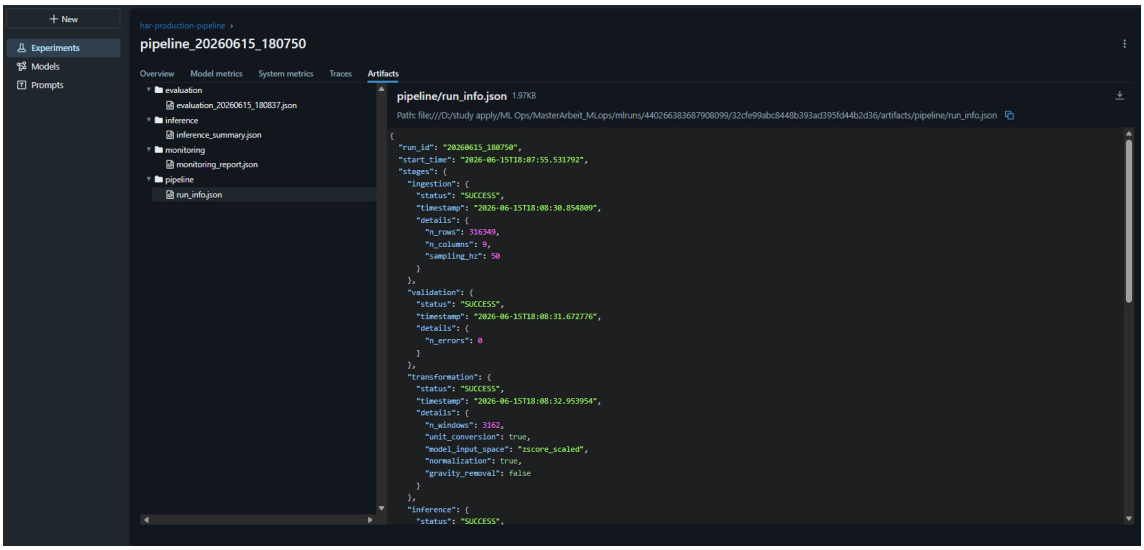


(b) Run overview with run-level tags (author, model type, task, version) and the source commit.

Figure E.5: Har-production-pipeline experiment (continued in Figure E.5): run list and tagged provenance.



(c) The thirteen curated stage metrics and run parameters: seven stages completed, `overall_status=SUCCESS`, all retrain flags `False`.



(d) `run_info.json`: per-stage status and details (rows, windows, unit conversion, model input space), the run-scoped artifact trail.

Figure E.5: Har-production-pipeline experiment (continued): the curated stage metrics and the run-scoped artifact trail.